SWIFT

CHEAT SHEETS

A Complete Reference for Modern Swift

Optionals · Strings · Control Flow · Closures · Collections · Structs & Classes · Enums · Protocols · Generics · Codable · async / await · Actors · ARC

108 REFERENCE SHEETS

CONTENTS

THE LANGUAGE

001 Introduction & Setup
006 Output & Comments
008 Variables & Data Types
016 Strings
022 Operators
029 Control Flow

TYPES & LIBRARIES

037 Error Handling
042 Functions & Closures
051 Data Structures
060 Types & OOP

ADVANCED

076 Files & I/O
081 Modules & Packages
087 Testing & Common Libraries
093 Advanced Topics
098 Concurrency
104 Memory Management
107 Quick Reference

Table of Contents

Introduction & History

WHAT SWIFT IS

Swift is Apple's general-purpose, compiled language for iOS, macOS, watchOS, tvOS and visionOS, and it also runs on Linux and Windows. It was built to replace Objective-C with something **safe by default** (no null pointers without ?, no unchecked overflow, no uninitialized variables) while staying as fast as C-family code. It is open source and evolves through public proposals.

NOTES

- Announced at WWDC 2014; open-sourced in December 2015
- Created by Chris Lattner and the Apple developer tools team (Lattner also started LLVM)
- File extension `.swift`; compiled through LLVM to native code
- Changes go through **Swift Evolution** proposals (SE-0001 …) at swift.org

HELLO, WORLD

```
// hello.swift
print("Hello, World!")            // Hello, World!

let name = "Swift"                // constant, type inferred as String
var year = 2014                   // variable, type inferred as Int
year += 12
print("\(name) is \(year - 2014) years old")   // Swift is 12 years old

// Run it:  swift hello.swift
// Compile: swiftc hello.swift -o hello && ./hello
```

CODE EXPLAINED

- → `print` — writes to standard output; no `main()` needed in a script
- → `let` — constant; `var` — variable
- → `\(…)` — string interpolation
- → `swift file.swift` — runs directly; `swiftc` — builds a binary

MILESTONES

VERSION	YEAR	HEADLINE
1.0	2014	First release with Xcode 6
2.2	2016	First open-source release, Linux
5.0	2019	ABI stability on Apple platforms
5.5	2021	async / await, actors
5.9	2023	Macros, if/switch expressions
6.0	2024	Swift 6 mode, data-race safety

THE SWIFTY WAY

✗ Starting new code in Swift 5 language mode
✓ Swift 6 language mode (strict concurrency on) — new projects should opt into data-race checking from day one

WHERE SWIFT RUNS

PLATFORM	NOTES
iOS / iPadOS	Primary target, with SwiftUI or UIKit
macOS	Apps, CLIs, scripts
watchOS / tvOS	Same language, smaller SDKs
visionOS	Apple Vision Pro apps
Linux / Windows	Servers, CLIs, cross-platform libraries

COMING FROM…

- **Kotlin** Closest cousin: modern, null-safe, same era
- **TypeScript** Like TS with a real compiler and value types
- **Java** Compiled AOT to native, no JVM, no GC (uses ARC)
- **Python** Same readability, but static types and much faster

⚠ COMMON MISTAKES

- Treating Swift as "just for iPhone": the language, package manager and stdlib work on Linux and Windows too
- Learning from pre-Swift 5 tutorials: syntax and APIs changed a lot (`String` indexing, Codable, concurrency)
- Assuming ARC is garbage collection: there is no collector, and reference cycles leak

✓ TIPS & BEST PRACTICES

- Bookmark **docs.swift.org** (The Swift Programming Language book) and **developer.apple.com/documentation**
- Check a feature's version in its Swift Evolution proposal before using it
- Use a Playground or `swift repl` to try snippets from this set

QUICK SUMMARY

- `Swift` — Apple's compiled, safe-by-default language (2014)
- `swift.org` — open source home, Evolution proposals
- `Swift 6` — current major version, data-race safety

Xcode & Toolchain

THE TOOLCHAIN

On a Mac, **Xcode** is the toolchain: it bundles the Swift compiler, the SDKs for every Apple platform, simulators, Instruments and the debugger. The **Command Line Tools** package gives you `swift` and `swiftc` without the full IDE. On Linux, or to manage several toolchain versions, use **swiftly**.

NOTES

- Install Xcode from the Mac App Store or developer.apple.com
- Each Xcode release pins one Swift version; check with `swift --version`
- `xcode-select` picks which Xcode the command line uses
- Simulators are downloaded per platform from Xcode › Settings › Components

SETUP COMMANDS

```
# Command Line Tools only (no IDE)
xcode-select --install

# Which Swift / which Xcode?
swift --version
xcode-select -p                 # /Applications/Xcode.app/Contents/Developer
sudo xcode-select -s /Applications/Xcode-beta.app

# swiftly: install and switch toolchains (macOS + Linux)
swiftly install latest
swiftly use 6.0
swiftly list

# List simulators and boot one
xcrun simctl list devices
xcrun simctl boot "iPhone 16"
```

CODE EXPLAINED

- → `xcode-select --install` — compiler + SDK without Xcode
- → `xcode-select -s` — switch the active Xcode
- → `swiftly` — official toolchain manager
- → `xcrun` — runs a tool from the active Xcode

TOOLCHAIN PIECES

TOOL	WHAT IT DOES
`swiftc`	Compiler: .swift → binary
`swift`	Driver: run, build, test, repl, package
`lldb`	Debugger (po, p, breakpoints)
`sourcekit-lsp`	Language server for VS Code etc.
`xcodebuild`	Build Xcode projects from the CLI
`simctl`	Control iOS simulators

THE SWIFTY WAY

✗ Downloading toolchain tarballs by hand
✓ swiftly install latest — swiftly handles paths and switching

EDITOR OPTIONS

EDITOR	NOTES
Xcode	Required for Apple app builds and signing
VS Code	Official Swift extension (sourcekit-lsp)
Cursor / Zed	Use sourcekit-lsp
Neovim	LSP via sourcekit-lsp

COMING FROM…

- **Kotlin** Xcode ≈ Android Studio + SDK Manager
- **TypeScript** swiftly ≈ nvm for Node versions
- **Python** swiftly ≈ pyenv

⚠ COMMON MISTAKES

- Running `swift` from an old Xcode: `xcode-select -p` shows which one is active
- Mixing Homebrew Swift with Xcode's Swift: pick one toolchain per project
- Expecting VS Code to build iOS apps: it edits and builds packages, but signing and simulators still need Xcode

✓ TIPS & BEST PRACTICES

- Keep the release Xcode and a beta side by side; switch with `xcode-select -s`
- `xcrun --find swiftc` shows the exact compiler path in use
- Use `xcodebuild -scheme App test` in CI

QUICK SUMMARY

- `Xcode` — IDE + compiler + SDKs + simulators
- `xcode-select` — choose the active toolchain
- `swiftly` — install and switch Swift versions

Running Swift

WAYS TO RUN CODE

Swift can be run four ways: interactively in a **Playground** or the **REPL**, as a **script** interpreted on the fly, as a **Swift package** built with `swift run`, or as an app built by Xcode. Scripts and the REPL are great for learning; packages are how real command-line tools and libraries are built.

NOTES

- Playgrounds show each line's result in a sidebar
- `swift repl` (or just `swift`) starts an interactive session
- A script can start with a shebang and be made executable
- `#Preview` in Xcode is the app-world equivalent for UI

RUNNING SWIFT

```
# Interactive REPL
$ swift repl
  1> let x = 21 * 2
x: Int = 42
  2> :quit

# Run a single file (compiled in memory, then run)
$ swift greet.swift Ada

# Executable script
#!/usr/bin/env swift
print("Hi, \(CommandLine.arguments[1])")
$ chmod +x greet.swift && ./greet.swift Ada

# Package: create, build, run
$ swift package init --type executable --name greet
$ swift run greet Ada
```

CODE EXPLAINED

- → `swift repl` — read-eval-print loop; `:quit` exits
- → `swift file.swift` — interpret a single file
- → `CommandLine.arguments` — args; `[0]` is the program
- → `swift run` — build and run a package's executable

THE SWIFTY WAY

✗ Growing a 300-line script file
✓ swift package init --type executable — packages give you modules, tests and dependencies

COMING FROM…

- **Kotlin** REPL ≈ kotlinc; Playground ≈ Kotlin scratch file
- **TypeScript** swift file.swift ≈ npx tsx file.ts
- **Python** swift repl ≈ python; scripts work the same way

RUN MODES

MODE	BEST FOR
Playground	Learning, visual results
swift repl	Quick experiments
swift file.swift	One-file scripts
swift run	CLI tools in a package
Xcode ▶ Run	Apps on simulator or device

REPL COMMANDS

COMMAND	EFFECT
:help	List commands
:quit	Exit
:type expr	Show the type of an expression
:print x	Print a variable

⚠ COMMON MISTAKES

- Forgetting `chmod +x` on a shebang script
- Reading `CommandLine.arguments[1]` without checking the count (traps if missing)
- Expecting a Playground to run in the iOS app lifecycle; use a real app target or `#Preview` instead

✓ TIPS & BEST PRACTICES

- `swift -e 'print(1+1)'` runs a one-liner
- Add `import Foundation` at the top of scripts that need dates, files or networking
- Use Swift Playgrounds on iPad to learn and prototype away from the Mac

QUICK SUMMARY

- `swift repl` — interactive
- `swift file.swift` — run a script
- `swift run` — run a package's executable

Program Structure

HOW A PROGRAM IS LAID OUT

A Swift file is a list of declarations and statements. In a script or a file named `main.swift`, top-level statements run in order: that file **is** the entry point. Everywhere else (apps and libraries), execution starts at the type marked `@main`. Statements end at the line break, and blocks always use braces.

NOTES

- Only `main.swift` (or a script) may contain top-level executable code
- `@main` marks the entry type in apps and CLIs with multiple files
- Semicolons are optional; only needed for two statements on one line
- No header files: every file in a module sees every other file

ENTRY POINTS

```swift
// ── main.swift (top-level code runs in order) ──
import Foundation                    // bring in a module

let args = CommandLine.arguments
print("Got \(args.count - 1) args")

// ── Tool.swift (multi-file CLI using @main) ──
@main
struct Tool {
    static func main() async throws {   // may be async/throws
        print("Started at \(Date.now)")
    }
}

// ── SwiftUI app (also @main) ──
@main
struct MyApp: App {
    var body: some Scene { WindowGroup { Text("Hi") } }
}
```

CODE EXPLAINED

- → `import` — load a module (Foundation, SwiftUI, a package)
- → `main.swift` — the one file allowed to run top-level code
- → `@main` — type whose `static func main()` starts the program
- → `async throws` — entry point can await and throw

THE SWIFTY WAY

✗ main.swift + @main in the same target
✓ pick one entry point per target — having both is a compile error

FILE RULES

RULE	DETAIL
Module	All files in a target share one namespace
Order	Declarations can appear in any order
main.swift	Top-level code allowed only here
@main	Exactly one per executable target

COMING FROM…

- **Kotlin** fun main() ≈ @main struct with static main
- **Java** public static void main ≈ static func main()
- **Python** main.swift ≈ a module run as __main__

STATEMENT SYNTAX

SYNTAX	MEANING
{ }	Blocks: always required for if / for / func
;	Optional; separates statements on one line
// /* */	Comments; block comments nest
_	Wildcard: ignore a value

⚠ COMMON MISTAKES

- Putting statements at top level in a non-main file: "expressions are not allowed at the top level"
- Naming a file `main.swift` in an app that also has `@main`
- Leaving out braces for one-line `if` bodies: Swift always requires them

✓ TIPS & BEST PRACTICES

- Use `@main` with `static func main() async throws` for CLIs that need concurrency
- Group related types in one file, but let each major type have its own
- Add the `ArgumentParser` package for real CLI flags (`@main struct Cmd: ParsableCommand`)

QUICK SUMMARY

- `main.swift` — top-level code entry
- `@main` — entry type for apps and multi-file CLIs
- `import` — use another module

Swift Beyond Apple

SWIFT OFF APPLE PLATFORMS

The same compiler and standard library run on **Linux** and **Windows**, so Swift is used for servers, CLIs and cross-platform libraries. Foundation is available everywhere through **swift-foundation** and swift-corelibs-foundation, but Apple-only frameworks (SwiftUI, UIKit, Combine) are not. Newer targets include **Embedded Swift** for microcontrollers and **WebAssembly**.

NOTES

- Official Linux toolchains for Ubuntu, Debian, Fedora, Amazon Linux and others
- Windows has been officially supported since Swift 5.3
- Server frameworks: **Vapor** and **Hummingbird**, both on SwiftNIO
- Docker images: `swift:latest`, `swift:6.0-noble`

LINUX / SERVER BASICS

```
# Docker: build and test a package on Linux
docker run --rm -v "$PWD":/app -w /app swift:latest swift test

# Guard Apple-only code with #if
#if canImport(UIKit)
import UIKit
#elseif os(Linux)
import Glibc
#endif

// Minimal Vapor route (Package dep: vapor/vapor)
import Vapor
let app = try await Application.make(.detect())
app.get("hello") { _ in "Hello from Linux" }
try await app.execute()
```

CODE EXPLAINED

→ `swift:latest` — official Docker image

→ `#if canImport(X)` — compile only where a module exists

→ `#if os(Linux)` — platform condition

→ `app.get` — Vapor route returning a String

THE SWIFTY WAY

✗ #if os(iOS) to test for UIKit

✓ #if canImport(UIKit) — checks for the module itself, not a guessed OS

TARGETS

TARGET	STATUS / USE
`Linux`	Servers, CI, CLIs
`Windows`	Toolchain via swift.org installer
`Embedded Swift`	Microcontrollers (subset of the language)
`WebAssembly`	Swift SDKs for Wasm
`Android`	Community and workgroup SDKs

COMING FROM…

- **`Kotlin`** Like Kotlin/JVM on servers plus Kotlin/Native
- **`TypeScript`** Vapor ≈ Express / Fastify
- **`Python`** Hummingbird ≈ FastAPI; small and async

CONDITION CHECKS

CONDITION	TRUE WHEN
`os(macOS)`	Building for macOS
`os(Linux)`	Building for Linux
`canImport(M)`	Module M is available
`arch(arm64)`	Apple silicon / ARM
`swift(>=6.0)`	Compiler is 6.0 or newer

⚠ COMMON MISTAKES

- Importing SwiftUI or Combine in a package meant for Linux: the build fails there
- Assuming every Foundation API exists on Linux; test there in CI
- Using `os(iOS)` checks when you mean module availability (`canImport`)

✓ TIPS & BEST PRACTICES

- Run `swift test` in a `swift:latest` container in CI to catch Linux-only failures
- Keep business logic in a platform-neutral package; keep UI in the app target
- Swift Package Index (swiftpackageindex.com) shows which platforms each package supports

QUICK SUMMARY

- `Linux / Windows` — full compiler + stdlib + Foundation
- `Vapor / Hummingbird` — server-side Swift
- `#if canImport` — write portable code

Print & Debug Output

PRINTING OUTPUT

`print` writes values to standard output, separated by spaces and ending with a newline by default. `debugPrint` shows the debug form (strings get quotes), and `dump` prints a whole object tree. A type controls how it prints by conforming to `CustomStringConvertible`.

NOTES

- `print` takes any number of values (`Any...`)
- `separator:` and `terminator:` change the joiners
- `print(_:to:)` writes into any `TextOutputStream`, including a String
- In apps, prefer `Logger` (OSLog) over print for real logging

PRINT · DEBUGPRINT · DUMP

```
print("Hello")                          // Hello
print(1, 2, 3)                          // 1 2 3
print(1, 2, 3, separator: ", ")         // 1, 2, 3
print("no newline", terminator: "")     // no newline
print([1, 2], ["a": 1])                 // [1, 2] ["a": 1]

debugPrint("hi", 1.5)                   // "hi" 1.5
let name: String? = "Ada"
print(name as Any)                      // Optional("Ada")

struct Point: CustomStringConvertible {
    var x = 0, y = 0
    var description: String { "(\(x), \(y))" }
}
print(Point(x: 3, y: 4))                // (3, 4)
dump(Point())                           // ▿ (0, 0) - x: 0 - y: 0

var log = ""
print("saved", to: &log)                // log == "saved\n"
```

CODE EXPLAINED

- → `separator:` — between values (default " ")
- → `terminator:` — after the last value (default "\n")
- → `debugPrint` — quotes strings, shows debug form
- → `description` — what `print` shows for your type
- → `dump` — recursive tree of all properties

THE SWIFTY WAY

✗ `print("\(a) \(b)")`
✓ `print(a, b)`
✗ `print(optional!)`
✓ `print(optional ?? "none")` — no crash just to print

OUTPUT FUNCTIONS

FUNCTION	USE
`print(_:)`	Normal output
`debugPrint(_:)`	Debug representation
`dump(_:)`	Full object tree
`FileHandle.standardError`	Write to stderr

PRINTING PROTOCOLS

PROTOCOL	PROPERTY
`CustomStringConvertible`	description
`CustomDebugStringConvertible`	debugDescription
`CustomReflectable`	customMirror (for dump)

COMING FROM…

- **Kotlin** println(x) ≈ print(x)
- **TypeScript** console.log(a, b) ≈ print(a, b)
- **Python** print(a, sep=", ", end="") ≈ separator:/terminator:

⚠ COMMON MISTAKES

- Printing an optional directly: you get `Optional("x")` and a compiler warning; unwrap or use `??`
- Leaving `print` calls in shipping app code: they cost time and leak data to the device console
- Using `print` for errors in CLIs: write to stderr so pipes stay clean

✓ TIPS & BEST PRACTICES

- Send errors to stderr with `FileHandle.standardError.write(_:)`
- In LLDB, `po value` uses `debugDescription`
- Conform types to `CustomStringConvertible` to make logs readable

QUICK SUMMARY

- `print(a, b, separator:, terminator:)` — stdout
- `debugPrint` / `dump` — debugging forms
- `CustomStringConvertible` — custom text for your types

Sheet 007

Comments & Documentation

// · /* */ nesting · /// · - Parameters: · // MARK: · DocC

COMMENTS IN SWIFT

Swift has // line comments and /* */ block comments, and **block comments nest**, so you can comment out code that already has comments in it. Doc comments use /// (or /** */) with Markdown; Xcode shows them in Quick Help, and **DocC** builds them into a documentation site.

NOTES

- Doc comments go directly above the declaration
- Option-click a symbol in Xcode to see its doc comment
- `// MARK: -` adds a divider in Xcode's jump bar
- `#warning("…")` and `#error("…")` show up as build diagnostics

COMMENT SYNTAX

```swift
// Single-line comment
/* Block comment
   /* nested comments are fine */ */

/// Returns the area of a rectangle.
///
/// - Parameters:
///   - width: Width in points.
///   - height: Height in points.
/// - Returns: `width * height`.
/// - Throws: `GeometryError.negative` if a side is < 0.
func area(width: Double, height: Double) throws -> Double {
    width * height
}

// MARK: - Networking
// TODO: retry on timeout
// FIXME: handle 401
#warning("Remove before release")
```

CODE EXPLAINED

→ `///` — doc comment, Markdown supported

→ `- Parameters:` / `- Returns:` / `- Throws:` — Quick Help sections

→ `// MARK: -` — named section with a divider line

→ `TODO:` / `FIXME:` — listed in the jump bar

→ `#warning` — compiler warning you can't miss

THE SWIFTY WAY

✗ // This function adds two numbers
✓ /// Adds `a` and `b`, wrapping on overflow. — doc comments show in autocomplete; say what isn't obvious

COMING FROM…

- **Kotlin** KDoc /** */ ≈ /// with - Parameters
- **Java** Javadoc @param ≈ - Parameter name:
- **Python** docstrings ≈ /// above the func

DOC MARKUP

KEYWORD	SHOWS AS
`- Parameter x:`	Single parameter
`- Parameters:`	Parameter list
`- Returns:`	Return value
`- Throws:`	Errors thrown
`- Note: / - Warning:`	Callout boxes
``` ``Symbol`` ```	DocC link to a symbol

DOCC COMMANDS

COMMAND	EFFECT
`Product › Build Documentation`	Build docs in Xcode
`swift package generate-documentation`	CLI (swift-docc-plugin)
`Documentation.docc`	Catalog for articles and tutorials

⚠ COMMON MISTAKES

- Using `//` above a public API: only `///` appears in Quick Help
- Commenting what the code already says instead of why it does it
- Leaving a `#warning` in a target that builds with warnings-as-errors

✓ TIPS & BEST PRACTICES

- Type `///` above a function in Xcode to generate a doc template (or press ⌥⌘/)
- Use `// MARK: - Protocol Name` above each protocol conformance extension
- Write `- Complexity: O(n)` for performance-sensitive APIs

QUICK SUMMARY

- `//` and `/* */` — comments (block comments nest)
- `///` — doc comments shown in Quick Help and DocC
- `// MARK:` — organize the jump bar

Constants & Variables

LET VS VAR

`let` declares a **constant** that is set once; `var` declares a **variable** that can change. Swift infers the type from the initial value, or you can write it with a type annotation. Every value must be initialized before it is read: the compiler checks this for you.

NOTES

- Default to `let`; the compiler warns when a `var` is never mutated
- A `let` can be declared first and assigned once later
- Names are camelCase; types are UpperCamelCase
- Unicode names are legal but rarely a good idea

DECLARING VALUES

```
let maxUsers = 100                  // constant, Int
var score = 0                       // variable, Int
score += 10                         // 10

let pi: Double = 3.14159            // explicit type
var name: String                    // declared, not set
name = "Ada"                        // set before first use

let label: String                   // deferred constant
if score > 5 { label = "high" } else { label = "low" }

var x = 1, y = 2, z = 3             // several on one line
let (w, h) = (1920, 1080)           // tuple destructuring
let `class` = "ok"                  // backticks: keyword name

let total = 5                       // shadowing in inner scope
do { let total = "five"; print(total) }   // five
```

CODE EXPLAINED

→ `let` — assign once; `var` — reassignable

→ `: Double` — type annotation

→ `let label: String` — assigned once on every path

→ `let (w, h)` — destructure a tuple

→ `` `class` `` — backticks allow a keyword as a name

NAMING RULES

KIND	STYLE
`maxRetryCount`	Variables, functions, cases
`UserProfile`	Types and protocols
`isEnabled / hasItems`	Booleans read as assertions
`URLSession / userID`	Acronyms keep one case

THE SWIFTY WAY

✗ `var` for values that never change

✓ `let` everywhere, `var` only when needed — the compiler warns about unmutated vars

LET ON A STRUCT VS CLASS

DECLARATION	CAN MUTATE PROPERTIES?
`let s = MyStruct()`	No (whole value is constant)
`var s = MyStruct()`	Yes
`let c = MyClass()`	Yes (only the reference is constant)

COMING FROM…

- **Kotlin** `val` / `var` ≈ `let` / `var`
- **TypeScript** `const` / `let` ≈ `let` / `var`
- **Java** `final int x` ≈ `let x: Int`

⚠ COMMON MISTAKES

- Expecting `let` on a class instance to freeze it: only the reference is constant
- Reading a declared-but-unassigned `var`: compile error "used before being initialized"
- Shadowing an outer name by accident inside an `if` or loop

✓ TIPS & BEST PRACTICES

- Let the compiler infer obvious types; annotate when it helps a reader
- Use deferred `let` with `if` / `switch` instead of a `var` set in branches
- Turn on "Treat Warnings as Errors" to keep `var` → `let` warnings from piling up

QUICK SUMMARY

- `let` — constant, preferred
- `var` — variable
- `name: Type = value` — annotation + initial value

Integers

INTEGER TYPES

`Int` is the default integer and is 64 bits on all current Apple and Linux platforms. Fixed-size types (`Int8` … `Int64`, `UInt8` … `UInt64`) exist for interop and binary formats. Arithmetic that overflows **traps at runtime** instead of wrapping silently; the &+, &−, &∗ operators opt into wrapping.

LITERALS & LIMITS

```
let a = 42                      // Int
let big = 1_000_000             // 1000000
let hex = 0xFF                  // 255
let bin = 0b1010                // 10
let oct = 0o17                  // 15

Int.max                         // 9223372036854775807
Int8.min                        // -128
UInt8.max                       // 255

7 / 2                           // 3   (truncates toward zero)
-7 / 2                          // -3
7 % 3                           // 1
(-7) % 3                        // -1  (sign follows dividend)
10.isMultiple(of: 5)            // true
Int.max &+ 1                    // Int.min (wraps, no trap)
Int("42")                       // Optional(42)
```

NOTES

- Use `Int` unless you need a specific size
- `UInt` is not a way to say "never negative": it complicates arithmetic
- Underscores are allowed in literals: `1_000_000`
- Mixing `Int` and `Int32` needs an explicit conversion

CODE EXPLAINED

- → `0x` `0b` `0o` — hex, binary, octal literals
- → `.max` / `.min` — limits of each type
- → `/` — integer division truncates
- → `&+` — wrapping add; plain `+` traps on overflow
- → `Int("42")` — parsing returns an optional

THE SWIFTY WAY

- ✗ `n % 2 == 0`
- ✓ `n.isMultiple(of: 2)` — reads as intent
- ✗ `UInt` for counts
- ✓ `Int` (what collections use)

INTEGER TYPES

TYPE	RANGE
`Int` / `Int64`	±9.2 × 10^{18}
`Int32`	±2.1 × 10^{9}
`Int16`	-32,768 … 32,767
`Int8`	-128 … 127
`UInt8`	0 … 255 (bytes)

COMING FROM…

- **Kotlin** Overflow wraps silently; Swift traps
- **Java** `int` is 32-bit; Swift `Int` is 64-bit
- **Python** Unbounded ints; Swift's are fixed-size

USEFUL MEMBERS

MEMBER	RESULT
`abs(-5)`	5
`n.signum()`	-1, 0 or 1
`n.quotientAndRemainder(dividingBy:)`	(q, r) tuple
`n.nonzeroBitCount`	Set bits

⚠ COMMON MISTAKES

- Assuming overflow wraps: `Int.max + 1` crashes the app
- Dividing two Ints and expecting a fraction: convert to `Double` first
- Comparing `Int` with `Int32` without converting: compile error

✓ TIPS & BEST PRACTICES

- Use `addingReportingOverflow(_:)` when you need to detect overflow without crashing
- Use `UInt8` arrays (`[UInt8]`) or `Data` for raw bytes
- Use `Int.random(in: 1...6)` for random integers

QUICK SUMMARY

- `Int` — default, 64-bit
- `+ − ∗` — trap on overflow; `&+` `&− &∗` wrap
- `/` and `%` — truncate toward zero

Floating-Point & Decimal

FLOATING-POINT NUMBERS

`Double` (64-bit) is the default for decimal literals; `Float` (32-bit) is used mainly for graphics and ML APIs. Both follow IEEE 754, so `0.1 + 0.2` is not exactly `0.3`, and special values `.infinity` and `.nan` exist. For money, use Foundation's `Decimal`, which stores base-10 digits exactly.

NOTES

- `3.0` is a `Double`; write `Float(3.0)` or annotate for Float
- Dividing by zero gives `±.infinity`, not a crash
- `.nan != .nan` is `true`; test with `.isNaN`
- `rounded()` takes a `FloatingPointRoundingRule`

DOUBLES IN PRACTICE

```
let d = 3.75                          // Double
let f: Float = 3.75                   // Float
let e = 1.5e3                         // 1500.0

0.1 + 0.2 == 0.3                      // false
abs((0.1 + 0.2) - 0.3) < 1e-9         // true

d.rounded()                           // 4.0
d.rounded(.down)                      // 3.0
d.rounded(.towardZero)                // 3.0
(2.5).rounded(.toNearestOrEven)       // 2.0
Int(d)                                // 3 (truncates)

1.0 / 0                               // inf
(0.0 / 0).isNaN                       // true
7.5.truncatingRemainder(dividingBy: 2)   // 1.5
Double.pi                             // 3.141592653589793
```

CODE EXPLAINED

- → `1.5e3` — exponent notation
- → `rounded(_:)` — `.up` `.down` `.towardZero` `.toNearestOrEven`
- → `Int(d)` — conversion truncates the fraction
- → `truncatingRemainder` — `%` doesn't work on Double

THE SWIFTY WAY

✗ `price: Double` for currency
✓ `price: Decimal` — Decimal keeps exact cents

TYPES

TYPE	PRECISION
`Double`	~15–17 digits (default)
`Float`	~6–9 digits
`CGFloat`	Double on 64-bit (UI geometry)
`Decimal`	38 base-10 digits (Foundation)

COMING FROM…

- **Kotlin** Same IEEE types; `x % y` works on Double there
- **TypeScript** `number` ≈ `Double`; no separate Float
- **Python** `float` ≈ `Double`; `Decimal` ≈ `Decimal`

SPECIAL VALUES

EXPRESSION	MEANING
`.infinity`	Positive infinity
`.nan`	Not a number
`.ulpOfOne`	Machine epsilon
`.greatestFiniteMagnitude`	Largest finite value

⚠ COMMON MISTAKES

- Comparing floats with `==`: use a tolerance
- Using `%` on a `Double`: compile error; use `truncatingRemainder(dividingBy:)`
- Storing money in `Double`: rounding errors add up

✓ TIPS & BEST PRACTICES

- Format for display with `.formatted(.number.precision(…))` (Sheet 017)
- Use `Decimal(string: "19.99")` rather than `Decimal(19.99)` to avoid binary error
- Swift Numerics adds `isApproximatelyEqual(to:)`

QUICK SUMMARY

- `Double` — default floating type
- `rounded(_:)` — explicit rounding rules
- `Decimal` — exact base-10 for money

Booleans

THE BOOL TYPE

`Bool` has exactly two values, `true` and `false`. Swift has **no truthiness**: an `Int`, optional or string cannot stand in for a condition, so you always write an explicit comparison. This removes a whole class of bugs from C-family languages.

NOTES

- Conditions in `if`, `while` and `guard` must be `Bool`
- `toggle()` flips a `var` in place
- Name Bools as assertions: `isLoading`, `hasError`, `canSubmit`
- `Bool.random()` returns a coin flip

USING BOOLS

```
var isOn = false
isOn.toggle()                       // true
let canVote = age >= 18             // comparison → Bool

let count = 3
// if count { }                     // error: Int is not Bool
if count != 0 { print("has items") }

let name: String? = nil
// if name { }                      // error
if name != nil { }                  // explicit check
if let name { print(name) }         // unwrap instead

let items: [Int] = []
items.isEmpty                       // true
!items.isEmpty                      // false
Bool("true")                        // Optional(true)
Bool.random()                       // true or false
```

CODE EXPLAINED

- → `toggle()` — flips the value in place
- → `if count != 0` — explicit, no truthiness
- → `isEmpty` — idiomatic emptiness check
- → `Bool("true")` — parses only `"true"` / `"false"`

THE SWIFTY WAY

✗ `if items.count == 0`
✓ `if items.isEmpty` — O(1) on all collections
✗ `isOn = !isOn`
✓ `isOn.toggle()`

COMING FROM…

- **TypeScript** No truthy `""`, `0` or `null`
- **Python** `if items:` must be `if !items.isEmpty`
- **Kotlin** Same: conditions must be Boolean

BOOL OPERATIONS

EXPRESSION	RESULT
`true && false`	false
`true || false`	true
`!true`	false
`true == false`	false
`true != false`	true (acts as XOR)

⚠ COMMON MISTAKES

- Writing `if x` for an Int or optional: Swift has no truthiness
- Expecting `Bool("yes")` or `Bool("1")` to parse: both return `nil`
- Naming a Bool `status` or `flag`: callers can't tell what `true` means

✓ TIPS & BEST PRACTICES

- Prefer positive names (`isVisible`) over negative ones (`isNotHidden`)
- Use an enum instead of two Bools that can't both be true
- `allSatisfy(_:)` and `contains(where:)` compute Bools over collections

QUICK SUMMARY

- `Bool` — `true` / `false`, no truthiness
- `toggle()` — flip in place
- `isEmpty` — preferred emptiness test

Optionals — Basics

WHAT AN OPTIONAL IS

An optional T? holds either a value of type T or `nil`. Non-optional types can **never** be nil, so the compiler forces you to deal with missing values only where they can occur. You unwrap with `if let` or `guard let`; the force-unwrap `!` crashes if the value is nil.

NOTES

- `T?` is shorthand for `Optional<T>`
- Optionals are `nil` by default when declared as `var`
- Many APIs return optionals: `Int("x")`, `array.first`, `dict[key]`
- `!` is a promise to the compiler: break it and the app crashes

UNWRAPPING

```
var nickname: String? = nil          // no value
nickname = "Ace"

if let nickname {                    // shorthand (5.7+)
    print("Hi, \(nickname)")         // String, not String?
}

if let n = Int("42"), n > 0 {        // unwrap + condition
    print(n * 2)                     // 84
}

func greet(_ name: String?) {
    guard let name else { return }   // early exit if nil
    print("Hello, \(name)")
}

let first = [1, 2, 3].first!         // 1 — crashes if empty
```

CODE EXPLAINED

- → `String?` — may be nil
- → `if let x` — unwraps into a new non-optional constant
- → `, n > 0` — more conditions after the unwrap
- → `guard let … else` — unwrapped value lives after the guard
- → `!` — force unwrap; traps on nil

THE SWIFTY WAY

✗ `if x != nil { use(x!) }`
✓ `if let x { use(x) }` — one check, no force unwrap

COMING FROM…

- **Kotlin** `String?` and `?.` work almost the same
- **TypeScript** Like strictNullChecks, enforced
- **Java** `Optional<T>` but built into the language

SWIFT VERSIONS

- 5.7 — `if let x` shorthand (was `if let x = x`)

UNWRAPPING OPTIONS

SYNTAX	WHEN
`if let x`	Use the value in a branch
`guard let x else`	Exit early if missing
`x ?? default`	Fallback value
`x?.prop`	Chain, result stays optional
`x!`	You know it can't be nil

⚠ COMMON MISTAKES

- Force-unwrapping user input or network data: crashes on bad data
- Nesting `if let` pyramids: use `guard let` or one `if let a, let b`
- Comparing an optional Bool with `if flag == true` without realizing nil is false there

✓ TIPS & BEST PRACTICES

- Unwrap several at once: `if let a, let b, a > b { }`
- Use `guard let` at the top of functions to keep the main path unindented
- Search the project for `!` before shipping

QUICK SUMMARY

- `T?` — value or nil
- `if let` / `guard let` — safe unwrap
- `!` — force unwrap, crashes on nil

Optionals — Chaining & Coalescing

BEYOND IF LET

Optional **chaining** (`?.`) calls through an optional and short-circuits to `nil` if any link is missing. The **nil-coalescing** operator `??` supplies a default. Because `Optional` is an enum with `map` and `flatMap`, you can transform the wrapped value without unwrapping it first.

NOTES

- The result of a chain is always optional
- `??` is lazy: the right side runs only if needed
- `T!` (implicitly unwrapped) is mainly for IBOutlets
- `Optional` is `enum { case none, some(T) }`

CHAINING & COALESCING

```
struct User { var address: Address? }
struct Address { var city: String? }
let user: User? = User(address: Address(city: "Tripoli"))

user?.address?.city                  // Optional("Tripoli")
user?.address?.city?.count           // Optional(7)
user?.address?.city ?? "Unknown"     // "Tripoli"

let port = Int(env["PORT"] ?? "") ?? 8080   // chained ??

let n: Int? = 4
n.map { $0 * 2 }                     // Optional(8)
n.flatMap { $0 > 3 ? $0 : nil }      // Optional(4)

switch n {
case .some(let v): print(v)          // 4
case .none: print("nil")
}
```

CODE EXPLAINED

- → `?.` — call only if non-nil, else the chain is nil
- → `??` — default when nil
- → `map` — transform the value if present
- → `flatMap` — transform that itself may return nil
- → `.some` / `.none` — Optional's enum cases

THE SWIFTY WAY

✗ `if let c = city { return c } else { return "?" }`
✓ `return city ?? "?"`

COMING FROM…

- **Kotlin** `?.` same; `?:` ≈ `??`
- **TypeScript** `?.` and `??` same syntax
- **Python** No `?.`; `x if x is not None else d`

OPTIONAL OPERATORS

SYNTAX	RESULT TYPE
`a?.b`	B?
`a ?? b`	B (or B? if b is optional)
`a.map(f)`	U?
`a.flatMap(f)`	U? (f returns U?)
`a!`	A (traps on nil)

⚠ COMMON MISTAKES

- Expecting `user?.name.count` to be `Int`: anything after `?.` is optional
- Adding `?? ""` just to silence the compiler: it hides data that is really missing
- Declaring properties as `T!` to dodge initializers: crashes later instead of failing at compile time

✓ TIPS & BEST PRACTICES

- Use `?? ""` only when an empty string is a real answer, not to silence the compiler
- Collapse `T??` with `flatMap { $0 }`
- Optional chaining also works for assignment: `user?.name = "Ada"`

QUICK SUMMARY

- `?.` — optional chaining
- `??` — default value
- `map` / `flatMap` — transform without unwrapping

Type Conversion & Casting

CONVERTING BETWEEN TYPES

Swift never converts numeric types implicitly: `Int + Double` is a compile error. You convert with an **initializer** (`Double(x)`), and conversions that can fail, such as parsing a string, return an optional. For class hierarchies and protocols, `as?` casts safely, `as!` forces, and `as` upcasts or bridges.

NOTES

- Numeric initializers truncate or trap on out-of-range values
- `Int(exactly:)` returns nil instead of losing data
- `String(x)` and `"\(x)"` both stringify
- `as?` returns an optional; `as!` traps on failure

CONVERSIONS

```
let i = 7, d = 2.5
Double(i) * d                       // 17.5
Int(d)                              // 2 (truncates)
Int(d.rounded())                    // 3 (round first)
Int(exactly: 2.5)                   // nil
Int8(300)                           // CRASH: out of range
Int8(clamping: 300)                 // 127
UInt8(truncatingIfNeeded: 300)      // 44

Int("123")                          // Optional(123)
Int("12a")                          // nil
Int("ff", radix: 16)                // Optional(255)
Double("3.14")                      // Optional(3.14)
String(42)                          // "42"
String(255, radix: 2)               // "11111111"

let anyValue: Any = "hi"
anyValue as? String                 // Optional("hi")
anyValue as? Int                    // nil
```

CODE EXPLAINED

- → `Double(i)` — explicit numeric conversion
- → `Int(exactly:)` — nil if not representable
- → `clamping:` / `truncatingIfNeeded:` — non-trapping
- → `Int(_:radix:)` / `String(_:radix:)` — base conversion
- → `as?` — conditional cast

THE SWIFTY WAY

✗ `let n = Int(text)!`
✓ `guard let n = Int(text) else { … }` — user input fails sometimes

COMING FROM…

- **Java** No implicit widening; always explicit
- **TypeScript** `Number(s)` ≈ `Double(s)` but returns nil, not NaN
- **Python** `int(s)` raises; `Int(s)` returns nil

CAST OPERATORS

OPERATOR	USE
`as`	Upcast, literal type, bridging
`as?`	Downcast, optional result
`as!`	Downcast, traps on failure
`is`	Type check → Bool

⚠ COMMON MISTAKES

- Adding `Int` and `Double` directly: convert one side first
- `Int8(300)` and similar: out-of-range numeric init traps; use `clamping:`
- `Int(" 42")` with whitespace returns nil: trim first

✓ TIPS & BEST PRACTICES

- Use `Int(text.trimmingCharacters(in: .whitespaces))` for user input
- Prefer `as?` with `if let` over `is` followed by `as!`
- `Int(exactly:)` is the safe way to accept only whole-number Doubles

QUICK SUMMARY

- `Type(value)` — explicit conversion
- `Int("…")` — failable parsing
- `as?` / `as!` / `is` — runtime casts

Type Aliases & Type Checking

NAMING AND INSPECTING TYPES

`typealias` gives an existing type a second name, which is useful for long closure or generic types. At runtime, `type(of:)` returns a value's dynamic type and `is` tests it. `Any` holds a value of any type, `AnyObject` holds any class instance, and `Self` refers to the current type inside a type or protocol.

NOTES

- A typealias creates no new type: both names are interchangeable
- `Any` loses type safety; avoid it in APIs
- `T.self` is the metatype value for a type
- `Self` (capital S) = current type; `self` = current instance

ALIASES & RUNTIME TYPES

```
typealias UserID = String
typealias Completion = (Result<Data, Error>) -> Void
typealias Grid<T> = [[T]]           // generic alias

let id: UserID = "u_42"             // just a String

let values: [Any] = [1, "two", 3.0]
for v in values {
    print(type(of: v))              // Int, String, Double
    if v is String { print("string!") }
}

let t: Int.Type = Int.self          // metatype
t.init("5")                         // Optional(5)

struct Box {
    static func make() -> Self { Self() }   // Self = Box
}
```

CODE EXPLAINED

- → `typealias` — alternate name, same type
- → `type(of:)` — dynamic type at runtime
- → `is` — type test
- → `Int.self` — the type as a value
- → `Self` — the enclosing type

THE SWIFTY WAY

✗ `[String: Any]` for your model
✓ a `struct` with typed properties — Any defeats the compiler

TOP TYPES

TYPE	HOLDS
`Any`	Any value, including functions
`AnyObject`	Any class instance
`any P`	Any type conforming to P
`Never`	No value (function never returns)

COMING FROM…

- **TypeScript** `type X = …` ≈ typealias
- **Kotlin** `typealias` is identical
- **Python** `type(x)` ≈ `type(of: x)`; `isinstance` ≈ `is`

TYPE TOOLS

EXPRESSION	RESULT
`type(of: x)`	Dynamic type
`x is T`	Bool
`T.self`	Metatype value
`String(describing: T.self)`	Type name as text

⚠ COMMON MISTAKES

- Expecting a typealias to block mixing `UserID` and `String`: use a wrapper struct for real safety
- Using `Any` arrays and casting everywhere: model the data with an enum or protocol
- Confusing `Self` (type) with `self` (instance)

✓ TIPS & BEST PRACTICES

- Wrap IDs in a struct (`struct UserID: Hashable { let raw: String }`) for compile-time safety
- Use typealiases to name long closure types used in many places
- Use `Never` as a return type for functions that always `fatalError`

QUICK SUMMARY

- `typealias` — second name for a type
- `type(of:)` / `is` — runtime type checks
- `Any` / `AnyObject` / `Self` — special types

String Basics

STRINGS ARE VALUES

`String` is a **value type**: assigning or passing it makes an independent copy (cheaply, thanks to copy-on-write). Strings are collections of `Character`, which are full Unicode grapheme clusters, so `count` is what a user sees, not the byte count. Multiline literals use triple quotes and raw strings use #.

NOTES

- `count` is O(n): it walks the grapheme clusters
- Concatenate with `+`, `+=` or `append`
- `"""` lines are trimmed to the closing quote's indentation
- Raw strings `#"…"#` treat `\` literally

STRING LITERALS

```
var s = "Hello"
s += ", world"                        // "Hello, world"
s.count                               // 12
s.isEmpty                             // false

let poem = """
    Roses are red,
      violets are blue
    """                               // indentation stripped to """

let path = #"C:\Users\bob"#           // backslashes literal
let regex = #"\d+\.\d+"#
let named = #"Hi \#(name)"#           // interpolation in raw

var t = s                             // copy
t += "!"
s                                     // "Hello, world" (unchanged)

"café" == "cafe\u{301}"               // true (canonical equality)
"👨‍👩‍👧".count                        // 1
```

CODE EXPLAINED

- → `+=` — append to a `var` string
- → `"""` — multiline literal
- → `#"…"#` — raw string; `\#(x)` interpolates
- → `t = s` — independent copy
- → `==` — compares Unicode meaning, not bytes

THE SWIFTY WAY

✗ `s.count == 0`
✓ `s.isEmpty` — isEmpty is O(1); count is O(n)

COMING FROM…

- **Java** Value semantics; no `.equals` needed
- **TypeScript** `.length` counts UTF-16; `count` counts characters
- **Python** Like `str`, but `+=` mutates a `var` in place

SWIFT VERSIONS

- 5.0 — raw strings `#"…"#`
- 4.0 — multiline `"""` literals

ESCAPES

ESCAPE	MEANING
`\n \t`	Newline, tab
`\" \\`	Quote, backslash
`\u{1F600}`	Unicode scalar
`\ (line end)`	Join lines in """ literal

⚠ COMMON MISTAKES

- Calling `count` in a loop condition on a long string: it's O(n) each time
- Expecting `s[0]`: integer subscripting doesn't exist (see Sheet 019)
- Misaligned closing `"""`: lines indented less than it are a compile error

✓ TIPS & BEST PRACTICES

- Use raw strings for regex and Windows paths
- Build big strings with `append` in a loop or `joined()`, not repeated `+`
- Use `s.utf8.count` when you need the byte length

QUICK SUMMARY

- `String` — value type of Characters
- `"""` — multiline; `#"…"#` — raw
- `isEmpty` / `count` — emptiness and length

String Interpolation & Formatting

BUILDING STRINGS

String interpolation `\(expr)` embeds any expression in a literal. For numbers, dates and lists, Foundation's **FormatStyle** API (`.formatted()`) produces locale-aware text and is the modern replacement for `String(format:)` and the older `Formatter` classes.

NOTES

- Any expression can go inside `\( )`, including calls
- `.formatted()` respects the user's locale
- `String(format:)` uses C-style `%d %.2f %@` specifiers
- Custom interpolation: extend `DefaultStringInterpolation`

INTERPOLATION & FORMATTED()

```
import Foundation
let name = "Ada", score = 92.456, n = 1234567

"Hi \(name), you scored \(score)"   // ... 92.456
"Sum: \(2 + 3)"                     // Sum: 5

score.formatted(.number.precision(.fractionLength(1)))  // "92.5"
n.formatted()                       // "1,234,567"
0.256.formatted(.percent)           // "25.6%"
19.99.formatted(.currency(code: "USD"))   // "$19.99"
Date.now.formatted(date: .abbreviated, time: .omitted)
["a", "b", "c"].formatted()         // "a, b, and c"

String(format: "%.2f", score)       // "92.46"
String(format: "%05d", 42)          // "00042"
String(format: "%@ is %d", name, 36)    // "Ada is 36"
```

CODE EXPLAINED

- → `\(expr)` — interpolate any value
- → `.formatted()` — locale-aware formatting
- → `.precision(.fractionLength(n))` — decimal places
- → `.currency(code:)` / `.percent` — styles
- → `String(format:)` — printf-style

THE SWIFTY WAY

✗ `String(format: "%.1f%%", p * 100)`
✓ `p.formatted(.percent)` — locale-correct for free

COMING FROM…

- **TypeScript** `${x}` ≈ `\(x)`; Intl ≈ FormatStyle
- **Kotlin** `"$x"` ≈ `"\(x)"`
- **Python** f-strings; `f"{x:.2f}"` ≈ precision style

FORMAT SPECIFIERS

SPEC	TYPE
%d	Int
%.2f	Double, 2 places
%@	String / object
%x	Hex
%5d / %-5d	Pad right / left

⚠ COMMON MISTAKES

- Passing a Swift `Int` to `%@` or a String to `%d` in `String(format:)`: garbage or a crash
- Hard-coding `","` separators and `$` signs: breaks in other locales
- Interpolating an optional: prints `Optional(…)` and warns

✓ TIPS & BEST PRACTICES

- Use `.formatted(.number.grouping(.never))` to drop thousands separators
- Pass `locale: Locale(identifier: "en_US_POSIX")` for machine-readable output
- Use `Text(score, format: .percent)` in SwiftUI with the same styles

QUICK SUMMARY

- `\(x)` — interpolation
- `.formatted(...)` — modern, locale-aware
- `String(format:)` — C-style specifiers

Characters & Unicode

CHARACTERS AND UNICODE

A `Character` is one **extended grapheme cluster**: what a reader sees as one symbol, even if it is made of several Unicode scalars (like an emoji with a skin tone). A String also exposes **views** on its contents: `unicodeScalars`, `utf8` and `utf16`, each with its own count.

NOTES

- Character literals use double quotes: `let c: Character = "A"`
- `Character` has properties like `isLetter`, `isNumber`, `isUppercase`
- `utf16` count matches `NSString.length` and JavaScript `.length`
- Iterating a String yields Characters

CHARACTERS & VIEWS

```
let s = "Café 👍🏽"
s.count                              // 6  characters
s.unicodeScalars.count               // 7  (👍 + 🏽 modifier)
s.utf16.count                        // 9
s.utf8.count                         // 14 bytes

for ch in "a1B!" {
    print(ch, ch.isLetter, ch.isNumber, ch.isUppercase)
}                                    // a true false false …

let c: Character = "7"
c.wholeNumberValue                   // Optional(7)
Character("A").asciiValue            // Optional(65)
Character(UnicodeScalar(66))         // "B"

"é".unicodeScalars.map(\.value)      // [233]
Array("abc")                         // ["a", "b", "c"]
```

CODE EXPLAINED

→ `count` — user-visible characters

→ `unicodeScalars` / `utf16` / `utf8` — encoding views

→ `isLetter` / `isNumber` — Unicode-aware tests

→ `wholeNumberValue` / `asciiValue` — numeric values

THE SWIFTY WAY

✗ `Array(s.utf8)` to count letters

✓ `s.count` or `s.filter(\.isLetter).count` — bytes ≠ characters

COMING FROM…

- **TypeScript** `.length` ≈ `utf16.count`, not `count`
- **Java** `char` is UTF-16 unit; `Character` is a grapheme
- **Python** `len(s)` counts code points ≈ `unicodeScalars.count`

CHARACTER PROPERTIES

PROPERTY	TRUE FOR
`isLetter`	a, é, ж
`isNumber`	7, ½, ٣
`isWhitespace`	space, tab, newline
`isPunctuation`	! , .
`isUppercase`	A, É

⚠ COMMON MISTAKES

- Sending `s.count` to an API that expects UTF-16 or byte length
- Assuming `isNumber` means ASCII 0–9: it includes other numeral systems; use `isASCII && isNumber`
- Treating each `unicodeScalar` as a visible character

✓ TIPS & BEST PRACTICES

- Use `NSRange(s.startIndex..., in: s)` when bridging to NSString APIs
- Use `s.lowercased()` before comparing, or `localizedCaseInsensitiveCompare`
- `Character.isHexDigit` and `hexDigitValue` help parse hex

QUICK SUMMARY

- `Character` — one grapheme cluster
- `utf8` / `utf16` / `unicodeScalars` — views
- `isLetter` etc. — Unicode-aware tests

String Indices & Substrings

INDEXING INTO STRINGS

Because characters vary in size, Strings are indexed by `String.Index`, not `Int`. You move from `startIndex` with `index(_:offsetBy:)`. Slicing a String produces a `Substring`, which shares the original's storage; convert it to `String` before storing it long-term.

NOTES

- `endIndex` is one past the last character
- `prefix(n)` / `dropFirst(n)` avoid manual indices
- Substring has nearly all String methods
- `index(_:offsetBy:limitedBy:)` avoids out-of-bounds traps

INDICES & SUBSTRINGS

```swift
let s = "Hello, Swift"

s[s.startIndex]                      // "H"
let i = s.index(s.startIndex, offsetBy: 7)
s[i]                                 // "S"
s[i...]                              // "Swift" (Substring)
s[..<i]                              // "Hello, "

s.prefix(5)                          // "Hello"
s.suffix(5)                          // "Swift"
s.dropFirst(7)                       // "Swift"
s.dropLast(7)                        // "Hello"

if let comma = s.firstIndex(of: ",") {
    let word = String(s[..<comma])   // "Hello" as String
}
s.index(s.startIndex, offsetBy: 50, limitedBy: s.endIndex)  // nil
```

CODE EXPLAINED

- → `String.Index` — opaque position, not an Int
- → `index(_:offsetBy:)` — walk n characters
- → `s[i...]` / `s[..<i]` — range slices → Substring
- → `prefix` / `suffix` / `dropFirst` — easy slicing
- → `String(sub)` — copy into its own String

THE SWIFTY WAY

✗ `s[s.index(s.startIndex, offsetBy: 3)]` everywhere
✓ `Array(s)[3]` once, or `prefix` / `dropFirst` — arrays give O(1) Int indexing

COMING FROM…

- **TypeScript** `s.slice(0, 5)` ≈ `s.prefix(5)`
- **Python** `s[7:]` ≈ `s.dropFirst(7)`
- **Kotlin** `s[7]` works there; here use `String.Index`

INDEX MEMBERS

MEMBER	MEANING
`startIndex`	First character
`endIndex`	Past the end
`index(after:)`	Next index
`index(before:)`	Previous index
`indices`	All valid indices

⚠ COMMON MISTAKES

- Using an index from one string on another: undefined results or a trap
- Subscripting `s[s.endIndex]`: past the end, crashes
- Holding a small Substring of a huge String: keeps the whole buffer alive

✓ TIPS & BEST PRACTICES

- For lots of random access, convert once: `let chars = Array(s)`
- `s.split(separator:)` and `firstIndex(of:)` usually beat manual index math
- Use `limitedBy:` when the offset comes from input

QUICK SUMMARY

- `String.Index` — positions, not Ints
- `prefix` / `suffix` / `dropFirst` — easy slices
- `Substring` → `String(sub)` before storing

String Methods — Search

FINDING THINGS IN STRINGS

Searching uses methods on String (many from Foundation): `contains`, `hasPrefix`, `hasSuffix`, `firstIndex(of:)`, and `range(of:)` for substrings. Comparisons are **case-sensitive** by default; pass `options:` such as `.caseInsensitive` to `range(of:)`, or lowercase both sides.

NOTES

- Add `import Foundation` for `range(of:)` and `components(separatedBy:)`
- `contains` takes a Character, a String, or a predicate
- `localizedStandardContains` is what Finder-style search uses
- Regex-based search: see Sheet 093

SEARCH METHODS

```
import Foundation
let s = "Swift is fast. Swift is safe."

s.contains("fast")                      // true
s.contains("FAST")                      // false
s.localizedCaseInsensitiveContains("FAST")   // true
s.hasPrefix("Swift")                    // true
s.hasSuffix("safe.")                    // true

s.firstIndex(of: ".")                   // Index of first "."
s.contains(where: \.isNumber)           // false

if let r = s.range(of: "is") {
    s[r.upperBound...]                  // " fast. Swift is safe."
}
s.range(of: "SWIFT", options: .caseInsensitive) != nil   // true
s.ranges(of: "Swift").count             // 2
s.starts(with: "Sw")                    // true
```

CODE EXPLAINED

- → `contains(_:)` — substring or Character
- → `hasPrefix` / `hasSuffix` — start / end tests
- → `range(of:)` — `Range<String.Index>?` of a match
- → `options: .caseInsensitive` — Foundation compare options
- → `ranges(of:)` — every match (5.7+)

THE SWIFTY WAY

✗ `s.lowercased().contains(q.lowercased())`
✓ `s.localizedStandardContains(q)` — handles case and diacritics

COMING FROM…

- **TypeScript** `includes` / `startsWith` ≈ `contains` / `hasPrefix`
- **Python** `in` / `startswith` / `find` ≈ same here
- **Java** `indexOf` returns -1; `range(of:)` returns nil

COMPARE OPTIONS

OPTION	EFFECT
`.caseInsensitive`	A == a
`.diacriticInsensitive`	é == e
`.backwards`	Search from the end
`.anchored`	Match only at start
`.regularExpression`	Pattern is a regex

⚠ COMMON MISTAKES

- Expecting `firstIndex(of:)` to take a String: it takes a Character; use `range(of:)`
- Forgetting `import Foundation`: `range(of:)` won't be found
- Comparing user search text case-sensitively

✓ TIPS & BEST PRACTICES

- Use `localizedStandardContains` for user-facing search boxes
- Combine options: `[.caseInsensitive, .diacriticInsensitive]`
- `s.firstRange(of:)` (5.7+) works without Foundation

QUICK SUMMARY

- `contains` / `hasPrefix` / `hasSuffix` — tests
- `range(of:options:)` — locate a substring
- `localizedStandardContains` — user search

String Methods — Transform

CHANGING STRINGS

Transform methods return a **new** String (or array), leaving the original unchanged unless you use a mutating variant like `append`, `insert` or `removeAll(where:)`. Splitting returns `[Substring]` with `split`, or `[String]` with Foundation's `components(separatedBy:)`.

NOTES

- `split` drops empty pieces by default
- `trimmingCharacters(in:)` needs Foundation
- `replacing(_:with:)` is the 5.7+ stdlib form
- `joined(separator:)` turns `[String]` back into one String

TRANSFORM METHODS

```
import Foundation
let s = "  Hello, Swift  "

s.uppercased()                          // "  HELLO, SWIFT  "
s.lowercased()
s.trimmingCharacters(in: .whitespaces)    // "Hello, Swift"
s.replacing("Swift", with: "World")       // "  Hello, World  "

"a,b,,c".split(separator: ",")      // ["a", "b", "c"]
"a,b,,c".split(separator: ",", omittingEmptySubsequences: false)
"a,b,,c".components(separatedBy: ",")     // ["a","b","","c"]
["a", "b", "c"].joined(separator: "-")    // "a-b-c"

String("abc".reversed())            // "cba"
"swift".capitalized                 // "Swift"

var t = "Hi"
t.append("!")                       // "Hi!"
t.insert(contentsOf: ">> ", at: t.startIndex)   // ">> Hi!"
```

CODE EXPLAINED

- → `uppercased` / `lowercased` / `capitalized` — case
- → `trimmingCharacters(in:)` — strip edges
- → `replacing(_:with:)` — every occurrence
- → `split` → `[Substring]`; `components` → `[String]`
- → `reversed()` — wrap in `String(...)`

THE SWIFTY WAY

✗ `s.replacingOccurrences(of: a, with: b)`
✓ `s.replacing(a, with: b)` — stdlib, no Foundation needed (5.7+)

COMING FROM…

- **TypeScript** `trim()` ≈ `trimmingCharacters(in: .whitespaces)`
- **Python** `s.strip().split(",")` ≈ same chain
- **Kotlin** `split` keeps empties there; Swift drops them

CHARACTER SETS

SET	CONTAINS
`.whitespaces`	Space, tab
`.whitespacesAndNewlines`	Plus newlines
`.punctuationCharacters`	. , ! ?
`.decimalDigits`	0–9 and other digits
`CharacterSet(charactersIn:)`	Your own set

⚠ COMMON MISTAKES

- Expecting `split` to keep empty fields when parsing CSV
- Calling `reversed()` and getting a `ReversedCollection` instead of a String
- Using `.whitespaces` and leaving trailing newlines: use `.whitespacesAndNewlines`

✓ TIPS & BEST PRACTICES

- `s.split(whereSeparator: \.isWhitespace)` splits on any whitespace
- `s.filter { !$0.isPunctuation }` removes characters by test
- Use `localizedCapitalized` for user-visible title case

QUICK SUMMARY

- `uppercased` / `trimming…` / `replacing` — new Strings
- `split` / `components` — break apart
- `joined(separator:)` — put back together

Arithmetic Operators

MATH OPERATORS

Swift has the usual `+ - * / %` operators, but with two safety rules: both sides must be the **same type**, and overflow **traps** instead of wrapping. There are no ++ or -- operators (removed in Swift 3); use `+= 1`. The & versions (`&+ &- &*`) wrap on overflow when you want that.

NOTES

- `/` on integers truncates toward zero
- `%` is remainder (sign follows the left operand), not modulo
- `+` also concatenates Strings and Arrays
- Unary minus: `-x`; unary plus exists but does nothing

ARITHMETIC

```
let a = 17, b = 5
a + b                           // 22
a - b                           // 12
a * b                           // 85
a / b                           // 3
a % b                           // 2
-a % b                          // -2
Double(a) / Double(b)           // 3.4

var count = 0
count += 1                      // no count++ in Swift

UInt8.max &+ 1                  // 0   (wraps)
Int8.min &- 1                   // 127 (wraps)
// UInt8.max + 1                // compile error (constant)
// let x: UInt8 = 255; x + 1    // runtime trap

[1, 2] + [3]                    // [1, 2, 3]
"ab" + "cd"                     // "abcd"
```

CODE EXPLAINED

- → `/` — integer division truncates
- → `%` — remainder keeps the left sign
- → `+= 1` — replaces `++`
- → `&+ &- &*` — wrapping (overflow) operators
- → `+` — also joins Strings and Arrays

THE SWIFTY WAY

- ✗ `i++`
- ✓ `i += 1` — `++` was removed in Swift 3
- ✗ `(a % n + n) % n` copied around
- ✓ a small `mod(_:_:)` helper — Swift has no built-in modulo

COMING FROM…

- **Java** No `++`; overflow traps instead of wrapping
- **Python** `7 // 2` floors; Swift `7 / 2` truncates toward zero
- **TypeScript** Int and Double never mix without conversion

ARITHMETIC OPERATORS

OP	NAME	EXAMPLE	RESULT
`+`	Add	`7 + 2`	9
`-`	Subtract	`7 - 2`	5
`*`	Multiply	`7 * 2`	14
`/`	Divide	`7 / 2`	3
`%`	Remainder	`-7 % 2`	-1
`&+`	Wrapping add	`Int8.max &+ 1`	-128

⚠ COMMON MISTAKES

- Expecting `7 / 2` to be `3.5`: convert to Double first
- Using `%` as Python-style modulo on negative numbers
- Mixing `Int` and `Double` in one expression without conversion

✓ TIPS & BEST PRACTICES

- `x.quotientAndRemainder(dividingBy: y)` returns both at once
- Use `multipliedReportingOverflow(by:)` to detect overflow safely
- `pow` is in Foundation and works on Double: `pow(2.0, 10)`

QUICK SUMMARY

- `+ - * / %` — same-type operands only
- Overflow traps; `&+` etc. wrap
- `+= 1` — no `++`

Comparison Operators

COMPARING VALUES

Comparison operators return `Bool`. == and != work for any `Equatable` type; <, >, <=, >= need `Comparable`. For class instances, === checks whether two references point to the **same object**, while == compares values. Tuples of up to 6 Equatable elements compare element by element.

NOTES

- Structs and enums get `==` for free by declaring `: Equatable`
- Strings compare by Unicode canonical equivalence
- Optionals compare: `nil == nil` is true; `nil < 1` is an error
- `===` / `!==` only work on class instances

COMPARISONS

```
3 == 3                              // true
3 != 4                              // true
"apple" < "banana"                  // true (lexicographic)
(1, "b") < (1, "c")                 // true (tuple order)

struct Point: Equatable { var x, y: Int }
Point(x: 1, y: 2) == Point(x: 1, y: 2)   // true (synthesized)

class Node {}
let a = Node(), b = a, c = Node()
a === b                             // true  (same object)
a === c                             // false

let score: Int? = 90
score == 90                         // true
max(3, 9, 4)                        // 9
min("pear", "fig")                  // "fig"
(1...10).contains(5)                // true
```

CODE EXPLAINED

- → `== !=` — Equatable
- → `< > <= >=` — Comparable
- → `: Equatable` — compiler writes `==` for you
- → `===` — identity of class instances
- → `max` / `min` — variadic, any Comparable

THE SWIFTY WAY

✗ `if x >= 1 && x <= 10`
✓ `if (1...10).contains(x)`

COMING FROM…

- **Java** `==` compares values here; `===` ≈ Java's `==` on objects
- **TypeScript** No `==` / `===` coercion split: `==` is strict
- **Kotlin** `==` ≈ `==`, `===` ≈ `===` (same meaning)

COMPARISON OPERATORS

OP	REQUIRES	EXAMPLE
`== !=`	Equatable	`a == b`
`< >`	Comparable	`a < b`
`<= >=`	Comparable	`a >= b`
`=== !==`	Class instances	`obj1 === obj2`
`~=`	Pattern match	`1...5 ~= 3`

⚠ COMMON MISTAKES

- Using `==` on class instances without implementing `Equatable`: compile error
- Using `===` to compare struct values: it only works on classes
- Comparing Doubles with `==` after arithmetic

✓ TIPS & BEST PRACTICES

- Add `: Comparable` and implement `<` to unlock `sorted()`, `min()`, `max()`
- Compare Strings for users with `localizedStandardCompare`
- `clamped` isn't built in: write `min(max(x, lo), hi)`

QUICK SUMMARY

- `==` — value equality (Equatable)
- `<` — ordering (Comparable)
- `===` — same object (classes)

Logical Operators

BOOLEAN LOGIC

The logical operators && (and), || (or) and ! (not) work only on `Bool`. && and || **short-circuit**: the right side is skipped if the left side already decides the result. In `if`, `guard` and `while`, a comma also means "and", and it lets you mix in `let` unwrapping.

NOTES

- `&&` binds tighter than `||`
- Use parentheses whenever you mix `&&` and `||`
- Comma-separated conditions are evaluated left to right
- `!` must touch its operand: `!isReady`, not `! isReady`

LOGICAL OPERATORS

```swift
let isAdmin = false, isOwner = true, isLocked = false

isAdmin || isOwner                  // true
isOwner && !isLocked                // true
!(isAdmin || isOwner)               // false

// short-circuit: second call never runs
func expensive() -> Bool { print("ran"); return true }
false && expensive()                // false, prints nothing
true  || expensive()                // true, prints nothing

// mixing needs parentheses for clarity
if (isAdmin || isOwner) && !isLocked { print("can edit") }

// commas in conditions = && plus unwrapping
let input: String? = "42"
if let s = input, let n = Int(s), n > 0 {
    print(n)                        // 42
}
```

CODE EXPLAINED

- → `||` — true if either side is true
- → `&&` — true if both sides are true
- → `!` — negation
- → short-circuit — right side may never run
- → `if a, b, c` — all must hold; can include `let`

THE SWIFTY WAY

✗ `if a == true && b == false`
✓ `if a && !b`

✗ nested `if` for each condition
✓ `if let x, let y, x > y { }`

COMING FROM…

- **Python** `and` `or` `not` ≈ `&&` `||` `!`
- **TypeScript** Same symbols; operands must be Bool
- **Kotlin** Same; plus commas in `if` here

TRUTH TABLE

A	B	A && B	A || B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

PRECEDENCE (HIGH → LOW)

OPERATOR	GROUP
!	Prefix
== < >	Comparison
&&	Conjunction
||	Disjunction
? :	Ternary

⚠ COMMON MISTAKES

- Relying on `&&`-over-`||` precedence without parentheses: hard to read and easy to break
- Putting a side effect on the right of `&&` / `||` and expecting it to always run
- Writing `! x` with a space: parsed differently or a compile error

✓ TIPS & BEST PRACTICES

- Name complex conditions: `let canEdit = (isAdmin || isOwner) && !isLocked`
- Put cheap checks first so short-circuiting skips costly ones
- Use `allSatisfy` / `contains(where:)` for collection-wide logic

QUICK SUMMARY

- `&& || !` — Bool-only logic
- Short-circuit evaluation
- `if a, b` — comma means and

Range Operators

RANGES

`a...b` is a **closed** range including both ends; `a..<b` is **half-open** and excludes b. One-sided ranges (`a...`, `..<b`) run to the end or from the start of a collection. Ranges drive `for` loops, slicing, `switch` cases and `contains` checks, and `stride` gives custom steps.

NOTES

- `..<` matches zero-based indices: `0..<array.count`
- `a...b` requires `a <= b`, otherwise it traps
- `stride(from:to:by:)` excludes the end; `through:` includes it
- `~=` is the pattern-match operator `switch` uses

RANGE OPERATORS

```
for i in 1...3 { print(i) }          // 1 2 3
for i in 0..<3 { print(i) }          // 0 1 2

let nums = [10, 20, 30, 40, 50]
nums[1...3]                          // [20, 30, 40]
nums[2...]                           // [30, 40, 50]
nums[..<2]                           // [10, 20]

(1...10).contains(7)                 // true
(1..<10).count                       // 9
let r = 5...                         // PartialRangeFrom<Int>
r.contains(1_000)                    // true

for x in stride(from: 0, to: 10, by: 3) { }       // 0 3 6 9
for x in stride(from: 10, through: 0, by: -5) { } // 10 5 0
for x in (1...3).reversed() { }      // 3 2 1
1...5 ~= 3                           // true
```

CODE EXPLAINED

- → `...` — closed; `..<` — half-open
- → `nums[2...]` / `nums[..<2]` — one-sided slices
- → `stride(from:to:by:)` — custom step, end excluded
- → `through:` — end included
- → `.reversed()` — count down

THE SWIFTY WAY

✗ `for i in 0...arr.count - 1`
✓ `for i in arr.indices` — no crash on an empty array

COMING FROM…

- **Kotlin** `1..3` ≈ `1...3`; `until` ≈ `..<`
- **Python** `range(0, 3)` ≈ `0..<3`
- **TypeScript** No ranges; `for (let i=0;i<3;i++)` ≈ `0..<3`

RANGE TYPES

SYNTAX	TYPE
`a...b`	ClosedRange
`a..<b`	Range
`a...`	PartialRangeFrom
`...b`	PartialRangeThrough
`..<b`	PartialRangeUpTo

⚠ COMMON MISTAKES

- `0...arr.count - 1` on an empty array: `0...-1` traps
- Building `5...1` to count down: use `reversed()` or `stride`
- Iterating a `PartialRangeFrom` with no `break`: it never ends

✓ TIPS & BEST PRACTICES

- Use `array.indices` instead of building index ranges by hand
- Ranges of `Double` work for `contains` but not for `for` loops; use `stride`
- `x.clamped(to:)` is easy to add as an extension on Comparable

QUICK SUMMARY

- `a...b` / `a..<b` — closed / half-open
- `a...` / `..<b` — one-sided
- `stride` — custom steps

Bitwise Operators

BIT OPERATORS

Bitwise operators work on the binary representation of integers: AND &, OR |, XOR ^, NOT ~, and shifts << and >>. Shifting past the bit width gives 0 (or -1 for negative signed values) instead of undefined behavior. The &<< and &>> forms mask the shift amount. The **Bitwise** set covers patterns in depth.

NOTES

- `>>` on signed types is arithmetic (keeps the sign)
- `OptionSet` is the Swift way to model flag bits
- Binary literals: `0b1010`; print with `String(n, radix: 2)`
- See the Bitwise set for masks, tricks and interview patterns

BIT MANIPULATION

```swift
let a: UInt8 = 0b1100_1010          // 202
let b: UInt8 = 0b1010_0110          // 166

a & b                               // 0b1000_0010  (130)
a | b                               // 0b1110_1110  (238)
a ^ b                               // 0b0110_1100  (108)
~a                                  // 0b0011_0101  (53)
a << 1                              // 0b1001_0100  (148)
a >> 2                              // 0b0011_0010  (50)

String(a, radix: 2)                 // "11001010"
a.nonzeroBitCount                   // 4
a.leadingZeroBitCount               // 0

struct Perms: OptionSet {
    let rawValue: Int
    static let read = Perms(rawValue: 1 << 0)
    static let write = Perms(rawValue: 1 << 1)
}
let p: Perms = [.read, .write]      // p.contains(.write) == true
```

CODE EXPLAINED

- → `& | ^ ~` — and, or, xor, not
- → `<< >>` — shifts; over-shift is safe
- → `nonzeroBitCount` — popcount
- → `OptionSet` — named flags with set syntax

THE SWIFTY WAY

✗ `if flags & 2 != 0`
✓ `if perms.contains(.write)` — OptionSet names every bit

COMING FROM…

- **Java** No `>>>`: use an unsigned type for logical shift
- **TypeScript** No 32-bit coercion; widths come from the type
- **Python** Same operators; fixed widths here

BITWISE OPERATORS

OP	NAME	0B1100 OP 0B1010
&	AND	`0b1000`
|	OR	`0b1110`
^	XOR	`0b0110`
~	NOT (unary)	`~0b1100 = …0011`
<< >>	Shift	`0b1100 << 1 = 0b11000`

⚠ COMMON MISTAKES

- Forgetting parentheses: `a & b == 0` is `a & (b == 0)`, a type error
- Using `>>` on a negative Int and expecting zeros shifted in
- Hand-rolling flag constants instead of an `OptionSet`

✓ TIPS & BEST PRACTICES

- Use `UInt8` / `UInt32` for bit fields so shifts don't sign-extend
- `n & (n - 1) == 0` (for n > 0) tests for a power of two
- The Bitwise set has a full page on these patterns

QUICK SUMMARY

- `& | ^ ~ << >>` — bit operators
- `OptionSet` — flags done right
- `String(n, radix: 2)` — see the bits

Assignment & Compound Assignment

ASSIGNING VALUES

= assigns but, unlike C, **returns nothing**, so `if x = y` is a compile error rather than a silent bug. Compound operators (+=, -=, *=, /=, %=, &=, |=, <<= …) update a `var` in place. Tuples let you assign or swap several values at once.

NOTES

- Assignment returns `Void`, so it can't be used as a condition
- Compound assignment works on any type that defines it (`String +=`)
- `swap(&a, &b)` and `(a, b) = (b, a)` both swap
- `??=` doesn't exist; write `x = x ?? default`

ASSIGNMENT FORMS

```
var n = 10
n += 5                          // 15
n -= 3                          // 12
n *= 2                          // 24
n /= 5                          // 4
n %= 3                          // 1
n <<= 3                         // 8

var s = "Hi"
s += "!"                        // "Hi!"
var list = [1]
list += [2, 3]                  // [1, 2, 3]

var (x, y) = (1, 2)             // tuple assignment
(x, y) = (y, x)                 // swap: x 2, y 1
swap(&x, &y)                    // back: x 1, y 2
list.swapAt(0, 2)               // [3, 2, 1]

let (q, _) = (7, 99)            // ignore with _
// if n = 5 { }                 // error: '=' returns Void
```

CODE EXPLAINED

- → `+= -= *= /= %=` — arithmetic compound
- → `<<= &= |= ^=` — bitwise compound
- → `(x, y) = (y, x)` — swap via tuples
- → `swapAt(_:_:)` — swap array elements
- → `_` — discard part of a tuple

THE SWIFTY WAY

✗ `let t = a; a = b; b = t`
✓ `(a, b) = (b, a)`

COMPOUND OPERATORS

OP	SAME AS
`a += b`	`a = a + b`
`a -= b`	`a = a - b`
`a *= b`	`a = a * b`
`a /= b`	`a = a / b`
`a <<= n`	`a = a << n`

COMING FROM…

- **Python** `a, b = b, a` ≈ `(a, b) = (b, a)`
- **Kotlin** No tuple swap; Swift has it
- **TypeScript** `[a, b] = [b, a]` ≈ tuple swap

⚠ COMMON MISTAKES

- Expecting `a = b = 0` to chain: `=` returns Void
- Calling `swap(&arr[0], &arr[1])` : overlapping access error; use `swapAt`
- Using compound assignment on a `let`

✓ TIPS & BEST PRACTICES

- Use `list.append(contentsOf:)` for clarity over `+=` when appending sequences
- Use `defer` or a local `var` instead of chained assignments
- Use `inout` + `&` to let a function assign to a caller's variable

QUICK SUMMARY

- `=` — returns Void
- `+=` etc. — update in place
- `(a, b) = (b, a)` — swap

Operator Overloading & Custom Operators

DEFINING OPERATORS

Types can overload existing operators by declaring a `static func` with the operator's name. You can also declare entirely new operators with `prefix`, `postfix` or `infix operator`, and give infix operators a **precedence group**. Use custom operators sparingly: a well-named method is usually clearer.

NOTES

- Operator functions are `static` members of a type or extension
- `Equatable` / `Comparable` / `Hashable` conformance = overloads of `==` / `<`
- New operators are declared at file scope
- Allowed characters: `/ = - + ! * % < > & | ^ ~ ?` and some Unicode

OVERLOADING & NEW OPERATORS

```
struct Vec: Equatable { var x, y: Double }

extension Vec {
    static func + (l: Vec, r: Vec) -> Vec { Vec(x: l.x + r.x, y: l.y + r.y) }
    static func * (v: Vec, k: Double) -> Vec { Vec(x: v.x * k, y: v.y * k) }
    static prefix func - (v: Vec) -> Vec { Vec(x: -v.x, y: -v.y) }
    static func += (l: inout Vec, r: Vec) { l = l + r }
}

var p = Vec(x: 1, y: 2) + Vec(x: 3, y: 4)  // (4, 6)
p += Vec(x: 1, y: 1)                       // (5, 7)
-p * 2                                     // (-10, -14)

infix operator **: PowerPrecedence
precedencegroup PowerPrecedence {
    higherThan: MultiplicationPrecedence
    associativity: right
}
func ** (b: Double, e: Double) -> Double { pow(b, e) }
2 ** 3 ** 2                                // 512.0
```

CODE EXPLAINED

- → `static func +` — overload an existing operator
- → `prefix func -` — unary operator
- → `inout` — compound assignment mutates the left side
- → `infix operator **` — declare a new operator
- → `precedencegroup` — precedence and associativity

THE SWIFTY WAY

✗ a `<*>` operator only you understand
✓ a method like `combined(with:)` — readers can search for a name

COMING FROM…

- **Kotlin** `operator fun plus` ≈ `static func +`
- **Python** `__add__` ≈ `static func +`
- **Java** No operator overloading at all

STANDARD PRECEDENCE GROUPS

GROUP	OPERATORS
BitwiseShift	<< >>
Multiplication	* / % &
Addition	+ - | ^
Comparison	== < >
NilCoalescing	??
Assignment	= += -=

⚠ COMMON MISTAKES

- Forgetting `static` on the operator function
- Defining a new operator without a precedence group, then mixing it with `+`
- Overloading `==` by hand when `: Equatable` would synthesize it

✓ TIPS & BEST PRACTICES

- Overload operators for math-like types (vectors, money, matrices) only
- Conform to `AdditiveArithmetic` to get `+ -` and `.zero` together
- Keep the operator and a named method in sync, e.g. `+` and `adding(_:)`

QUICK SUMMARY

- `static func +` — overload
- `infix/prefix/postfix operator` — declare new
- `precedencegroup` — control binding

if / else

BRANCHING WITH IF

`if` runs a block when its condition is `true`, with optional `else if` and `else` branches. Swift needs **no parentheses** around the condition but **always requires braces**. Conditions can be separated by commas and can include `let` unwrapping and `case` patterns.

NOTES

- Condition must be `Bool`: no truthiness
- Braces are mandatory even for one-line bodies
- Variables unwrapped with `if let` exist only inside that branch
- Since 5.9, `if` can also be an expression (Sheet 031)

IF / ELSE IF / ELSE

```
let temp = 72

if temp > 85 {
    print("Hot")
} else if temp > 60 {
    print("Nice")                    // Nice
} else {
    print("Cold")
}

// multiple conditions with commas (all must pass)
let input: String? = "18"
if let s = input, let age = Int(s), age >= 18 {
    print("Adult: \(age)")           // Adult: 18
}

// pattern in a condition
let result: Result<Int, Error> = .success(3)
if case .success(let v) = result, v > 2 { print(v) }    // 3

if #available(iOS 17, macOS 14, *) { /* new API */ }
```

CODE EXPLAINED

- → `if cond { }` — no parens, braces required
- → `else if` — chained checks, first match wins
- → `if let …, cond` — unwrap, then test
- → `if case` — match an enum pattern
- → `#available` — OS version check

THE SWIFTY WAY

✗ `if (x > 0) { … }`
✓ `if x > 0 { … }` — parentheses are noise in Swift

COMING FROM…

- **Kotlin** Same shape; `if` is also an expression
- **TypeScript** No parens, braces always
- **Python** `elif` ≈ `else if`; braces instead of indentation

CONDITION FORMS

FORM	EXAMPLE
Bool	`if isReady`
Unwrap	`if let user`
Pattern	`if case .ok(let v) = r`
Availability	`if #available(iOS 17, *)`
Combined	`if let x, x > 0, !done`

⚠ COMMON MISTAKES

- Writing a long `else if` chain over one value: use `switch`
- Using the unwrapped name after the `if` block ends: it's out of scope (use `guard`)
- `if #available` without `*`: required for future platforms

✓ TIPS & BEST PRACTICES

- Put the common case first for readability
- Use `guard` for preconditions so the happy path isn't nested
- Prefer `switch` when checking one value against several cases

QUICK SUMMARY

- `if / else if / else` — branching
- Commas join conditions, including `let`
- `if case` / `#available` — special conditions

guard

EARLY EXIT WITH GUARD

`guard` checks a condition and, if it fails, **must leave** the current scope with `return`, `throw`, `break`, `continue` or a `Never` call. Values unwrapped by `guard let` stay available for the rest of the scope. This keeps the main path at the left margin instead of nested inside `if` blocks.

NOTES

- The `else` block is required and must exit
- Unwrapped names live after the guard, unlike `if let`
- Works in functions, loops (`continue` / `break`) and closures
- `guard let self else { return }` is common in closures

GUARD IN PRACTICE

```swift
func register(name: String?, age: String?) throws -> User {
    guard let name, !name.isEmpty else {
        throw FormError.missingName
    }
    guard let ageText = age, let years = Int(ageText) else {
        throw FormError.badAge
    }
    guard years >= 13 else { throw FormError.tooYoung }

    return User(name: name, age: years)   // name, years usable here
}

for line in lines {
    guard !line.hasPrefix("#") else { continue }   // skip comments
    process(line)
}

loader.fetch { [weak self] data in
    guard let self else { return }
    self.show(data)
}
```

CODE EXPLAINED

→ `guard cond else { exit }` — continue only if true

→ `guard let x` — x stays in scope afterward

→ `throw` / `return` / `continue` — required exits

→ `guard let self` — safe self in closures

THE SWIFTY WAY

✗ `if let a { if let b { if let c { … } } }`

✓ three `guard let` lines, then the logic — flat beats nested

VALID EXITS

STATEMENT	WHERE
`return`	Functions, closures
`throw`	Throwing functions
`continue / break`	Loops, switch
`fatalError()`	Anywhere (returns Never)

GUARD VS IF LET

	GUARD LET	IF LET
Scope	Rest of block	Inside braces
Else	Required, must exit	Optional
Use for	Preconditions	Optional branches

COMING FROM…

- **Kotlin** `val x = y ?: return` ≈ `guard let x = y else { return }`
- **TypeScript** `if (!x) return;` early-return pattern
- **Python** `if x is None: return` at top

⚠ COMMON MISTAKES

- An `else` block that doesn't exit: compile error
- Using `guard` for normal branching where both paths continue
- Forgetting that `guard let` names can't be redeclared later in the same scope

✓ TIPS & BEST PRACTICES

- Validate all inputs with guards at the top, then write the real work
- Pair `guard` with `throw` to report why the input was bad
- `guard case .loaded(let data) = state else { return }` works with patterns

QUICK SUMMARY

- `guard … else { exit }` — early exit
- Unwrapped values stay in scope
- Keeps the happy path unindented

Conditional Expressions

CHOOSING A VALUE

The ternary operator `cond ? a : b` picks one of two values. Since Swift 5.9, `if` and `switch` can also be **expressions**: each branch is a single expression and the whole statement produces a value you can assign or return. This replaces the old pattern of declaring a `var` and setting it in each branch.

NOTES

- Each branch of an `if` / `switch` expression must be one expression
- All branches must produce the same type
- `if` expressions require an `else`
- Use ternary for short choices, expressions for anything longer

TERNARY & EXPRESSIONS

```
let age = 20
let label = age >= 18 ? "adult" : "minor"      // "adult"

// if expression (5.9+)
let fee = if age < 12 { 0.0 }
          else if age >= 65 { 5.0 }
          else { 10.0 }                        // 10.0

// switch expression
let grade = 87
let letter = switch grade {
    case 90...: "A"
    case 80..<90: "B"
    case 70..<80: "C"
    default: "F"
}                                              // "B"

func icon(for ok: Bool) -> String {
    if ok { "checkmark" } else { "xmark" }     // implicit return
}
```

CODE EXPLAINED

- → `c ? a : b` — ternary
- → `let x = if … { } else { }` — if expression
- → `let x = switch v { case …: value }` — switch expression
- → implicit return — single-expression function body

THE SWIFTY WAY

✗ `var x: String` then set in every branch
✓ `let x = if … { } else { }` — one `let`, checked for completeness

COMING FROM…

- **Kotlin** `val x = if (…) a else b` — same idea
- **TypeScript** Only ternary; no if-expressions
- **Python** `a if c else b` ≈ `c ? a : b`

SWIFT VERSIONS

- 5.9 — `if` and `switch` expressions
- 5.1 — implicit return for single-expression bodies

WHICH TO USE

FORM	BEST WHEN
`c ? a : b`	Two short values
`if expression`	Several conditions
`switch expression`	Matching one value
`Nested ternary`	Never: hard to read

⚠ COMMON MISTAKES

- Putting statements (like `print`) in a branch of an `if` expression: each branch must be one expression
- Leaving off `else` in an `if` expression
- Nesting ternaries three deep

✓ TIPS & BEST PRACTICES

- Use `switch` expressions to map enums to display strings
- Branches can call `fatalError()` or `throw`, since those return `Never`
- Format multi-branch `if` expressions one branch per line

QUICK SUMMARY

- `? :` — ternary
- `let x = if …` — if expression (5.9)
- `let x = switch …` — switch expression

switch — Basics

THE SWITCH STATEMENT

`switch` compares a value against cases and runs the **first** match. It must be **exhaustive**: every possible value is covered, either by listing every case (common with enums) or with `default`. There is **no implicit fallthrough**, so no `break` is needed at the end of a case.

NOTES

- A case can list several values: `case "a", "e", "i":`
- Cases can match ranges, tuples and patterns
- `fallthrough` exists but is rarely used
- Every case needs at least one statement (`break` if empty)

SWITCH BASICS

```
let ch: Character = "e"
switch ch {
case "a", "e", "i", "o", "u":
    print("vowel")                  // vowel
case "y":
    print("sometimes")
default:
    print("consonant")
}

let score = 85
switch score {
case 90...100: print("A")
case 80..<90:  print("B")           // B
case ..<0:     break                // do nothing
default:       print("other")
}

enum Direction { case north, south, east, west }
let d = Direction.east
switch d {                          // no default needed
case .north, .south: print("vertical")
case .east, .west:   print("horizontal")
}
```

CODE EXPLAINED

- → `case a, b:` — several values
- → `case 80..<90:` — range match
- → `default:` — catch-all
- → `break` — empty case body
- → enum switch — exhaustive without default

THE SWIFTY WAY

✗ `default:` on your own enum
✓ list every case — the compiler then flags new cases you forgot

CASE FORMS

CASE	MATCHES
`case 5:`	Exact value
`case 1, 2, 3:`	Any listed value
`case 1...9:`	Range
`case .north:`	Enum case
`case let x where x > 0:`	Binding + condition
`default:`	Everything else

COMING FROM…

- **Java** No `break` needed; no fall-through bugs
- **Kotlin** `when` ≈ `switch`; both exhaustive on enums
- **Python** `match` (3.10) is similar

⚠ COMMON MISTAKES

- Expecting fall-through into the next case like C
- Leaving a case body empty: needs at least `break`
- Adding `default` to enum switches and missing new cases later

✓ TIPS & BEST PRACTICES

- Use `@unknown default:` when switching over enums from Apple frameworks
- Switch on tuples to handle two values at once (Sheet 033)
- Use a switch expression when every case produces a value

QUICK SUMMARY

- `switch` — exhaustive, first match wins
- No implicit fallthrough
- `case a, b` / ranges / enums

Pattern Matching

PATTERNS

Swift patterns go far beyond equality. You can **bind** values (`case let x`), add a `where` clause, match **tuples** and enum **associated values**, and use the same patterns outside `switch` with `if case`, `guard case` and `for case`. Under the hood, value matching uses the ~= operator.

NOTES

- `_` matches anything and binds nothing
- `let` can go before the whole pattern or each binding
- `where` adds a Boolean condition to a case
- `case let x?` matches non-nil optionals

PATTERN MATCHING

```
let point = (3, 0)
switch point {
case (0, 0):           print("origin")
case (_, 0):           print("on x-axis")       // on x-axis
case (let x, let y) where x == y: print("diagonal")
default:               print("elsewhere")
}

enum Shape { case circle(r: Double), rect(w: Double, h: Double) }
let s = Shape.rect(w: 2, h: 3)
switch s {
case .circle(let r):            print(3.14 * r * r)
case let .rect(w, h) where w == h: print("square")
case .rect(let w, let h):       print(w * h)    // 6.0
}

if case .circle(let r) = s, r > 1 { }          // single case
let items: [Int?] = [1, nil, 3]
for case let n? in items { print(n) }          // 1 3
```

CODE EXPLAINED

- → `(_, 0)` — tuple pattern with wildcard
- → `case (let x, let y) where …` — bind + condition
- → `case .rect(let w, let h)` — extract associated values
- → `if case` — one pattern without switch
- → `for case let n?` — loop only non-nil

THE SWIFTY WAY

✗ `switch e { case .a(let v): use(v); default: break }`
✓ `if case .a(let v) = e { use(v) }`

COMING FROM…

- **Kotlin** `when` with `is` and destructuring ≈ patterns
- **Python** `match` with captures and guards ≈ same
- **TypeScript** Discriminated unions + `switch (x.kind)`

PATTERN KINDS

PATTERN	EXAMPLE
Wildcard	`_`
Binding	`let x`
Tuple	`(0, let y)`
Enum case	`.ok(let v)`
Optional	`let x?`
Type cast	`let s as String`

⚠ COMMON MISTAKES

- Writing `if case x = .ok` with the sides reversed: pattern on the left, value on the right
- Forgetting `?` in `for case let n?`: then `n` is still optional
- Putting expensive work in `where`: it runs for every case tried

✓ TIPS & BEST PRACTICES

- Overload `~=` to let your types match in `switch` (e.g. regex or predicate)
- Use `case let (x, y)` to bind every element at once
- Match type casts in a `switch` over `Any` with `case let s as String:`

QUICK SUMMARY

- `let` / `_` / tuples / enums — pattern kinds
- `where` — extra condition
- `if case` / `for case` — patterns outside switch

for-in Loops

LOOPING WITH FOR-IN

`for-in` iterates over any `Sequence`: ranges, arrays, dictionaries, sets, strings and more. There is no C-style `for (i = 0; …)` loop. Use `enumerated()` when you need the index, `zip` to walk two sequences together, and a `where` clause to filter.

NOTES

- The loop variable is a `let`; use `var` in the pattern to mutate it
- Dictionary iteration yields `(key, value)` tuples, unordered
- `_` ignores the loop value: `for _ in 1...3`
- `forEach` can't `break` or `continue`

FOR-IN FORMS

```
for i in 1...3 { print(i) }                  // 1 2 3
for _ in 0..<2 { print("hi") }               // hi hi

let names = ["Ada", "Grace", "Linus"]
for name in names { print(name) }

for (i, name) in names.enumerated() {
    print("\(i): \(name)")                   // 0: Ada …
}

let ages = ["Ada": 36, "Grace": 85]
for (name, age) in ages.sorted(by: { $0.key < $1.key }) {
    print(name, age)
}

for (n, a) in zip(names, [36, 85]) { }       // pairs up, stops at shorter
for n in 1...20 where n.isMultiple(of: 7) { print(n) }   // 7 14
for ch in "hey" { print(ch) }                // h e y
for i in stride(from: 10, to: 0, by: -3) { } // 10 7 4 1
```

CODE EXPLAINED

- → `for x in seq` — any Sequence
- → `enumerated()` — `(offset, element)` pairs
- → dictionary — `(key, value)`, unordered
- → `zip(a, b)` — walk two sequences
- → `where` — filter inside the loop header

THE SWIFTY WAY

✗ `for i in 0..<names.count { names[i] }`
✓ `for name in names` or `enumerated()`

COMING FROM…

- **Kotlin** `for ((i, x) in list.withIndex())` ≈ `enumerated()`
- **Python** `enumerate` / `zip` work the same way
- **TypeScript** `for…of` ≈ `for-in`; Swift has no `for…in` over keys

ITERATION HELPERS

HELPER	GIVES
`enumerated()`	(offset, element)
`indices`	Valid indices
`zip(a, b)`	(a[i], b[i])
`reversed()`	Back to front
`stride(from:to:by:)`	Stepped numbers

⚠ COMMON MISTAKES

- Using `enumerated()` offsets as indices into an ArraySlice (slices don't start at 0)
- Mutating an array while iterating it: you iterate the original copy
- Expecting dictionary loops in insertion order

✓ TIPS & BEST PRACTICES

- Use `zip(arr.indices, arr)` for real indices on slices
- Use `for case let x?` to skip nils
- `lazy` + `for` avoids building intermediate arrays

QUICK SUMMARY

- `for x in seq` — the only for loop
- `enumerated()` / `zip` — indices and pairs
- `where` — filter in the header

while & repeat-while

CONDITION-DRIVEN LOOPS

`while` checks its condition **before** each pass and may run zero times. `repeat-while` checks **after** each pass, so its body always runs at least once (Swift's name for do-while). Use these when you don't know the number of iterations in advance: reading input, retrying, or walking a linked structure.

NOTES

- `repeat { } while cond` replaces C's `do { } while`
- `while let` keeps looping while an optional has a value
- `while true` with `break` is fine for event loops
- Make sure something in the body changes the condition

WHILE & REPEAT-WHILE

```
var n = 27, steps = 0
while n != 1 {                          // Collatz
    n = n.isMultiple(of: 2) ? n / 2 : 3 * n + 1
    steps += 1
}
print(steps)                            // 111

var attempt = 0
repeat {
    attempt += 1
    print("try \(attempt)")
} while attempt < 3 && !connect()    // body runs at least once

var stack = [1, 2, 3]
while let top = stack.popLast() {    // until empty
    print(top)                          // 3 2 1
}

while let line = readLine() {        // stdin until EOF
    print(line.uppercased())
}
```

CODE EXPLAINED

- → `while cond` — check first, may run 0 times
- → `repeat { } while cond` — runs at least once
- → `while let` — loop while unwrapping succeeds
- → `readLine()` — next stdin line or nil at EOF

THE SWIFTY WAY

✗ `while !stack.isEmpty { let x = stack.removeLast() … }`

✓ `while let x = stack.popLast() { … }`

COMING FROM…

- **Java** `do { } while` ≈ `repeat { } while`
- **Kotlin** `do { } while` ≈ `repeat { } while`
- **Python** No do-while; `while let` ≈ walrus `while (x := f())`

LOOP CHOICE

LOOP	USE WHEN
`for-in`	Known sequence
`while`	Condition first
`repeat-while`	Body must run once
`while let`	Draining an optional source

⚠ COMMON MISTAKES

- Writing `do { } while`: in Swift `do` starts a scope or `do/catch`
- Forgetting to update the condition variable: infinite loop
- Busy-waiting in `while` for async work: use `await` instead

✓ TIPS & BEST PRACTICES

- Add a max-attempt counter to retry loops
- `while let` works with iterators: `var it = seq.makeIterator(); while let x = it.next()`
- Use `Task.sleep(for:)` inside async retry loops

QUICK SUMMARY

- `while` — 0 or more passes
- `repeat-while` — 1 or more passes
- `while let` — drain optionals

Loop Control & Labels

CONTROLLING LOOPS

`break` ends the innermost loop (or `switch`), and `continue` skips to its next pass. To affect an **outer** loop from inside a nested one, give the outer loop a **label** and write `break label` or `continue label`. `return` leaves the whole function.

NOTES

- Labels work on `for`, `while`, `repeat`, `if` and `switch`
- Inside a `switch` within a loop, plain `break` exits the switch
- `forEach` closures can't use `break` / `continue`
- `return` inside `forEach` only skips that element

BREAK · CONTINUE · LABELS

```
for n in 1...10 {
    if n.isMultiple(of: 2) { continue }    // skip evens
    if n > 7 { break }                     // stop at 9
    print(n)                               // 1 3 5 7
}

let grid = [[1, 2], [3, 42], [5, 6]]
search: for (r, row) in grid.enumerated() {
    for (c, value) in row.enumerated() {
        if value == 42 {
            print("found at \(r),\(c)")    // found at 1,1
            break search                   // exits BOTH loops
        }
    }
}

outer: for i in 1...3 {
    for j in 1...3 {
        if j == 2 { continue outer }       // next i
        print(i, j)                        // 1 1, 2 1, 3 1
    }
}
```

CODE EXPLAINED

- → `continue` — next iteration
- → `break` — exit innermost loop
- → `label:` — name a loop
- → `break label` / `continue label` — target an outer loop

THE SWIFTY WAY

✗ a `found` flag checked after the inner loop
✓ a labeled `break` — or move the search into a function and `return`

CONTROL TRANSFER

STATEMENT	EFFECT
`continue`	Next pass of loop
`break`	Leave loop or switch
`break label`	Leave labeled statement
`return`	Leave the function
`fallthrough`	Run next switch case

COMING FROM…

- **Java** `break outer;` ≈ `break outer`
- **Kotlin** `break@outer` ≈ `break outer`
- **Python** No labels; needs a flag or a function

⚠ COMMON MISTAKES

- Using `break` inside a `switch` inside a loop and expecting the loop to stop
- Trying `break` in `forEach`: compile error
- Giving labels vague names like `loop1`

✓ TIPS & BEST PRACTICES

- Prefer extracting nested loops into a function that returns the result
- Use `first(where:)` or `contains(where:)` instead of loop + break
- Name labels after the job: `search:`, `rows:`

QUICK SUMMARY

- `break` / `continue` — innermost loop
- `label:` + `break label` — outer loop
- `first(where:)` — often replaces the loop

Error Handling Basics

HOW SWIFT ERRORS WORK

Swift errors are values of any type that conforms to the `Error` protocol. A function marked `throws` can `throw` one; every call to it must be marked with `try`, and the error is handled in a `do`/`catch` block or passed further up by another `throws` function. Unlike exceptions in many languages, errors are cheap and **always visible** at the call site.

NOTES

- `try` is required on every throwing call: no hidden control flow
- `try?` turns the result into an optional (nil on error)
- `try!` crashes on error: only when failure is truly impossible
- Inside `catch` with no pattern, the error is named `error`

THROW · TRY · DO/CATCH

```swift
enum LoginError: Error { case emptyPassword, locked(minutes: Int) }

func login(_ user: String, _ pass: String) throws -> String {
    guard !pass.isEmpty else { throw LoginError.emptyPassword }
    if user == "bob" { throw LoginError.locked(minutes: 5) }
    return "token-123"
}

do {
    let token = try login("ada", "secret")
    print(token)                                    // token-123
} catch LoginError.locked(let mins) {
    print("Locked for \(mins) min")
} catch {
    print("Failed: \(error)")                       // any other error
}

let t = try? login("ada", "")                       // nil
let sure = try! login("ada", "x")                   // crash if it throws
```

CODE EXPLAINED

- → `: Error` — any type can be an error
- → `throws` — function may throw
- → `try` — mark the throwing call
- → `catch Pattern` — match specific errors
- → `try?` / `try!` — optional / force

THE SWIFTY WAY

✗ `try!` on network or file calls
✓ `do/catch` or `try?` with a fallback — those fail in real life

COMING FROM…

- **Java** Like checked exceptions, but `try` per call
- **Kotlin** Kotlin has no checked errors; Swift does
- **Python** `raise` ≈ `throw`; `except` ≈ `catch`

TRY FORMS

FORM	ON ERROR
`try`	Propagates to caller / catch
`try?`	Returns nil
`try!`	Traps (crash)
`try await`	Async + throwing call

⚠ COMMON MISTAKES

- Using `try?` everywhere: you lose the reason for the failure
- Catching too broadly and printing: handle or rethrow instead
- Forgetting that a `catch` list must be exhaustive unless the enclosing function also throws

✓ TIPS & BEST PRACTICES

- Model each error case with an enum and associated values
- Let errors propagate to a layer that can show a message
- In LLDB, `Swift Error Breakpoint` stops where an error is thrown

QUICK SUMMARY

- `throws` + `throw` — raise an error
- `try` + `do/catch` — handle it
- `try?` / `try!` — optional / crash

Custom Errors

DESIGNING ERROR TYPES

Errors are usually **enums**: one case per failure, with associated values for detail. Conform to `LocalizedError` to provide user-facing text through `errorDescription`, which shows up as `error.localizedDescription`. Structs work too when an error needs many fields.

NOTES

- Group errors per feature or layer: `NetworkError`, `ParseError`
- `LocalizedError` adds `errorDescription`, `recoverySuggestion`
- Make errors `Equatable` to compare them in tests
- Wrap low-level errors in an `underlying: Error` case

CUSTOM ERRORS

```
enum NetworkError: Error, LocalizedError, Equatable {
    case offline
    case badStatus(code: Int)
    case decoding(String)

    var errorDescription: String? {
        switch self {
        case .offline: "You're offline."
        case .badStatus(let c): "Server returned \(c)."
        case .decoding(let field): "Couldn't read \(field)."
        }
    }
}

do {
    throw NetworkError.badStatus(code: 503)
} catch let e as NetworkError {
    print(e.localizedDescription)          // Server returned 503.
} catch {
    print(error)
}

struct ValidationError: Error { let field: String; let reason: String }
```

CODE EXPLAINED

- → `enum … : Error` — one case per failure
- → associated values — carry details
- → `errorDescription` — user-facing message
- → `catch let e as NetworkError` — typed catch
- → struct errors — when you need fields

THE SWIFTY WAY

✗ `throw NSError(domain: "x", code: 1)`
✓ `throw MyError.specificCase` — type-safe and matchable

COMING FROM…

- **Java** Custom exception class ≈ `enum` case here
- **Kotlin** Sealed class of errors ≈ error enum
- **Python** Exception subclasses ≈ enum cases

ERROR PROTOCOLS

PROTOCOL	ADDS
`Error`	Throwable
`LocalizedError`	errorDescription, failureReason
`CustomNSError`	errorDomain, errorCode
`Equatable`	Compare in tests

⚠ COMMON MISTAKES

- Throwing strings or Ints: they aren't `Error` (unless you extend them, which you shouldn't)
- Showing `"\(error)"` to users: it's a debug dump; use `localizedDescription`
- One giant app-wide error enum with 50 cases

✓ TIPS & BEST PRACTICES

- Add `case underlying(any Error)` to wrap system errors
- Implement `LocalizedError` so SwiftUI `.alert` text is ready to use
- Test with `#expect(throws: NetworkError.offline) { … }`

QUICK SUMMARY

- `enum X: Error` — idiomatic error type
- `LocalizedError` — user text
- `catch let e as X` — handle a type

Typed Throws & Result

TYPED ERRORS

Since Swift 6, a function can declare **exactly** which error type it throws: `throws(ParseError)`. Callers then get a concrete error type in `catch`, without casting. `Result<Success, Failure>` stores a success or an error as a value, which is handy for callbacks and caching. `rethrows` marks a function that throws only if its closure argument does.

NOTES

- Plain `throws` is the same as `throws(any Error)`
- Use typed throws for closed, internal error sets
- `Result.get()` converts back into a throwing call
- `throws(Never)` means the function never throws

TYPED THROWS & RESULT

```
enum ParseError: Error { case empty, notNumber(String) }

func parse(_ s: String) throws(ParseError) -> Int {
    guard !s.isEmpty else { throw .empty }      // type inferred
    guard let n = Int(s) else { throw .notNumber(s) }
    return n
}

do {
    let n = try parse("x")
} catch {                                       // error: ParseError
    switch error {
    case .empty: print("empty")
    case .notNumber(let s): print("bad: \(s)")  // bad: x
    }
}

let r: Result<Int, ParseError> = Result { try parse("42") }
switch r {
case .success(let n): print(n)                  // 42
case .failure(let e): print(e)
}
let value = try r.get()                         // 42
r.map { $0 * 2 }                                // .success(84)
```

CODE EXPLAINED

- → `throws(ParseError)` — only this error type
- → `throw .empty` — case inferred from the declared type
- → `catch` — `error` is `ParseError`, not `any Error`
- → `Result { try … }` — capture a throwing call
- → `get()` / `map` / `mapError` — use a Result

THE SWIFTY WAY

✗ `completion: (Data?, Error?) -> Void`
✓ `async throws -> Data` (or `Result<Data, Error>`)

COMING FROM…

- **Kotlin** `Result<T>` ≈ `Result` (error type fixed to Throwable there)
- **TypeScript** No typed errors; Swift checks them
- **Java** `throws IOException` ≈ `throws(IOError)`

SWIFT VERSIONS

- 6.0 — typed throws `throws(E)` (SE-0413)
- 5.0 — `Result` in the standard library

RESULT API

MEMBER	DOES
`.success(v) / .failure(e)`	Cases
`get()`	Return value or throw
`map / flatMap`	Transform success
`mapError`	Transform failure

⚠ COMMON MISTAKES

- Using typed throws on public library APIs: adding an error case later breaks clients
- Building new APIs on completion-handler `Result` when `async throws` fits
- Forgetting `rethrows` on a helper that takes a throwing closure

✓ TIPS & BEST PRACTICES

- Prefer untyped `throws` at module boundaries, typed `throws(E)` inside
- Generic code can use `throws(E)` to pass an error type through
- `Result` is great for storing outcomes (caches, test doubles)

QUICK SUMMARY

- `throws(E)` — typed throws (6.0)
- `Result<T, E>` — error as a value
- `rethrows` — throws only if the closure does

defer & Cleanup

RUNNING CODE ON EXIT

A `defer` block runs when the current scope exits, **however** it exits: normal end, `return`, `throw` or `break`. Put cleanup right next to the setup it undoes, so you can't forget it on an early-exit path. Multiple `defer`s in one scope run in **reverse** order.

NOTES

- `defer` runs at the end of its enclosing scope, not the function
- A `defer` inside an `if` or loop body runs at the end of that body
- Can't `return`, `throw` or `break` out of a defer
- Ideal for locks, files, timers and state flags

DEFER FOR CLEANUP

```
func load(path: String) throws -> String {
    let handle = try FileHandle(forReadingFrom: URL(filePath: path))
    defer { try? handle.close() }           // runs on every exit

    guard let data = try handle.readToEnd() else { return "" }
    return String(decoding: data, as: UTF8.self)
}

func demo() {
    defer { print("1st defer") }
    defer { print("2nd defer") }
    print("body")
}
demo()                      // body, 2nd defer, 1st defer

@MainActor func refresh() async {
    isLoading = true
    defer { isLoading = false }             // reset even if we bail out
    guard let data = try? await api.fetch() else { return }
    show(data)
}
```

CODE EXPLAINED

→ `defer { }` — runs when the scope exits

→ reverse order — last declared runs first

→ `try?` inside defer — defer can't throw

→ flag reset — classic loading-state pattern

THE SWIFTY WAY

✗ calling `close()` before every `return`

✓ one `defer { close() }` right after opening

COMING FROM…

- **Java** `finally` ≈ `defer` placed next to setup
- **Python** `try/finally` or `with` ≈ `defer`
- **TypeScript** `try/finally` ≈ `defer`

COMMON PAIRS

SETUP	DEFER
`lock.lock()`	`lock.unlock()`
`open file`	`handle.close()`
`isLoading = true`	`isLoading = false`
`startAccessing…()`	`stopAccessing…()`

⚠ COMMON MISTAKES

- Placing `defer` at the end of a function: it must come before the exits it should cover
- Expecting a defer inside an `if` to wait until the function returns
- Calling a throwing function in defer without `try?`

✓ TIPS & BEST PRACTICES

- Write the `defer` immediately after acquiring the resource
- Prefer `Mutex.withLock { }` over manual lock/unlock + defer when available
- Keep defer bodies short and side-effect free

QUICK SUMMARY

- `defer` — run on scope exit
- Runs on return, throw, break
- Multiple defers run in reverse

Assertions & Preconditions

CHECKING ASSUMPTIONS

These functions stop the program when something that "can't happen" does. `assert` checks only in **debug** builds; `precondition` also checks in **release** builds; `fatalError` always stops. Use them for programmer mistakes, not for recoverable failures like bad user input (throw an error for those).

NOTES

- `-O` builds skip `assert` entirely
- `-Ounchecked` builds skip `precondition` too
- `fatalError` returns `Never`, so it satisfies any return type
- Array out-of-bounds and force-unwrap nil are built-in preconditions

ASSERT · PRECONDITION · FATALERROR

```
func setVolume(_ v: Int) {
    assert((0...100).contains(v), "volume out of range: \(v)")
    // debug: stops here   release: keeps going
}

func withdraw(_ amount: Int, from balance: Int) -> Int {
    precondition(amount > 0, "amount must be positive")
    return balance - amount            // checked in release too
}

func cell(for row: Int) -> Cell {
    guard let cell = table.dequeue(row) else {
        fatalError("Cell not registered for row \(row)")
    }
    return cell
}

required init?(coder: NSCoder) {
    fatalError("init(coder:) not supported")
}
```

CODE EXPLAINED

- → `assert(cond, msg)` — debug-only check
- → `precondition(cond, msg)` — debug + release
- → `fatalError(msg)` — always stops, returns Never
- → `assertionFailure` / `preconditionFailure` — no condition

THE SWIFTY WAY

✗ `precondition(!email.isEmpty)` on user input
✓ `throw ValidationError.emptyEmail` — users make mistakes; code shouldn't crash

COMING FROM…

- **Java** `assert` ≈ `assert` (off by default there)
- **Kotlin** `require` ≈ `precondition`; `check` ≈ `assert`
- **Python** `assert` ≈ `assert`; `-O` disables both

WHEN EACH RUNS

FUNCTION	DEBUG	RELEASE
assert	✔	✖
assertionFailure	✔	✖
precondition	✔	✔
preconditionFailure	✔	✔
fatalError	✔	✔

⚠ COMMON MISTAKES

- Putting side effects inside `assert(...)`: they vanish in release builds
- Using `fatalError` for network or parsing failures
- Leaving `fatalError("TODO")` in shipped code paths

✓ TIPS & BEST PRACTICES

- Write messages that include the bad value
- Use `precondition` at public API boundaries in libraries
- Search for `fatalError(` before every release

QUICK SUMMARY

- `assert` — debug only
- `precondition` — always (except -Ounchecked)
- `fatalError` — always, returns Never

Defining & Calling Functions

FUNCTIONS

Declare a function with `func`, a parameter list, and an optional return type after `->`. A function with no `->` returns `Void`. If the body is a **single expression**, the `return` keyword can be dropped. Functions can return tuples to hand back several values at once.

NOTES

- Functions are first-class values (Sheet 044)
- Callers must use the result unless it's `@discardableResult`
- Return type `Never` means the function never returns
- Functions can be declared at file scope, in types, or nested

DEFINING & CALLING

```
func greet(name: String) -> String {
    return "Hello, \(name)!"
}
greet(name: "Ada")                          // "Hello, Ada!"

func square(_ x: Int) -> Int { x * x }      // implicit return
square(4)                                   // 16

func log(_ message: String) {               // returns Void
    print("[log] \(message)")
}

func minMax(_ a: [Int]) -> (min: Int, max: Int)? {
    guard let lo = a.min(), let hi = a.max() else { return nil }
    return (lo, hi)
}
if let r = minMax([3, 9, 1]) { print(r.min, r.max) }   // 1 9

@discardableResult
func save() -> Bool { true }
save()                                      // no unused-result warning
```

CODE EXPLAINED

- → `func name(param: Type) -> Ret` — declaration
- → implicit return — single-expression body
- → no `->` — returns `Void`
- → named tuple return — `(min: Int, max: Int)`
- → `@discardableResult` — result may be ignored

THE SWIFTY WAY

✗ `func f() -> Int { return x * 2 }`

✓ `func f() -> Int { x * 2 }`

COMING FROM…

- **Kotlin** `fun f(x: Int): Int = x * x` ≈ implicit return
- **TypeScript** `function f(x: number): number`
- **Python** `def f(x) -> int:` with tuple returns

FUNCTION SHAPES

SIGNATURE	MEANING
`func f()`	No params, returns Void
`func f() -> Int`	Returns Int
`func f() -> (Int, Int)`	Returns a tuple
`func f() -> Int?`	May return nil
`func f() -> Never`	Never returns

⚠ COMMON MISTAKES

- Ignoring a result: compiler warning unless `@discardableResult`
- Returning an unnamed tuple from a public API: use a struct when it grows
- Writing `-> Void` everywhere: just omit the arrow

✓ TIPS & BEST PRACTICES

- Keep functions small; extract when you'd write a comment heading
- Return optionals or throw rather than sentinel values like `-1`
- Use `_ = f()` to ignore a result on purpose

QUICK SUMMARY

- `func f(x: T) -> R` — declare
- implicit return — one expression
- tuple returns — multiple values

Argument Labels & Parameters

LABELS AND PARAMETERS

Each parameter has an **argument label** (used at the call site) and a **parameter name** (used in the body). By default they're the same; write two names to make them differ, or _ to drop the label. Parameters are constants; mark one `inout` to let the function change the caller's variable, passed with &.

NOTES

- Labels make calls read like sentences: `move(from: a, to: b)`
- Default values let callers skip arguments
- Parameters with defaults usually go last
- `inout` can't take a `let` or a literal

LABELS, DEFAULTS, INOUT

```
func move(from start: Int, to end: Int) -> Int { end - start }
move(from: 2, to: 9)                          // 7

func square(_ x: Int) -> Int { x * x }        // no label
square(5)                                     // 25

func greet(_ name: String, greeting: String = "Hello",
           punctuation: Character = "!") -> String {
    "\(greeting), \(name)\(punctuation)"
}
greet("Ada")                                  // "Hello, Ada!"
greet("Ada", greeting: "Hi")                  // "Hi, Ada!"

func doubleInPlace(_ n: inout Int) { n *= 2 }
var score = 21
doubleInPlace(&score)                         // score == 42

// x = 5    // error: parameters are let constants
```

CODE EXPLAINED

- → `from start:` — label `from`, name `start`
- → `_ x:` — no label at the call site
- → `= "Hello"` — default argument
- → `inout` + `&` — modify the caller's var

THE SWIFTY WAY

✗ `func add(a: Int, b: Int)`
✓ `func add(_ a: Int, to b: Int)` — read the call aloud: `add(3, to: 5)`

COMING FROM…

- **Kotlin** Named args optional there; labels required here
- **TypeScript** Options object `{from, to}` ≈ labels
- **Python** Keyword args ≈ labels, but order is fixed here

DECLARATION → CALL

DECLARATION	CALL
`f(x: Int)`	`f(x: 1)`
`f(_ x: Int)`	`f(1)`
`f(at x: Int)`	`f(at: 1)`
`f(x: Int = 0)`	`f() or f(x: 1)`
`f(_ x: inout Int)`	`f(&n)`

⚠ COMMON MISTAKES

- Passing labeled arguments in a different order: order is fixed
- Forgetting `&` when calling an `inout` function
- Using `inout` to return values instead of returning a tuple

✓ TIPS & BEST PRACTICES

- Follow the API Design Guidelines at swift.org for label naming
- Omit the first label when it would repeat the function name: `remove(item)`
- Prefer returning a new value over `inout` unless mutating in place matters

QUICK SUMMARY

- `label name: Type` — two names
- `_` — unlabeled argument
- `inout` / `&` — mutate caller's var

Variadics, Overloading & Function Types

VARIADICS, OVERLOADS, FUNCTION TYPES

A **variadic** parameter (`Int...`) accepts zero or more values, received as an array. Functions can be **overloaded** by parameter types or labels. Every function has a **function type** like `(Int, Int) -> Int`, so functions can be stored in variables, passed as arguments and returned.

NOTES

- A function may have several variadic parameters (5.4+)
- You can't pass an array directly to a variadic parameter
- Overloads that differ only by return type confuse inference
- Function types are what closures conform to

VARIADICS & FUNCTION VALUES

```
func sum(_ nums: Int...) -> Int { nums.reduce(0, +) }
sum()                                   // 0
sum(1, 2, 3)                            // 6

func area(of side: Double) -> Double { side * side }
func area(width: Double, height: Double) -> Double { width * height }
area(of: 3)                             // 9.0  (overload by label)

func add(_ a: Int, _ b: Int) -> Int { a + b }
func mul(_ a: Int, _ b: Int) -> Int { a * b }

var op: (Int, Int) -> Int = add         // store a function
op(3, 4)                                // 7
op = mul
op(3, 4)                                // 12

func apply(_ f: (Int, Int) -> Int, _ x: Int, _ y: Int) -> Int { f(x, y) }
apply(+, 2, 5)                          // 7 (operators are functions)

func makeAdder(_ n: Int) -> (Int) -> Int { { $0 + n } }
makeAdder(10)(5)                        // 15
```

FUNCTION TYPES

TYPE	EXAMPLE
`() -> Void`	func refresh()
`(String) -> Int`	func count(_ s: String)
`(Int) throws -> Int`	throwing function
`(Int) async -> Int`	async function

CODE EXPLAINED

- → `Int...` — variadic, arrives as `[Int]`
- → overloads — same name, different labels/types
- → `(Int, Int) -> Int` — function type
- → `apply(+, …)` — operators are functions
- → `-> (Int) -> Int` — return a function

THE SWIFTY WAY

✗ `func sum(_ a: [Int])` + `func sum(_ a: Int...)`
✓ one variadic version that forwards

COMING FROM…

- **Kotlin** `vararg nums: Int` ≈ `Int...`
- **TypeScript** `...nums: number[]` ≈ `Int...`
- **Python** `*args` ≈ variadic; no overloads there

SWIFT VERSIONS

- 5.4 — multiple variadic parameters
- 5.9 — parameter packs `each T` for variadic generics

IN SWIFTUI

- `ViewBuilder` closures are function types: `() -> some View`
- See SwiftUI O3 · Views & body

⚠ COMMON MISTAKES

- Passing `[1, 2, 3]` to an `Int...` parameter: type mismatch
- Overloading on return type alone: callers need annotations everywhere
- Storing a function that captures `self` strongly in a property: retain cycle

✓ TIPS & BEST PRACTICES

- Use a typealias for function types used in several places
- Pass operators directly: `reduce(0, +)`, `sorted(by: <)`
- Prefer overloads by label over `Any` parameters

QUICK SUMMARY

- `T...` — variadic
- Overloads by label or type
- `(A) -> B` — functions are values

Closures — Basics

CLOSURE SYNTAX

A closure is an unnamed function written inline: `{ (params) -> Return in body }`. Swift lets you shrink it step by step: infer types, drop `return` for one expression, use `$0` shorthand, and move it outside the parentheses as a **trailing closure**. Trailing closures are why Swift APIs (and SwiftUI) read like blocks.

NOTES

- If the closure is the only argument, drop the `()` too
- Multiple trailing closures: the first is unlabeled, the rest labeled
- `$0`, `$1` … refer to parameters by position
- Use named parameters when the closure is longer than a line

FROM FULL TO SHORT

```
let names = ["Chris", "Alex", "Ewa"]

names.sorted(by: { (a: String, b: String) -> Bool in
    return a < b
})
names.sorted(by: { a, b in a < b })          // inferred types
names.sorted { $0 < $1 }                     // trailing + shorthand
names.sorted(by: <)                          // operator as function

let shout = { (s: String) -> String in s.uppercased() + "!" }
shout("hi")                                  // "HI!"

names.map { $0.count }                       // [5, 4, 3]

UIView.animate(withDuration: 0.3) {          // multiple trailing
    view.alpha = 0
} completion: { _ in
    view.removeFromSuperview()
}
```

CODE EXPLAINED

- → `{ (a, b) -> Bool in … }` — full syntax
- → `in` — separates the signature from the body
- → `$0` `$1` — positional shorthand
- → trailing closure — after `)` or instead of `()`
- → `completion:` — label for later trailing closures

THE SWIFTY WAY

✗ `items.map({ (item: Item) -> String in return item.name })`
✓ `items.map(\.name)` — key paths work as functions

COMING FROM…

- **Kotlin** Trailing lambdas `list.map { it * 2 }` ≈ `{ $0 * 2 }`
- **TypeScript** `(a, b) => a < b` ≈ `{ a, b in a < b }`
- **Python** `lambda x: x * 2` ≈ `{ $0 * 2 }`

SHORTENING STEPS

STEP	FORM
Full	`{ (x: Int) -> Int in return x * 2 }`
Infer types	`{ x in return x * 2 }`
Implicit return	`{ x in x * 2 }`
Shorthand	`{ $0 * 2 }`

SWIFT VERSIONS

- 5.3 — multiple trailing closures

IN SWIFTUI

- `Button("Save") { save() }` — action is a trailing closure
- `VStack { … }` — content is a trailing `@ViewBuilder` closure
- See SwiftUI 20 · Buttons

⚠ COMMON MISTAKES

- Using `$0` / `$1` in multi-line closures: hard to follow; name them
- Trailing closure in an `if` condition: wrap the call in parentheses
- Mixing `$0` with named parameters in one closure: not allowed

✓ TIPS & BEST PRACTICES

- Read `in` as "where the body begins"
- Annotate the closure's type when inference produces confusing errors
- Prefer key paths (`\.name`) for simple property access

QUICK SUMMARY

- `{ params in body }` — closure
- `$0` — shorthand parameter
- trailing closures — cleaner calls

Closures — Capturing & Escaping

CAPTURING & ESCAPING

Closures **capture** the variables they use, keeping them alive. Capturing `self` strongly in a closure that `self` also holds creates a **retain cycle**, so use a capture list like `[weak self]`. A closure parameter that's stored or called later must be marked `@escaping`; non-escaping is the default and is cheaper.

NOTES

- Captured `var` s are shared by reference, not copied
- Capture lists copy values at creation: `[count]`
- `@escaping` closures must use `self.` explicitly (for classes)
- `@autoclosure` wraps an argument expression in a closure

CAPTURE LISTS & @ESCAPING

```
func makeCounter() -> () -> Int {
    var count = 0
    return { count += 1; return count }      // captures count
}
let next = makeCounter()
next(); next()                               // 1, 2

var x = 1
let byValue = { [x] in print(x) }            // copies 1 now
x = 99
byValue()                                    // 1

final class Loader {
    var onDone: (() -> Void)?
    func start(completion: @escaping () -> Void) {
        onDone = completion                  // stored → escaping
    }
    func setup() {
        start { [weak self] in
            guard let self else { return }
            self.finish()                    // no retain cycle
        }
    }
    func finish() {}
}
```

CODE EXPLAINED

- → capture — closure keeps `count` alive
- → `[x]` — capture list copies the current value
- → `@escaping` — closure outlives the call
- → `[weak self]` — break the self ↔ closure cycle
- → `guard let self` — strong self for the body

THE SWIFTY WAY

✗ `[unowned self]` by default

✓ `[weak self]` + `guard let self` — unowned crashes if self is gone

COMING FROM…

- **Kotlin** No cycles under GC; Swift's ARC needs `[weak self]`
- **TypeScript** Closures capture by reference too; no leaks under GC
- **Java** Lambdas need effectively-final vars; Swift allows mutation

CAPTURE OPTIONS

LIST	EFFECT
`[weak self]`	Optional self, no retain
`[unowned self]`	No retain, crash if freed
`[x]`	Copy x at creation
`[weak d = delegate]`	Rename + weak

IN SWIFTUI

- SwiftUI views are structs: `[weak self]` isn't needed inside `body`
- Do use `[weak self]` in long-lived Tasks in `@Observable` models
- See SwiftUI 16 · @Observable Models

⚠ COMMON MISTAKES

- Storing a closure that strongly captures `self` in `self` : memory leak
- Forgetting `@escaping` when storing a parameter: compile error
- Expecting a captured `var` to be frozen: it's shared unless in a capture list

✓ TIPS & BEST PRACTICES

- Use Xcode's Memory Graph Debugger to find closure retain cycles
- Non-escaping closures (e.g. `map`) never need `[weak self]`
- `Task { }` captures self strongly; cancel long tasks in `deinit` or use weak

QUICK SUMMARY

- closures capture by reference
- `[weak self]` — avoid cycles
- `@escaping` — closure stored or run later

Higher-Order Functions

HIGHER-ORDER FUNCTIONS

Collections have methods that take closures to transform data without manual loops. `map` transforms each element, `filter` keeps matches, `reduce` combines into one value, and `compactMap` transforms and drops `nil`s. They return new arrays and can be chained into readable pipelines.

NOTES

- `map` on a Dictionary returns an Array; use `mapValues` to keep it a Dictionary
- `flatMap` flattens nested sequences
- `reduce(into:)` is faster for building collections
- Add `.lazy` to avoid intermediate arrays on long chains

MAP · FILTER · REDUCE

```
let nums = [1, 2, 3, 4, 5, 6]

nums.map { $0 * $0 }                          // [1, 4, 9, 16, 25, 36]
nums.filter { $0.isMultiple(of: 2) }          // [2, 4, 6]
nums.reduce(0, +)                             // 21
nums.reduce(into: [:]) { $0[$1 % 3, default: 0] += 1 }
                                              // [1: 2, 2: 2, 0: 2]

["1", "x", "3"].compactMap { Int($0) }        // [1, 3]
[[1, 2], [3]].flatMap { $0 }                  // [1, 2, 3]
nums.contains { $0 > 5 }                      // true
nums.allSatisfy { $0 > 0 }                    // true
nums.first { $0 > 3 }                         // Optional(4)
nums.count(where: { $0 > 3 })                 // 3    (6.0+)

let total = orders
    .filter { $0.isPaid }
    .map(\.amount)
    .reduce(0, +)
```

CODE EXPLAINED

- → `map` — transform every element
- → `filter` — keep elements that pass
- → `reduce` — fold into one value
- → `compactMap` — transform + drop nil
- → `flatMap` — flatten one level

THE SWIFTY WAY

✗ `var r = []; for x in a { if p(x) { r.append(f(x)) } }`

✓ `a.filter(p).map(f)`

COMING FROM…

- **TypeScript** Same names; `compactMap` ≈ `.map().filter(Boolean)`
- **Kotlin** `mapNotNull` ≈ `compactMap`; `fold` ≈ `reduce`
- **Python** List comprehensions ≈ map + filter

MORE HELPERS

METHOD	RETURNS
`forEach`	Void (no break)
`sorted(by:)`	Sorted array
`min(by:) / max(by:)`	Optional element
`first(where:)`	Optional element
`count(where:)`	Int (6.0+)

⚠ COMMON MISTAKES

- Using `map` for side effects: use `for-in` or `forEach`
- `filter { … }.count` on large arrays: use `count(where:)`
- `flatMap` to drop nils: that's `compactMap` (old flatMap overload was renamed)

✓ TIPS & BEST PRACTICES

- Use key paths: `users.map(\.name)`
- `reduce(into:)` mutates an accumulator, avoiding copies
- Break long chains into named `let` steps when debugging

QUICK SUMMARY

- `map` / `filter` / `reduce` — core trio
- `compactMap` — drop nils
- `.lazy` — no intermediate arrays

Nested Functions & Scope

NESTING AND SCOPE

Functions can be declared **inside** other functions. A nested function is visible only in its enclosing function and can use (capture) the outer function's local variables. Every pair of braces creates a new scope; names declared inside are not visible outside, and an inner name can shadow an outer one.

NOTES

- Nested functions keep helpers private to one function
- They can capture and modify outer `var`s
- Global functions live at file scope in a module
- `do { }` creates a scope without any condition

NESTED FUNCTIONS

```
func validate(_ form: [String: String]) -> [String] {
    var errors: [String] = []

    func require(_ key: String) {              // nested helper
        if form[key, default: ""].isEmpty {
            errors.append("\(key) is required")   // captures errors
        }
    }

    require("name")
    require("email")
    return errors
}
validate(["name": "Ada"])                      // ["email is required"]

let level = "global"
func show() {
    let level = "local"                        // shadows the outer one
    print(level)                               // local
    do {
        let temp = 42                          // only lives in do { }
        print(temp)
    }
    // print(temp)                             // error: not in scope
}
```

CODE EXPLAINED

→ nested `func` — private helper

→ captures `errors` — can mutate outer locals

→ shadowing — inner `level` hides outer

→ `do { }` — bare scope block

THE SWIFTY WAY

✗ a private top-level helper used by one function
✓ nest it inside that function

SCOPE LEVELS

LEVEL	VISIBLE IN
`Block { }`	That block only
`Function`	Function body
`Type member`	Type + extensions (per access)
`File (private)`	That file
`Module (internal)`	Whole target

COMING FROM…

- **Kotlin** Local functions work the same
- **Python** Nested `def` with `nonlocal` ≈ captured var
- **TypeScript** Inner functions / closures, same rules

⚠ COMMON MISTAKES

- Calling a nested function before it's declared and expecting it to see later locals
- Accidentally shadowing a property with a local of the same name
- Deep nesting that hides what the outer function does

✓ TIPS & BEST PRACTICES

- Use nested functions for recursion helpers with extra parameters
- Shadowing in `if let name` is idiomatic; elsewhere pick new names
- Extract to a private method once a nested helper grows large

QUICK SUMMARY

- nested `func` — local helpers
- captures outer locals
- braces = scope

Recursion

RECURSION

A recursive function calls itself on a smaller input until it reaches a **base case**. Swift handles recursion fine for moderate depths, but it doesn't guarantee tail-call optimization, so very deep recursion can overflow the stack. **Indirect enums** let you define recursive data types such as trees and expressions.

NOTES

- Always write the base case first
- Main thread stack: 8 MB on macOS, 1 MB on iOS; others 512 KB
- Memoize repeated subproblems with a dictionary
- `indirect` lets an enum case contain its own type

RECURSIVE FUNCTIONS & DATA

```
func factorial(_ n: Int) -> Int {
    n <= 1 ? 1 : n * factorial(n - 1)        // base case: n <= 1
}
factorial(5)                                 // 120

var memo: [Int: Int] = [:]
func fib(_ n: Int) -> Int {
    if n < 2 { return n }
    if let cached = memo[n] { return cached }
    let r = fib(n - 1) + fib(n - 2)
    memo[n] = r
    return r
}
fib(50)                                      // 12586269025

indirect enum Expr {
    case num(Int)
    case add(Expr, Expr)
    case mul(Expr, Expr)
}
func eval(_ e: Expr) -> Int {
    switch e {
    case .num(let n): n
    case .add(let a, let b): eval(a) + eval(b)
    case .mul(let a, let b): eval(a) * eval(b)
    }
}
eval(.add(.num(2), .mul(.num(3), .num(4))))  // 14
```

CODE EXPLAINED

- → base case — stops the recursion
- → `memo` — cache results to avoid exponential work
- → `indirect enum` — recursive data type
- → `switch` + recursion — walk the structure

THE SWIFTY WAY

✗ naive `fib(n-1) + fib(n-2)`
✓ memoize, or iterate with two vars — naive fib is O(2ⁿ)

COMING FROM…

- **Kotlin** `tailrec` exists there; no Swift equivalent
- **Python** Recursion limit ~1000; Swift limit is stack size
- **TypeScript** Same: no reliable tail calls

RECURSION VS ITERATION

APPROACH	TRADE-OFF
Recursion	Clear for trees, risks deep stacks
Iteration	Constant stack, more bookkeeping
Memoized	Fast, uses memory
Explicit stack	Recursion without call stack

⚠ COMMON MISTAKES

- Missing or unreachable base case: stack overflow (EXC_BAD_ACCESS)
- Recursing over a 100k-element list: use a loop
- Forgetting `indirect`: "recursive enum is not marked indirect"

✓ TIPS & BEST PRACTICES

- Convert deep recursion to a loop with an explicit `[Node]` stack
- Use a nested helper function to hide accumulator parameters
- `indirect` can mark just one case instead of the whole enum

QUICK SUMMARY

- base case + smaller subproblem
- memoize repeated work
- `indirect enum` — recursive types

Result Builders

BUILDING DSLS

A **result builder** turns a closure full of plain statements into a single value by calling static `build…` methods behind the scenes. This is the feature behind SwiftUI's `@ViewBuilder`, RegexBuilder and Swift Testing traits. You annotate a type with `@resultBuilder`, then use it as an attribute on closure parameters or properties.

NOTES

- Each line in the closure becomes an argument to `buildBlock`
- `buildOptional` enables `if` without `else`
- `buildEither(first:)` / `(second:)` enable `if/else` and `switch`
- `buildArray` enables `for` loops

A TINY HTML BUILDER

```
@resultBuilder
struct ListBuilder {
    static func buildBlock(_ parts: String...) -> [String] { parts }
    static func buildOptional(_ p: [String]?) -> String { p?.joined() ?? "" }
    static func buildEither(first p: [String]) -> String { p.joined() }
    static func buildEither(second p: [String]) -> String { p.joined() }
}

func ul(@ListBuilder _ items: () -> [String]) -> String {
    "<ul>" + items().map { "<li>\($0)</li>" }.joined() + "</ul>"
}

let loggedIn = true
let html = ul {
    "Home"
    "About"
    if loggedIn { "Account" } else { "Sign in" }
}
// <ul><li>Home</li><li>About</li><li>Account</li></ul>
```

CODE EXPLAINED

- → `@resultBuilder` — marks the builder type
- → `buildBlock` — combines the lines
- → `buildEither` — supports if/else
- → `@ListBuilder` — applied to a closure parameter

THE SWIFTY WAY

✗ returning `[String]` built with `append` in every branch

✓ a result builder for declarative content — only when callers write lots of it

COMING FROM…

- **Kotlin** Type-safe builders (lambdas with receivers)
- **TypeScript** JSX ≈ result builders for UI
- **Python** No equivalent; context managers are closest

BUILDER METHODS

METHOD	ENABLES
`buildBlock`	Sequence of lines
`buildOptional`	if without else
`buildEither`	if/else, switch
`buildArray`	for loops
`buildExpression`	Convert each line

SWIFT VERSIONS

- 5.4 — result builders (SE-0289)
- 5.7 — `buildPartialBlock` for simpler builders

IN SWIFTUI

- `body` is implicitly `@ViewBuilder`
- Mark your own `content` parameters `@ViewBuilder content: () -> Content`
- See SwiftUI O3 · Views & body

⚠ COMMON MISTAKES

- Declaring variables with `let` inside an old builder: some builders don't allow it
- Expecting `guard` or `return` in a builder closure: not supported
- Building a DSL for something a plain array literal would do

✓ TIPS & BEST PRACTICES

- Read SwiftUI compiler errors with result builders in mind: they point at `buildBlock`
- Implement `buildPartialBlock` instead of many `buildBlock` overloads
- Mark helper functions returning views `@ViewBuilder` to use `if` inside

QUICK SUMMARY

- `@resultBuilder` — declarative DSLs
- `build…` methods — map statements to a value
- Powers `@ViewBuilder`, RegexBuilder

Arrays — Basics

WORKING WITH ARRAYS

`Array<Element>`, written `[Element]`, is an ordered, zero-indexed collection and a **value type**: assigning it copies it (lazily, via copy-on-write). A `let` array is fully immutable. Subscripting out of range **traps**, so check `indices` or use `first`/`last`, which return optionals.

NOTES

- All elements share one type (use `[any P]` for mixed types)
- `append` is amortized O(1); `insert(at: 0)` is O(n)
- `isEmpty` beats `count == 0`
- `first` / `last` are optional; `[0]` crashes on empty

CREATE, READ, CHANGE

```
var fruits = ["apple", "banana"]          // [String]
var empty: [Int] = []
let zeros = Array(repeating: 0, count: 3) // [0, 0, 0]

fruits.append("cherry")                   // [apple, banana, cherry]
fruits += ["date", "elder"]
fruits.insert("avocado", at: 1)
fruits[0] = "apricot"
fruits.remove(at: 2)                      // returns "banana"
fruits.removeLast()                       // returns "elder"

fruits.count                              // 4
fruits.first                              // Optional("apricot")
fruits.indices.contains(10)               // false
fruits.firstIndex(of: "cherry")           // Optional(2)

var copy = fruits                         // independent copy
copy.append("fig")
fruits.count                              // still 4
```

CODE EXPLAINED

- → `[String]` — array type shorthand
- → `Array(repeating:count:)` — prefilled
- → `append` / `insert(_:at:)` / `remove(at:)`
- → `first` / `last` — safe optional access
- → `copy = fruits` — value semantics

THE SWIFTY WAY

✗ `if arr.count > 0 { arr[0] }`
✓ `if let first = arr.first`

COMING FROM…

- **Kotlin** `mutableListOf` ≈ `var [T]`; `listOf` ≈ `let`
- **TypeScript** Arrays are values here, not shared refs
- **Java** `ArrayList<T>` ≈ `[T]`, but copied on assign

CORE MEMBERS

MEMBER	COMPLEXITY
`append(_:)`	O(1) amortized
`insert(_:at:)`	O(n)
`remove(at:)`	O(n)
`removeLast()`	O(1)
`contains(_:)`	O(n)
`[i]`	O(1)

⚠ COMMON MISTAKES

- `arr[arr.count]` or `arr[0]` on empty: index out of range crash
- Expecting two variables to share one array: arrays are values
- `removeLast()` on an empty array: crash; use `popLast()`

✓ TIPS & BEST PRACTICES

- `reserveCapacity(n)` before appending many items
- `arr.indices.contains(i) ? arr[i] : nil` for safe lookup
- Use `ContiguousArray` only after profiling

QUICK SUMMARY

- `[T]` — ordered value-type list
- `first` / `last` — optional access
- `append` / `insert` / `remove`

Arrays — Methods

ARRAY METHODS

Arrays inherit dozens of methods from `Sequence` and `Collection`. Methods ending in **-ed** or **-ing** (`sorted`, `reversed`, `shuffled`) return a new array; their imperative twins (`sort`, `reverse`, `shuffle`) mutate a `var` in place. Search methods return optionals.

NOTES

- `sort()` needs `var`; `sorted()` works on `let`
- Elements must be `Comparable` for `sort()` without arguments
- `Equatable` elements unlock `contains(_:)` and `firstIndex(of:)`
- `removeAll(where:)` is O(n) and safe while filtering

COMMON METHODS

```
var n = [5, 3, 8, 1, 9, 3]

n.sorted()                              // [1, 3, 3, 5, 8, 9]
n.sorted(by: >)                         // [9, 8, 5, 3, 3, 1]
n.sort()                                // mutates n
n.reversed()                            // ReversedCollection
Array(n.reversed())                     // [9, 8, 5, 3, 3, 1]

n.contains(8)                           // true
n.firstIndex(of: 3)                     // Optional(1)
n.lastIndex(of: 3)                      // Optional(2)
n.first(where: { $0 > 4 })              // Optional(5)
n.min(); n.max()                        // Optional(1), Optional(9)

n.removeAll { $0 == 3 }                 // [1, 5, 8, 9]
n.shuffle()
n.randomElement()                       // Optional(...)
n.swapAt(0, 1)
Array(Set([1, 1, 2]))                   // unique values
```

CODE EXPLAINED

- → `sorted` / `sort` — copy vs in place
- → `sorted(by: >)` — descending
- → `firstIndex(of:)` / `first(where:)` — search
- → `removeAll(where:)` — filter in place
- → `randomElement()` — optional random pick

THE SWIFTY WAY

✗ `for (i, x) in a.enumerated().reversed() { if bad(x) { a.remove(at: i) } }`
✓ `a.removeAll(where: bad)`

COMING FROM…

- **TypeScript** `sort()` mutates there; `sorted()` doesn't here
- **Python** `sorted(a)` ≈ `a.sorted()`; `a.sort()` same
- **Kotlin** `sorted()` / `sort()` same split

COPY VS IN-PLACE

RETURNS NEW	MUTATES
`sorted()`	`sort()`
`reversed()`	`reverse()`
`shuffled()`	`shuffle()`
`filter { }`	`removeAll(where:)`
`map { }`	(loop with indices)

⚠ COMMON MISTAKES

- Calling `n.sort()` and using the return value: it returns Void
- Forgetting `reversed()` returns a lazy view, not an Array
- `Array(Set(x))` to dedupe when order matters: order is lost

✓ TIPS & BEST PRACTICES

- Dedupe keeping order: `var seen = Set<T>(); a.filter { seen.insert($0).inserted }`
- `partition(by:)` splits an array in place in O(n)
- `elementsEqual(_:)` compares any two sequences

QUICK SUMMARY

- `sorted` vs `sort` — copy vs mutate
- `firstIndex` / `first(where:)` — search
- `removeAll(where:)` — in-place filter

Arrays — Slicing & Ranges

SLICES

Subscripting an array with a range returns an `ArraySlice`: a **view** that shares storage with the original. Slices keep the original's **indices**, so a slice of `a[2...4]` starts at index 2, not 0. Convert with `Array(slice)` before storing it or passing it to APIs that expect `[T]`.

NOTES

- Slicing is O(1); converting to Array is O(n)
- `prefix`, `suffix`, `dropFirst`, `dropLast` also return slices
- `split(separator:)` returns `[ArraySlice]`
- A slice keeps the whole original buffer alive

SLICING

```
let a = [10, 20, 30, 40, 50, 60]

let mid = a[2...4]                          // ArraySlice [30, 40, 50]
mid.startIndex                              // 2  (not 0!)
mid[2]                                      // 30
// mid[0]                                   // crash: index out of range
mid.first                                   // Optional(30)

a.prefix(2)                                 // [10, 20]
a.suffix(2)                                 // [50, 60]
a.dropFirst()                               // [20, 30, 40, 50, 60]
a.dropLast(3)                               // [10, 20, 30]
a.prefix(while: { $0 < 35 })                // [10, 20, 30]

let owned = Array(mid)                      // [30, 40, 50], indices 0…2
[1, 0, 2, 3, 0, 4].split(separator: 0)      // [[1], [2, 3], [4]]

var b = a
b.replaceSubrange(1...2, with: [0])         // [10, 0, 40, 50, 60]
b.removeSubrange(0..<2)                     // [40, 50, 60]
```

CODE EXPLAINED

- → `a[2...4]` — slice shares storage
- → `startIndex` — slice keeps original indices
- → `prefix` / `suffix` / `drop…` — slicing helpers
- → `Array(slice)` — re-index from 0
- → `replaceSubrange` / `removeSubrange` — edit ranges

THE SWIFTY WAY

✗ `slice[0]`
✓ `slice.first` or `slice[slice.startIndex]` — slices don't start at 0

COMING FROM…

- **Python** `a[2:5]` ≈ `a[2..<5]` (but re-indexed there)
- **TypeScript** `slice()` copies; Swift slices share
- **Go** Slices share storage too, re-indexed from 0

SLICING HELPERS

CALL	RESULT FOR [1,2,3,4]
`prefix(2)`	`[1, 2]`
`suffix(1)`	`[4]`
`dropFirst(2)`	`[3, 4]`
`a[1...]`	`[2, 3, 4]`
`a[..<1]`	`[1]`

⚠ COMMON MISTAKES

- Indexing a slice from 0: it uses the parent's indices
- Storing slices of large arrays long-term: memory isn't freed
- Passing `ArraySlice` where `[T]` is required: wrap in `Array(...)`

✓ TIPS & BEST PRACTICES

- Write functions against `some Collection<Int>` to accept arrays and slices
- Use `prefix(n)` instead of `a[0..<n]`: it never traps when n > count
- `chunks(ofCount:)` from Swift Algorithms splits into fixed sizes

QUICK SUMMARY

- `a[r]` — ArraySlice view
- Slices keep original indices
- `Array(slice)` — own copy

Tuples

TUPLES

A tuple groups a fixed number of values, possibly of different types, into one compound value: `(Int, String)`. Elements can be accessed by position (`.0`) or by **name** if labeled. Tuples are perfect for returning several values or short-lived groupings; use a `struct` once the data has meaning beyond one function.

NOTES

- Tuples up to 6 elements are `Equatable` / `Comparable` if elements are
- Tuples can't conform to protocols (not Hashable, not Codable)
- `()` is the empty tuple, which is `Void`
- Destructure with `let (a, b) = t`; ignore with `_`

TUPLES

```
let http404 = (404, "Not Found")
http404.0                                   // 404
http404.1                                   // "Not Found"

let status = (code: 200, message: "OK")     // named elements
status.code                                 // 200

let (code, message) = status                // destructure
let (onlyCode, _) = status                  // ignore part

func stats(_ xs: [Double]) -> (mean: Double, max: Double) {
    (xs.reduce(0, +) / Double(xs.count), xs.max() ?? 0)
}
let s = stats([2, 4, 9])
print(s.mean, s.max)                        // 5.0 9.0

(1, "b") == (1, "b")                        // true
(2, "a") < (2, "b")                         // true

for (name, score) in [("Ada", 9), ("Bo", 7)] { print(name, score) }
```

CODE EXPLAINED

- → `.0` `.1` — positional access
- → `(code: …, message: …)` — named elements
- → `let (a, b) = t` — destructuring
- → tuple return — lightweight multi-value
- → `==` / `<` — element-wise comparison

THE SWIFTY WAY

✗ a 4-element tuple passed between files
✓ a small `struct` — names, docs and conformances

COMING FROM…

- **Python** Tuples work almost the same, but types are fixed
- **TypeScript** `[number, string]` tuple types ≈ `(Int, String)`
- **Kotlin** `Pair` / `Triple` ≈ tuples; Swift's can be named

TUPLE VS STRUCT

NEED	USE
Return 2-3 values	Tuple
Dictionary key	Struct (Hashable)
Codable / JSON	Struct
Used across files	Struct
Loop over pairs	Tuple

⚠ COMMON MISTAKES

- Trying to use a tuple as a Dictionary key or Set element: not Hashable
- Accessing `.2` on a 2-element tuple: compile error
- Long unnamed tuples that force readers to count positions

✓ TIPS & BEST PRACTICES

- Name tuple elements in return types; callers get autocomplete
- `zip` and `enumerated` yield tuples you can destructure in `for`
- Swap with tuples: `(a, b) = (b, a)`

QUICK SUMMARY

- `(A, B)` — fixed-size group
- `.0` or `.name` — access
- Struct when it outgrows one use

Dictionaries — Basics

WORKING WITH DICTIONARIES

`Dictionary<Key, Value>`, written `[Key: Value]`, stores values by **unique, Hashable keys**. Lookups return an **optional** because the key might be missing. Assigning `nil` to a key removes it. Order is **not** preserved; sort the keys or use `OrderedDictionary` when order matters.

NOTES

- Keys must be `Hashable` (String, Int, enums, your structs)
- `dict[key, default: v]` reads with a fallback
- `updateValue` returns the old value
- Iteration yields `(key: K, value: V)` tuples

CREATE, READ, CHANGE

```
var ages = ["Ada": 36, "Grace": 45]
var empty: [String: Int] = [:]

ages["Ada"]                              // Optional(36)
ages["Zoe"]                              // nil
ages["Zoe", default: 0]                  // 0

ages["Linus"] = 28                       // insert
ages["Ada"] = 37                         // update
ages["Grace"] = nil                      // remove
ages.updateValue(29, forKey: "Linus")    // Optional(28) (old)
ages.removeValue(forKey: "Nope")         // nil

ages.count                               // 2
ages.keys.sorted()                       // ["Ada", "Linus"]
Array(ages.values)                       // [37, 29] (any order)

var counts: [Character: Int] = [:]
for ch in "hello" { counts[ch, default: 0] += 1 }
counts["l"]                              // Optional(2)
```

CODE EXPLAINED

- → `[String: Int]` — dictionary type
- → `d[k]` — optional lookup
- → `d[k] = nil` — remove a key
- → `default:` — fallback for reads and `+=`
- → `keys` / `values` — views, unordered

THE SWIFTY WAY

✗ `if d[k] == nil { d[k] = 0 }; d[k]! += 1`
✓ `d[k, default: 0] += 1`

COMING FROM…

- **Python** `d.get(k, 0)` ≈ `d[k, default: 0]`
- **TypeScript** `Map` ≈ `[K: V]`, but a value type here
- **Kotlin** `getOrDefault` ≈ `default:`; `mutableMapOf` ≈ `var`

CORE MEMBERS

MEMBER	RETURNS
`d[k]`	V?
`d[k, default: v]`	V
`updateValue(_:forKey:)`	Old V?
`removeValue(forKey:)`	Removed V?
`keys / values`	Collections

⚠ COMMON MISTAKES

- Force-unwrapping `d[k]!` for keys that may be missing
- Expecting iteration in insertion order
- Using a class with mutable fields as a key: its hash can change

✓ TIPS & BEST PRACTICES

- Use enums as keys for fixed sets of options
- `Dictionary(uniqueKeysWithValues:)` builds from pairs (traps on duplicates)
- Sort before display: `d.sorted { $0.key < $1.key }`

QUICK SUMMARY

- `[K: V]` — hashed key/value store
- `d[k]` returns optional
- `default:` — fallback & counting

Dictionaries — Methods

DICTIONARY METHODS

Dictionaries have their own transforms: `mapValues` and `compactMapValues` keep the keys, `filter` returns a Dictionary, and `merge` combines two with a rule for duplicate keys. Initializers like `Dictionary(grouping:by:)` turn arrays into dictionaries in one line.

NOTES

- `map` on a Dictionary returns an Array of whatever you return
- `filter` returns a Dictionary (since Swift 4)
- `uniquingKeysWith:` decides which value wins
- `merging` returns new; `merge` mutates

TRANSFORM & COMBINE

```
let prices = ["apple": 1.2, "pear": 0.9, "kiwi": 2.5]

prices.mapValues { $0 * 2 }                // same keys, doubled
prices.filter { $0.value > 1 }             // ["apple": 1.2, "kiwi": 2.5]
prices.map { "\($0.key): \($0.value)" }    // [String]
["a": "1", "b": "x"].compactMapValues(Int.init)   // ["a": 1]

let words = ["apple", "avocado", "banana", "blueberry", "cherry"]
Dictionary(grouping: words, by: { $0.first! })
// ["a": ["apple", "avocado"], "b": [...], "c": ["cherry"]]

let pairs = [("a", 1), ("b", 2), ("a", 3)]
Dictionary(pairs, uniquingKeysWith: +)     // ["a": 4, "b": 2]
Dictionary(pairs, uniquingKeysWith: { _, new in new })  // ["a": 3, "b": 2]

var stock = ["apple": 5]
stock.merge(["apple": 3, "pear": 2]) { old, new in old + new }
// ["apple": 8, "pear": 2]
prices.min { $0.value < $1.value }?.key    // "pear"
```

CODE EXPLAINED

- → `mapValues` — transform values, keep keys
- → `compactMapValues` — drop nil results
- → `Dictionary(grouping:by:)` — bucket an array
- → `uniquingKeysWith:` — resolve duplicate keys
- → `merge(_:uniquingKeysWith:)` — combine in place

THE SWIFTY WAY

✗ loop + `if d[k] == nil { d[k] = [] }; d[k]!.append(x)`
✓ `Dictionary(grouping: xs, by: key)`

COMING FROM…

- **Kotlin** `groupBy` ≈ `Dictionary(grouping:by:)`
- **Python** `defaultdict(list)` ≈ grouping; `{**a, **b}` ≈ merging
- **TypeScript** `Object.groupBy` ≈ grouping

BUILDERS

INITIALIZER	USE
`Dictionary(uniqueKeysWithValues:)`	Pairs, no dupes
`Dictionary(_:uniquingKeysWith:)`	Pairs with dupes
`Dictionary(grouping:by:)`	Group into arrays
`zip(keys, values)`	Pair two arrays

⚠ COMMON MISTAKES

- Using `map` and expecting a Dictionary back: use `mapValues`
- `uniqueKeysWithValues` with duplicate keys: runtime crash
- Forgetting `merge` needs a closure even when there are no conflicts

✓ TIPS & BEST PRACTICES

- `Dictionary(uniqueKeysWithValues: zip(ids, users))` pairs arrays
- `users.reduce(into: [:]) { $0[$1.id] = $1 }` indexes by ID
- `OrderedDictionary` (swift-collections) keeps insertion order

QUICK SUMMARY

- `mapValues` / `filter` — keep a Dictionary
- `grouping:by:` — bucket
- `merge` — combine with a rule

Sets

SETS

`Set<Element>` is an unordered collection of **unique**, Hashable values with O(1) average `contains`, `insert` and `remove`. Use it for membership tests and deduplication, and for set algebra: union, intersection, difference and symmetric difference.

NOTES

- A set literal needs a type: `let s: Set = [1, 2]`
- `insert` returns `(inserted: Bool, memberAfterInsert:)`
- Iteration order is unspecified
- Each operation has a mutating `form…` twin

SET OPERATIONS

```
var tags: Set = ["swift", "ios", "ui"]
tags.insert("ios").inserted                // false (already there)
tags.insert("mac")
tags.contains("ui")                        // true
tags.remove("ui")                          // Optional("ui")

let a: Set = [1, 2, 3, 4]
let b: Set = [3, 4, 5]
a.union(b)                                 // [1, 2, 3, 4, 5]
a.intersection(b)                          // [3, 4]
a.subtracting(b)                           // [1, 2]
a.symmetricDifference(b)                   // [1, 2, 5]

a.isSubset(of: [1, 2, 3, 4, 5])            // true
a.isDisjoint(with: [9])                    // true
Set([3, 1, 3, 2]).sorted()                 // [1, 2, 3]

var seen = Set<Int>()
[4, 1, 4, 2, 1].filter { seen.insert($0).inserted }   // [4, 1, 2]
```

CODE EXPLAINED

- → `Set` — unique, unordered
- → `insert(_:).inserted` — was it new?
- → `union` / `intersection` / `subtracting`
- → `isSubset` / `isDisjoint` — relations
- → `seen.insert` trick — order-preserving dedupe

THE SWIFTY WAY

✗ `array.contains(x)` in a hot loop
✓ build a `Set` once, then `set.contains(x)` — O(1) vs O(n)

COMING FROM…

- **Python** `set` with `|`, `&`, `-` ≈ named methods here
- **TypeScript** `Set` exists, but no union/intersection built in
- **Kotlin** `union`, `intersect`, `subtract` — same idea

SET ALGEBRA

METHOD	MUTATING TWIN
union	formUnion
intersection	formIntersection
subtracting	subtract
symmetricDifference	formSymmetricDifference

⚠ COMMON MISTAKES

- Expecting a stable order when printing or iterating
- Writing `let s = [1, 2]` and getting an Array: annotate `Set`
- Putting non-Hashable types in a Set: compile error

✓ TIPS & BEST PRACTICES

- Make your struct `Hashable` (often just `: Hashable`) to store it in a Set
- Use `OptionSet` for bit flags, `Set` for runtime collections
- `OrderedSet` from swift-collections keeps insertion order

QUICK SUMMARY

- `Set` — unique, fast membership
- union / intersection / subtracting
- `form…` — in-place versions

Sequence & Collection Protocols

THE COLLECTION PROTOCOLS

Every collection type builds on a small protocol stack. A **Sequence** can be iterated once with an iterator; a **Collection** adds indices and multi-pass access; **BidirectionalCollection** and **RandomAccessCollection** add backward and O(1) indexing. Conform your own types to inherit `map`, `filter`, `contains` and the rest for free.

NOTES

- Sequence needs only `makeIterator()`
- An `IteratorProtocol` needs only `next() -> Element?`
- `lazy` delays work until elements are needed
- Generic code should ask for the weakest protocol it needs

CONFORMING TO SEQUENCE

```
struct Countdown: Sequence {
    let start: Int
    func makeIterator() -> some IteratorProtocol<Int> {
        var current = start
        return AnyIterator {
            guard current > 0 else { return nil }
            defer { current -= 1 }
            return current
        }
    }
}
for n in Countdown(start: 3) { print(n) }  // 3 2 1
Countdown(start: 5).map { $0 * 10 }        // [50, 40, 30, 20, 10]
Countdown(start: 5).contains(2)            // true

let firstBig = (1...1_000_000).lazy
    .map { $0 * $0 }
    .first { $0 > 500 }                    // 529, only 23 squares computed

func total(_ xs: some Sequence<Int>) -> Int { xs.reduce(0, +) }
total([1, 2]); total(1...3); total(Set([4]))
```

CODE EXPLAINED

- → `makeIterator()` — the one Sequence requirement
- → `AnyIterator { }` — iterator from a closure
- → free methods — map, contains, etc.
- → `.lazy` — compute on demand
- → `some Sequence<Int>` — accept any sequence

THE SWIFTY WAY

✗ `func f(_ a: [Int])` that only iterates
✓ `func f(_ a: some Sequence<Int>)` — accepts ranges, sets, slices

COMING FROM…

- **Kotlin** `Sequence` ≈ lazy Swift sequence; `Iterable` ≈ Sequence
- **Python** `__iter__` / `__next__` ≈ makeIterator / next
- **TypeScript** `[Symbol.iterator]` ≈ Sequence

PROTOCOL STACK

PROTOCOL	ADDS
Sequence	for-in, map, filter
Collection	indices, count, subscript
BidirectionalCollection	last, reversed()
RandomAccessCollection	O(1) index math
MutableCollection	Assign through subscript

⚠ COMMON MISTAKES

- Iterating a single-pass Sequence twice: the second pass may be empty
- Using `.lazy` and then calling `count`: it runs the whole chain
- Conforming to Collection with an O(n) subscript: breaks performance promises

✓ TIPS & BEST PRACTICES

- `sequence(first:next:)` builds a sequence from a seed
- `stride` and `zip` are lazy sequences too
- Use `AnySequence` only when you must erase types

QUICK SUMMARY

- Sequence → Collection → RandomAccess
- `makeIterator` / `next` — custom sequences
- `.lazy` — deferred work

Sorting, Searching, Stacks & Queues

SORTING, SEARCHING, STACKS & QUEUES

Swift's sort is **stable** and O(n log n). Sort by a property with a closure, or by several keys with `KeyPathComparator`. For stacks, a plain Array with `append` and `popLast` is ideal. Arrays make poor queues (`removeFirst` is O(n)); use `Deque` from swift-collections, or an index-based queue.

NOTES

- `sort` has been stable since Swift 5
- No built-in binary search; write one or use a sorted index
- `min(by:)` / `max(by:)` find extremes without sorting
- `Heap` (swift-collections) is a priority queue

SORT, SEARCH, STACK, QUEUE

```
struct P { let name: String; let age: Int }
let people = [P(name: "Bo", age: 30), P(name: "Al", age: 25),
              P(name: "Cy", age: 30)]

people.sorted { $0.age < $1.age }              // Al, Bo, Cy
people.sorted(using: [KeyPathComparator(\.age, order: .reverse),
                      KeyPathComparator(\.name)])   // Bo, Cy, Al
people.max { $0.age < $1.age }?.name           // "Bo"

func bsearch(_ a: [Int], _ t: Int) -> Int? {
    var lo = 0, hi = a.count - 1
    while lo <= hi {
        let mid = (lo + hi) / 2
        if a[mid] == t { return mid }
        if a[mid] < t { lo = mid + 1 } else { hi = mid - 1 }
    }
    return nil
}

var stack = [1, 2]; stack.append(3); stack.popLast()   // 3
import DequeModule
var queue: Deque = [1, 2]; queue.append(3); queue.popFirst()  // 1
```

CODE EXPLAINED

- → `sorted { }` — custom order, stable
- → `KeyPathComparator` — multi-key sort (Foundation)
- → `max(by:)` — extreme without sorting
- → binary search — O(log n) on sorted arrays
- → `append` + `popLast` = stack; `Deque` = queue

THE SWIFTY WAY

✗ `queue.removeFirst()` on an Array
✓ `Deque.popFirst()` — O(1) instead of O(n)

COMING FROM…

- **Java** `ArrayDeque` ≈ `Deque`; `PriorityQueue` ≈ `Heap`
- **Python** `collections.deque` ≈ `Deque`; `heapq` ≈ `Heap`
- **Kotlin** `sortedWith(compareBy(...))` ≈ KeyPathComparator

STRUCTURE CHOICE

NEED	USE
LIFO stack	Array append / popLast
FIFO queue	Deque
Priority queue	Heap
Sorted unique	Set + sorted()
Fast lookup	Dictionary / Set

⚠ COMMON MISTAKES

- Using `removeFirst()` on arrays in BFS loops: quadratic time
- Binary searching an unsorted array
- Sorting just to find the min or max

✓ TIPS & BEST PRACTICES

- Add `swift-collections` for Deque, Heap, OrderedSet (Sheet 091)
- For BFS without dependencies, keep a `head` index into an array
- `partitioningIndex(where:)` in Swift Algorithms is a ready-made binary search

QUICK SUMMARY

- `sorted(by:)` / `KeyPathComparator` — sorting
- Array = stack; Deque = queue
- binary search on sorted data

Structs

VALUE TYPES WITH STRUCT

A `struct` bundles stored properties and methods into a **value type**: every assignment or pass makes an independent copy. Swift writes a **memberwise initializer** for you. Methods that change properties must be marked `mutating`, and they can only be called on a `var`. Most Swift types, including `String`, `Array` and `Int`, are structs.

NOTES

- Structs can't inherit, but they can adopt protocols
- The memberwise init uses defaults for properties that have them
- Writing your own `init` in the struct body removes the memberwise one
- Copies are cheap: large stdlib types use copy-on-write

DEFINING A STRUCT

```
struct Counter {
    var name: String
    var count = 0                           // default value

    mutating func increment(by n: Int = 1) { count += n }
    func describe() -> String { "\(name): \(count)" }
}

var a = Counter(name: "clicks")             // memberwise init
a.increment()
a.increment(by: 4)
a.describe()                                // "clicks: 5"

var b = a                                   // copy
b.increment()
a.count                                     // 5 (unchanged)
b.count                                     // 6

let c = Counter(name: "fixed", count: 9)
// c.increment()                            // error: c is a let

extension Counter {                         // keeps memberwise init
    init(_ name: String) { self.init(name: name) }
}
```

CODE EXPLAINED

- → `struct` — value type
- → memberwise init — generated from properties
- → `mutating` — method may change `self`
- → `b = a` — independent copy
- → `init` in an extension — keeps the generated one

THE SWIFTY WAY

✗ `class` for plain data models
✓ `struct` — value semantics avoid shared-state bugs

COMING FROM…

- **Kotlin** `data class` ≈ struct, but Kotlin's is a reference
- **TypeScript** Objects are shared refs; structs copy
- **Java** `record` ≈ immutable struct

STRUCT FEATURES

FEATURE	SUPPORTED
Properties & methods	✔
Initializers	✔ (memberwise free)
Protocols & extensions	✔
Inheritance	✖
deinit	✖ (except ~Copyable)

IN SWIFTUI

- Every SwiftUI `View` is a struct: cheap to create, recreated on each update
- Model data as structs passed into views
- See SwiftUI O3 · Views & body

⚠ COMMON MISTAKES

- Calling a `mutating` method on a `let` instance
- Adding an `init` in the struct body and losing the memberwise init
- Expecting a change through one variable to show up in a copy

✓ TIPS & BEST PRACTICES

- Start every type as a struct; switch to class only when you need identity
- Conform to `Equatable`, `Hashable`, `Codable` with one line each
- Use `private(set) var` to expose read-only state

QUICK SUMMARY

- `struct` — value type, copied
- memberwise init for free
- `mutating` — changes self

Classes

REFERENCE TYPES WITH CLASS

A `class` is a **reference type**: variables hold references, so two variables can point at the same object and see each other's changes. Classes support **inheritance**, identity comparison with ===, and a `deinit` that runs when the last reference goes away (ARC). Classes get no memberwise init: all stored properties need defaults or an `init`.

NOTES

- Mark classes `final` unless you design for subclassing
- `let obj = MyClass()` still allows changing its `var` properties
- No `mutating` keyword: methods can always change properties
- Classes are needed for Objective-C interop and shared mutable state

DEFINING A CLASS

```
final class BankAccount {
    let owner: String
    private(set) var balance: Double

    init(owner: String, balance: Double = 0) {
        self.owner = owner
        self.balance = balance
    }
    deinit { print("closing \(owner)'s account") }

    func deposit(_ amount: Double) { balance += amount }
}

let a = BankAccount(owner: "Ada")
let b = a                                   // same object
b.deposit(100)
a.balance                                   // 100.0 (shared)
a === b                                     // true

var c: BankAccount? = BankAccount(owner: "Temp")
c = nil                                     // closing Temp's account
```

CODE EXPLAINED

- → `final class` — no subclasses, faster dispatch
- → `init` — required to set every stored property
- → `deinit` — runs when the last reference is released
- → `b = a` — copies the reference, not the object
- → `===` — same instance

THE SWIFTY WAY

✗ `class` with no `final`
✓ `final class` until a subclass is needed — clearer intent, static dispatch

COMING FROM…

- **Java** Classes work like Java objects, minus GC
- **Kotlin** Swift classes are open by default; mark `final`
- **Python** Same reference semantics; `deinit` ≈ `__del__`

CLASS-ONLY FEATURES

FEATURE	WHY
`Inheritance`	Subclass and override
`deinit`	Cleanup on release
`=== identity`	Same object?
`weak / unowned refs`	Break cycles
`@objc interop`	Objective-C runtime

⚠ COMMON MISTAKES

- Expecting `let` to make a class instance immutable
- Creating retain cycles between objects (parent ↔ child): use `weak`
- Sharing a class instance across threads without isolation (Swift 6 flags this)

✓ TIPS & BEST PRACTICES

- Use `@Observable final class` for SwiftUI view models
- Use `actor` instead of `class` for shared mutable state across tasks
- Add a `deinit` print while debugging leaks

QUICK SUMMARY

- `class` — reference type
- `init` required; `deinit` available
- `final` by default

Struct vs Class

CHOOSING STRUCT OR CLASS

Apple's guidance: **use structs by default**. Choose a class when you need identity (one shared, changing thing), inheritance, Objective-C interop, or `deinit`. When many tasks share mutable state, choose an **actor**. Value types make code easier to reason about because no one else can change your copy.

NOTES

- Stdlib collections are structs with copy-on-write storage
- A struct holding a class reference shares that object
- Enums are value types too
- Actors are reference types with built-in isolation

SAME DATA, TWO BEHAVIORS

```
struct PointS { var x = 0 }
final class PointC { var x = 0 }

var s1 = PointS(); var s2 = s1
s2.x = 9
s1.x                                    // 0  (copy)

let c1 = PointC(); let c2 = c1
c2.x = 9
c1.x                                    // 9  (shared)

func bump(_ p: PointS) -> PointS { var p = p; p.x += 1; return p }
func bump(_ p: PointC) { p.x += 1 }          // mutates caller's object

// struct with a class inside: copies share the object
struct Wrapper { let ref = PointC() }
let w1 = Wrapper(); let w2 = w1
w2.ref.x = 5
w1.ref.x                                // 5

actor Inventory { var items: [String] = [] }   // shared + safe
```

CODE EXPLAINED

- → struct copy — changes stay local
- → class reference — changes are shared
- → functions — structs return new values, classes mutate
- → nested class in struct — still shared
- → `actor` — shared state with isolation

THE SWIFTY WAY

✗ `class Settings` passed everywhere and mutated
✓ `struct Settings` + one owner that updates it

COMING FROM…

- **Java** Everything is a class there; here most things are structs
- **C#** Same struct/class split as C#
- **Kotlin** `data class` is a reference; Swift struct copies

DECISION GUIDE

NEED	CHOOSE
Plain data / model	struct
Fixed set of options	enum
Identity, shared, inheritance	class
Shared across tasks	actor
SwiftUI view model	@Observable class

⚠ COMMON MISTAKES

- Assuming a struct is deeply immutable when it holds class references
- Choosing class "for performance" without measuring
- Making a large class hierarchy where protocols + structs would do

✓ TIPS & BEST PRACTICES

- If you'd want `==` to compare contents, it's probably a struct
- If you'd want `===` (same thing), it's probably a class
- Implement copy-on-write manually only for big custom value types

QUICK SUMMARY

- struct by default
- class for identity / inheritance
- actor for shared mutable state

Enums — Basics

ENUMERATIONS

An `enum` defines a type with a fixed set of **cases**. Swift enums are full types: they can have raw values (`String`, `Int`), methods, computed properties and protocol conformances. `switch` over an enum must cover every case, so adding a case shows you every place that needs updating.

ENUMS WITH RAW VALUES

```
enum Suit: String, CaseIterable {
    case hearts = "♥", diamonds = "♦", clubs = "♣", spades = "♠"

    var isRed: Bool { self == .hearts || self == .diamonds }
}

let s = Suit.hearts
s.rawValue                              // "♥"
Suit(rawValue: "♣")                     // Optional(.clubs)
Suit(rawValue: "x")                     // nil
Suit.allCases.count                     // 4
s.isRed                                 // true

enum Level: Int, Comparable {
    case low = 1, medium, high          // 1, 2, 3 (auto)
    static func < (a: Level, b: Level) -> Bool { a.rawValue < b.rawValue }
}
Level.high.rawValue                     // 3
Level.low < .high                       // true

var dir: Suit = .spades                 // shorthand
```

NOTES

- Cases are lowerCamelCase: `case north`
- Raw-value enums get `init?(rawValue:)` for free
- `CaseIterable` adds `allCases`
- Use `.case` shorthand when the type is known

CODE EXPLAINED

- → `enum X: String` — raw values
- → `init?(rawValue:)` — failable from raw
- → `CaseIterable` / `allCases` — every case
- → computed property on an enum
- → auto-incrementing Int raw values

THE SWIFTY WAY

✗ `let status = "active"`
✓ `enum Status { case active, paused }` — typos become compile errors

COMING FROM…

- **Kotlin** `enum class` ≈ raw-value enum
- **TypeScript** String-literal unions ≈ String enums
- **Java** Java enums ≈ these, plus `allCases`

IN SWIFTUI

- `ForEach(Suit.allCases, id: \.self)` lists every case
- Enums drive `Picker` selections and tab tags
- See SwiftUI 22 · Pickers

ENUM CAPABILITIES

FEATURE	EXAMPLE
Raw value	`case a = "A"`
Associated value	`case ok(Int)` (Sheet 064)
Methods / props	`var isRed: Bool`
Static members	`static let default = .low`
Protocols	`: Codable, CaseIterable`

⚠ COMMON MISTAKES

- Using `rawValue` strings around the app instead of the enum
- Adding `default:` to switches over your own enum: hides new cases
- Force-unwrapping `init(rawValue:)` on external data

✓ TIPS & BEST PRACTICES

- String raw values are auto-filled with the case name if omitted
- Enums with String raw values are `Codable` for free
- Use a caseless `enum` as a namespace for constants

QUICK SUMMARY

- `enum` — fixed set of cases
- raw values + `init?(rawValue:)`
- `CaseIterable` — `allCases`

Enums — Associated Values

ENUMS THAT CARRY DATA

Each enum case can carry its own **associated values** of any types, making enums Swift's tagged unions. This models states precisely: a `.loaded` case carries data, a `.failed` case carries an error, and impossible combinations simply can't be represented. Extract the values with `switch` or `if case`.

NOTES

- Associated values can be labeled: `case rect(w: Double, h: Double)`
- An enum can't have both raw values and associated values
- `Optional` and `Result` are enums with associated values
- Synthesized `Equatable` / `Hashable` work if all payloads conform

MODELING STATE

```
enum LoadState {
    case idle
    case loading(progress: Double)
    case loaded([String])
    case failed(Error)

    var isBusy: Bool {
        if case .loading = self { true } else { false }
    }
}

var state = LoadState.loading(progress: 0.4)

switch state {
case .idle:                      print("tap to load")
case .loading(let p):            print("\(Int(p * 100))%")   // 40%
case .loaded(let items) where items.isEmpty: print("nothing")
case .loaded(let items):         print(items.count)
case .failed(let error):         print(error.localizedDescription)
}

state = .loaded(["a", "b"])
if case .loaded(let items) = state { print(items) }   // ["a", "b"]
state.isBusy                                          // false
```

CODE EXPLAINED

- → `case loading(progress: Double)` — labeled payload
- → `case .loading(let p)` — bind the payload
- → `where` — refine a case
- → `if case` — test one case
- → computed property — query the state

THE SWIFTY WAY

✗ `var isLoading: Bool; var data: [String]?; var error: Error?`

✓ one `enum LoadState` — no invalid combinations

COMING FROM…

- **Kotlin** `sealed class` ≈ enum with associated values
- **TypeScript** Discriminated union `{kind: 'loaded', data}`
- **Python** No direct equivalent; dataclasses + match

BUILT-IN EXAMPLES

ENUM	CASES
`Optional<T>`	.none / .some(T)
`Result<S, F>`	.success(S) / .failure(F)
`Never`	(no cases)

⚠ COMMON MISTAKES

- Using several optionals + Bools instead of one state enum
- Writing `case .loaded(_)` everywhere and never using the data
- Expecting `==` on enums with an `Error` payload: Error isn't Equatable

✓ TIPS & BEST PRACTICES

- Store the enum in your view model and `switch` in the view
- Use `indirect` for recursive cases (Sheet 049)
- Give cases static helper factories for common payloads

QUICK SUMMARY

- associated values — per-case data
- `switch` / `if case` — extract
- model state, not flags

Properties

STORED & COMPUTED PROPERTIES

Stored properties hold values; **computed** properties calculate one each time with a getter (and optional setter). **Property observers** `willSet` and `didSet` run code around changes to stored properties. `lazy var` delays creating a value until it's first used, and `static` properties belong to the type itself.

NOTES

- Computed properties must be `var`
- A get-only computed property can drop the `get { }` wrapper
- `didSet` gives you `oldValue`; `willSet` gives `newValue`
- Observers don't fire during `init`

PROPERTY KINDS

```
struct Temperature {
    var celsius: Double {                       // stored + observer
        didSet { print("changed from \(oldValue)") }
    }
    var fahrenheit: Double {                    // computed get/set
        get { celsius * 9 / 5 + 32 }
        set { celsius = (newValue - 32) * 5 / 9 }
    }
    var isFreezing: Bool { celsius <= 0 }       // get-only shorthand

    static let boiling = 100.0                  // type property
}

var t = Temperature(celsius: 20)
t.fahrenheit                                    // 68.0
t.fahrenheit = 212                              // changed from 20.0
t.celsius                                       // 100.0
Temperature.boiling                             // 100.0

final class Report {
    lazy var formatter = DateFormatter()        // created on first use
    private(set) var pages = 0                  // read-only outside
}
```

CODE EXPLAINED

→ `didSet` / `oldValue` — react after a change

→ `get` / `set` / `newValue` — computed property

→ get-only — no `get { }` needed

→ `static let` — one value for the type

→ `lazy var` — built on first access

THE SWIFTY WAY

✗ `func getFullName() -> String`

✓ `var fullName: String { … }` — no-argument, cheap, no side effects → property

COMING FROM…

- **Kotlin** `val x get() = …` ≈ computed property
- **TypeScript** `get x()` / `set x(v)` accessors
- **Python** `@property` ≈ computed property

PROPERTY KINDS

KIND	SYNTAX
Stored	`var x = 0`
Computed	`var y: Int { x * 2 }`
Observed	`var x = 0 { didSet { } }`
Lazy	`lazy var z = Expensive()`
Type	`static let k = 1`

IN SWIFTUI

- `@State`, `@Binding`, `@Environment` are property wrappers on stored properties
- `var body: some View` is a computed property
- See SwiftUI 14 · @State

⚠ COMMON MISTAKES

- Doing slow work in a computed property: callers assume it's cheap
- Expecting `didSet` to run when setting the value in `init`
- Using `lazy` on a `let`: not allowed

✓ TIPS & BEST PRACTICES

- Use `private(set)` to allow reading but not writing from outside
- Use `didSet` to validate or clamp: `didSet { x = min(x, 100) }`
- `lazy` isn't thread-safe; avoid it in shared classes

QUICK SUMMARY

- stored vs computed properties
- `willSet` / `didSet` — observers
- `lazy` / `static` — deferred / type-level

Initializers

INITIALIZATION

An initializer must give **every stored property** a value before the instance is used. Structs get a memberwise init; classes need their own. Classes have **designated** initializers (full setup, calling `super.init`) and **convenience** initializers (delegating to another init on the same class). A failable `init?` returns `nil` for invalid input.

NOTES

- Two-phase init: set own properties, then call `super.init`, then customize
- `required init` forces every subclass to implement it
- `self.init(...)` delegates within the same type
- Structs can delegate with `self.init` too (no `convenience` keyword)

DESIGNATED, CONVENIENCE, FAILABLE

```
class Vehicle {
    let wheels: Int
    init(wheels: Int) { self.wheels = wheels }            // designated
    convenience init() { self.init(wheels: 4) }           // delegates
}

final class Bike: Vehicle {
    let hasBell: Bool
    init(hasBell: Bool) {
        self.hasBell = hasBell          // 1. own properties first
        super.init(wheels: 2)           // 2. then super
        print("bike ready")             // 3. now self is usable
    }
}

struct Email {
    let value: String
    init?(_ raw: String) {                                // failable
        guard raw.contains("@") else { return nil }
        value = raw.lowercased()
    }
}
Email("ada@example.com")?.value         // "ada@example.com"
Email("nope")                           // nil
```

CODE EXPLAINED

- → designated init — sets all properties
- → `convenience init` — must call `self.init`
- → two-phase — own props, `super.init`, then use self
- → `init?` — returns nil on bad input

THE SWIFTY WAY

✗ a `validate()` call after init
✓ `init?` or `init(…) throws` — invalid instances never exist

INITIALIZER TYPES

KIND	RULE
`designated`	Init all props, call super
`convenience`	Call self.init
`init?`	May return nil
`init() throws`	May throw
`required`	Subclasses must implement

COMING FROM…

- **Kotlin** Primary/secondary constructors ≈ designated/convenience
- **Java** Constructor chaining `this(...)` ≈ `self.init`
- **Python** `__init__` with `super().__init__()` (order reversed)

⚠ COMMON MISTAKES

- Calling a method on `self` before every property is set: compile error
- Calling `super.init` before setting the subclass's own properties
- Using `init?` and losing the reason for failure: use `throws` when callers need it

✓ TIPS & BEST PRACTICES

- Prefer default property values over long init parameter lists
- Put extra struct inits in an extension to keep the memberwise one
- Use `throws` inits for parsing where errors explain what failed

QUICK SUMMARY

- every stored property set before use
- designated vs convenience
- `init?` / `init() throws` — can fail

Access Control

ACCESS LEVELS

Access control limits which code can see a declaration. From most to least open: `open`, `public`, `package`, `internal` (the default), `fileprivate` and `private`. The boundary that matters most is the **module**: `internal` code is visible across your whole app target but not to other modules.

NOTES

- `internal` is the default; you rarely need to write it
- `private` is visible in the enclosing declaration and its extensions in the same file
- `open` allows subclassing/overriding from other modules; `public` does not
- A type's members can't be more visible than the type

ACCESS MODIFIERS

```
// In a framework module "Payments"
public struct Price {
    public let cents: Int
    public init(cents: Int) { self.cents = cents }   // must be public
    var display: String { "$\(cents / 100)" }        // internal
}

open class PaymentView {                  // subclassable elsewhere
    open func render() {}                 // overridable elsewhere
    public func reset() {}                // callable, not overridable
}

package func sharedHelper() {}            // visible in same package

final class Cart {
    private(set) var items: [String] = [] // read outside, write inside
    private var token = ""                // this type + same-file extensions
    fileprivate func audit() {}           // whole file
}
extension Cart {
    func add(_ s: String) { items.append(s); token = s }  // ok: same file
}
```

CODE EXPLAINED

- → `public` — usable from other modules
- → `open` — plus subclass/override
- → `package` — same Swift package only
- → `private(set)` — public read, private write
- → `private` — type + same-file extensions

THE SWIFTY WAY

✗ everything `public` in an app target
✓ default `internal`; `private` for details — access control matters at module borders

LEVELS (OPEN → CLOSED)

LEVEL	VISIBLE TO
`open`	All modules, subclassable
`public`	All modules
`package`	Modules in the same package
`internal`	Same module (default)
`fileprivate`	Same file
`private`	Enclosing scope + same-file extensions

COMING FROM…

- **Java** `package` ≈ package-private; `private` ≈ private
- **Kotlin** `internal` means module there too
- **TypeScript** `export` ≈ `public`; `#private` ≈ `private`

SWIFT VERSIONS

- 5.9 — `package` access level
- 4.0 — `private` includes same-file extensions

⚠ COMMON MISTAKES

- Forgetting a `public init` on a public struct: other modules can't create it
- Using `open` when `public` is enough: you promise subclass support forever
- Making properties `public var` when callers should only read: use `public private(set)`

✓ TIPS & BEST PRACTICES

- Split apps into local packages so `internal` actually hides things
- Default to `private`; widen only when the compiler asks
- `@testable import` lets tests reach `internal` symbols

QUICK SUMMARY

- open › public › package › internal › fileprivate › private
- `internal` — default
- `private(set)` — read-only outside

Inheritance

SUBCLASSING

A class can inherit from **one** superclass, gaining its properties and methods. Override behavior with `override`, call the parent's version with `super`, and stop further overriding with `final`. Swift checks overrides at compile time, so a typo in a method name can't silently create a new method.

NOTES

- Only classes inherit; structs and enums use protocols
- `override` is required, not optional
- `static` members can't be overridden; `class` members can
- Prefer composition and protocols over deep hierarchies

INHERITANCE

```
class Animal {
    let name: String
    init(name: String) { self.name = name }

    func speak() -> String { "..." }
    class var kingdom: String { "Animalia" }    // overridable type prop
    final func id() -> String { "#\(name)" }    // can't override
}

class Dog: Animal {
    override func speak() -> String { "Woof" }
}

final class Puppy: Dog {
    override func speak() -> String { super.speak() + "!" }
}

let pets: [Animal] = [Dog(name: "Rex"), Puppy(name: "Bit")]
for p in pets { print(p.name, p.speak()) }      // Rex Woof, Bit Woof!

if let d = pets[0] as? Dog { print("dog:", d.name) }
pets[1] is Dog                                  // true
```

CODE EXPLAINED

- → `class Dog: Animal` — single inheritance
- → `override` — replace a method
- → `super.speak()` — call the parent version
- → `final` — block further overrides
- → `as?` / `is` — downcast and check

THE SWIFTY WAY

✗ a 4-level class hierarchy
✓ a protocol + small structs, or composition

KEYWORDS

KEYWORD	MEANING
`override`	Replace parent member
`super`	Parent implementation
`final`	No overriding / subclassing
`class func / var`	Overridable type member
`static func / var`	Non-overridable type member

COMING FROM…

- **Java** `extends` ≈ `:` ; `@Override` is required here
- **Kotlin** Classes are final by default there; open here unless `final`
- **Python** Single inheritance only; no MRO

⚠ COMMON MISTAKES

- Forgetting `override` : compile error (good!)
- Overriding without calling `super` where the parent does required work (`viewDidLoad`)
- Downcasting with `as!` when the type isn't guaranteed

✓ TIPS & BEST PRACTICES

- Mark classes `final` unless they're designed to be subclassed
- Use protocol default implementations to share behavior across structs
- Call `super` first in setup methods and last in teardown

QUICK SUMMARY

- `class B: A` — inherit
- `override` + `super`
- `final` — stop inheritance

Protocols — Basics

PROTOCOLS

A `protocol` lists the methods and properties a type must provide. Structs, enums, classes and actors can all **conform**, and one type can adopt many protocols. Protocols are the main tool for abstraction in Swift: code can depend on "anything that can `fetch()`" instead of a concrete class.

NOTES

- Property requirements say `{ get }` or `{ get set }`
- Mutating requirements need `mutating` for structs to conform
- Conformance can be added in an extension
- Use `some P` / `any P` when a protocol is a parameter type (Sheet 074)

DEFINING & ADOPTING

```
protocol Shape {
    var name: String { get }
    func area() -> Double
}

struct Circle: Shape {
    let r: Double
    var name: String { "circle" }
    func area() -> Double { .pi * r * r }
}

struct Square: Shape {
    let side: Double
    let name = "square"                          // stored also satisfies { get }
    func area() -> Double { side * side }
}

let shapes: [any Shape] = [Circle(r: 1), Square(side: 2)]
for s in shapes { print(s.name, s.area()) }     // circle 3.14…, square 4.0

func largest(_ list: [any Shape]) -> (any Shape)? {
    list.max { $0.area() < $1.area() }
}

protocol Resettable { mutating func reset() }
```

CODE EXPLAINED

- → `protocol` — list of requirements
- → `{ get }` — readable property (stored or computed)
- → `: Shape` — conformance
- → `[any Shape]` — mixed conforming types
- → `mutating func` — lets structs conform

THE SWIFTY WAY

✗ a base class just to share an interface
✓ a protocol — works for structs and enums too

COMING FROM…

- **Java** `interface` ≈ protocol
- **Kotlin** `interface` ≈ protocol
- **TypeScript** `interface`, but conformance is declared, not structural

REQUIREMENT FORMS

REQUIREMENT	SATISFIED BY
`var x: T { get }`	let, var or computed
`var x: T { get set }`	var or get/set
`func f()`	Method
`mutating func f()`	Struct mutating method
`init()`	required init (classes)
`static func make()`	Static or class method

IN SWIFTUI

- `struct MyView: View` — conform by providing `var body: some View`
- `Identifiable` lets `List` and `ForEach` track rows
- See SwiftUI O3 · Views & body

⚠ COMMON MISTAKES

- Declaring a `{ get set }` requirement and conforming with a `let`
- Forgetting `mutating` in the protocol, then structs can't change state
- Using a protocol as a type without `any` (warning in Swift 6 mode)

✓ TIPS & BEST PRACTICES

- Name capability protocols with -able/-ing (`Sendable`) and roles with nouns (`Collection`)
- Keep protocols small; compose with `P & Q`
- Use protocols to inject fakes in tests

QUICK SUMMARY

- `protocol` — requirements
- any type can conform
- `any P` — protocol as a type

Protocol Extensions & POP

PROTOCOL EXTENSIONS

Extensions on a protocol add **default implementations** that every conforming type gets for free. This is the heart of **protocol-oriented programming**: share behavior through protocols and extensions instead of base classes. A `where` clause limits an extension to conforming types that meet extra constraints.

NOTES

- Conforming types can still provide their own version
- Methods only in the extension (not the protocol) are statically dispatched
- `extension Collection where Element: Numeric` targets a subset
- This is how the stdlib gives every Collection `map`, `filter` …

DEFAULTS & CONSTRAINTS

```
protocol Greeter {
    var name: String { get }
    func greet() -> String                        // requirement
}
extension Greeter {
    func greet() -> String { "Hello, \(name)" }  // default
    func shout() -> String { greet().uppercased() }  // extra helper
}

struct English: Greeter { let name: String }
struct Pirate: Greeter {
    let name: String
    func greet() -> String { "Ahoy, \(name)" }   // custom
}
English(name: "Ada").greet()                      // "Hello, Ada"
Pirate(name: "Jack").shout()                      // "AHOY, JACK"

extension Collection where Element: Numeric {
    var total: Element { reduce(0, +) }
}
[1, 2, 3].total                                   // 6
[1.5, 2.5].total                                  // 4.0
```

CODE EXPLAINED

- → `extension Greeter` — default implementation
- → custom override — conforming type wins
- → extra helper — available to all conformers
- → `where Element: Numeric` — constrained extension

THE SWIFTY WAY

✗ `class BaseGreeter` with overridable methods
✓ `protocol Greeter` + extension defaults

DISPATCH RULES

WHERE DECLARED	DISPATCH
In protocol + extension	Dynamic (type's version wins)
Only in extension	Static (variable's type decides)
Class method	Dynamic (vtable)

COMING FROM…

- **Java** Default methods in interfaces ≈ protocol extensions
- **Kotlin** Interface default methods + extension functions
- **TypeScript** No equivalent; mixins come closest

⚠ COMMON MISTAKES

- Adding a method only in the extension, overriding it in a type, then calling through `any P`: the default runs
- Huge protocol extensions that hide where behavior comes from
- Constraining with `where Self: SomeClass` when a separate protocol is clearer

✓ TIPS & BEST PRACTICES

- List a method in the protocol body if conformers should be able to customize it
- Use constrained extensions to add APIs only where they make sense
- Combine with `some P` generics for zero-cost abstractions

QUICK SUMMARY

- extension on protocol = defaults
- `where` — constrained extensions
- protocol-oriented over class-based

Common Standard Protocols

EVERYDAY PROTOCOLS

A handful of standard protocols unlock most of the language's conveniences. For structs and enums, the compiler can **synthesize** `Equatable`, `Hashable`, `Comparable` (enums), and `Codable` just from the declaration, as long as every stored property conforms too.

NOTES

- Synthesis needs all stored properties to conform
- `Identifiable` needs an `id` that's `Hashable`
- `Sendable` marks types safe to share across concurrency domains
- `CustomStringConvertible` controls `print` output

CONFORMANCE IN ONE LINE

```
struct User: Identifiable, Hashable, Codable, Sendable {
    let id: UUID
    var name: String
    var age: Int
}

let a = User(id: UUID(), name: "Ada", age: 36)
let b = a
a == b                                  // true  (Equatable via Hashable)
Set([a, b]).count                       // 1     (Hashable)
let json = try JSONEncoder().encode(a)  // Codable

enum Priority: Comparable { case low, medium, high }
Priority.low < .high                    // true (synthesized order)
[Priority.high, .low].sorted()          // [.low, .high]

extension User: Comparable {
    static func < (l: User, r: User) -> Bool { l.name < r.name }
}
[a].sorted()                            // by name
```

CODE EXPLAINED

- → `Identifiable` — stable `id`
- → `Hashable` — Sets, Dictionary keys (includes Equatable)
- → `Codable` — JSON and plists
- → `Sendable` — safe across tasks
- → enum `Comparable` — case order, synthesized

THE SWIFTY WAY

✗ hand-written `==` comparing every field
✓ `: Equatable` (synthesized) — stays correct when fields change

COMMON PROTOCOLS

PROTOCOL	UNLOCKS
Equatable	== !=
Hashable	Set, Dictionary key
Comparable	< sorted() min() max()
Codable	Encode / decode
Identifiable	SwiftUI lists, diffing
Sendable	Cross-task sharing

COMING FROM…

- **Kotlin** `data class` gives equals/hashCode ≈ synthesis
- **Java** `record` auto-generates equals/hashCode
- **Python** `@dataclass(eq=True, frozen=True)`

IN SWIFTUI

- `List(users)` needs `Identifiable`
- `NavigationLink(value:)` needs `Hashable` values
- See SwiftUI 26 · Lists

⚠ COMMON MISTAKES

- Using an array index or changing value as `id`: breaks diffing
- Expecting synthesis when one property doesn't conform
- Marking a class `Sendable` with mutable state: compiler error

✓ TIPS & BEST PRACTICES

- Customize synthesis by implementing just `==` or `hash(into:)`
- Exclude a property from Codable with a custom `CodingKeys`
- Use `UUID()` or server IDs for `Identifiable.id`

QUICK SUMMARY

- Equatable / Hashable / Comparable
- Codable / Identifiable / Sendable
- synthesized for structs & enums

Extensions

EXTENDING TYPES

An `extension` adds computed properties, methods, initializers, nested types, subscripts and protocol conformances to an **existing** type, even ones you don't own like `String` or `Int`. Extensions can't add stored properties. They're also used to organize your own types: one extension per protocol conformance.

NOTES

- Extensions can't add stored properties or override existing methods
- Inits added in an extension keep a struct's memberwise init
- `private` members in the type are visible in same-file extensions
- Extending other modules' types with their protocols needs `@retroactive`

EXTENSIONS

```
extension Int {
    var isEven: Bool { isMultiple(of: 2) }
    func times(_ body: () -> Void) { for _ in 0..<self { body() } }
}
4.isEven                                    // true
3.times { print("hi") }                     // hi hi hi

extension String {
    var trimmed: String { trimmingCharacters(in: .whitespacesAndNewlines) }
}
"  hey \n".trimmed                          // "hey"

struct Player { var name: String; var score: Int }

// MARK: - CustomStringConvertible
extension Player: CustomStringConvertible {
    var description: String { "\(name) (\(score))" }
}

extension Player {
    enum Rank { case rookie, pro }          // nested type
    var rank: Rank { score > 100 ? .pro : .rookie }
}
```

CODE EXPLAINED

- → `extension Int` — add to a built-in type
- → computed property — no stored properties allowed
- → `extension X: Protocol` — conformance
- → nested type in an extension
- → `// MARK:` — one extension per concern

THE SWIFTY WAY

✗ `StringUtils.trim(s)` helper class
✓ `extension String { var trimmed }`

EXTENSIONS CAN / CAN'T

ADD	ALLOWED?
Computed properties	✔
Methods / subscripts	✔
Initializers	✔ (convenience for classes)
Protocol conformance	✔
Stored properties	✖
Overrides	✖

COMING FROM…

- **Kotlin** Extension functions ≈ extensions (but can add conformance here)
- **C#** Extension methods ≈ extensions
- **TypeScript** Declaration merging/prototype hacks; extensions are safe

⚠ COMMON MISTAKES

- Trying to add `var cache: [Int] = []` in an extension: stored property
- Polluting `String` with dozens of app-specific helpers
- Name clashes when two modules extend the same type with the same method

✓ TIPS & BEST PRACTICES

- Keep extensions on stdlib types `internal` or `private`
- Organize big types: main declaration + one extension per protocol
- Use constrained extensions: `extension Array where Element == User`

QUICK SUMMARY

- `extension T` — add behavior
- add conformance after the fact
- no stored properties

Generics

GENERIC CODE

Generics let one function or type work with **any type** that meets stated requirements. Type parameters go in angle brackets (<T>), and constraints (`T: Comparable`) or `where` clauses restrict them. Protocols declare their own placeholder types with `associatedtype`. The compiler specializes generic code, so it stays fast.

NOTES

- Array, Dictionary, Optional and Result are all generic
- Constraints: `<T: Hashable>` or `where T: Hashable`
- `associatedtype` = a generic placeholder inside a protocol
- Type inference usually fills in `T` at the call site

GENERIC FUNCTIONS & TYPES

```
func largest<T: Comparable>(_ items: [T]) -> T? {
    items.max()
}
largest([3, 9, 2])                          // Optional(9)
largest(["b", "a"])                         // Optional("b")

struct Stack<Element> {
    private var items: [Element] = []
    mutating func push(_ x: Element) { items.append(x) }
    mutating func pop() -> Element? { items.popLast() }
}
var s = Stack<Int>(); s.push(1); s.pop()    // Optional(1)

func countMatches<C: Collection>(_ c: C, _ x: C.Element) -> Int
    where C.Element: Equatable {
    c.filter { $0 == x }.count
}
countMatches("banana", "a")                 // 3

protocol Container {
    associatedtype Item
    mutating func push(_ x: Item)
}
extension Stack: Container {}               // Item inferred as Element
```

CODE EXPLAINED

- → `<T: Comparable>` — constrained type parameter
- → `Stack<Element>` — generic type
- → `where C.Element: Equatable` — extra constraints
- → `C.Element` — associated type of Collection
- → `associatedtype Item` — protocol placeholder

THE SWIFTY WAY

✗ `func f(_ x: Any) -> Any`
✓ `func f<T>(_ x: T) -> T` — keeps the type information

CONSTRAINT SYNTAX

SYNTAX	MEANING
`<T>`	Any type
`<T: P>`	Conforms to P
`<T: P & Q>`	Conforms to both
`where T == Int`	Exact type
`where C.Element: P`	Associated type constraint

COMING FROM…

- **Java** No type erasure: Swift specializes generics
- **Kotlin** `<T : Comparable<T>>` ≈ `<T: Comparable>`
- **TypeScript** `<T extends X>` ≈ `<T: X>`

⚠ COMMON MISTAKES

- Using generics where a concrete type or `some P` would be simpler
- Forgetting a constraint and getting "binary operator cannot be applied"
- Over-constraining with long `where` clauses nobody can read

✓ TIPS & BEST PRACTICES

- Start concrete, generalize when you have a second use
- `some Collection<Int>` often replaces a full generic signature
- Name type parameters meaningfully (`Element`, `Key`) unless it's a one-liner

QUICK SUMMARY

- `<T>` — type parameter
- `T: P` / `where` — constraints
- `associatedtype` — protocol generics

Opaque & Existential Types

SOME VS ANY

`some P` is an **opaque type**: one specific concrete type that conforms to P, chosen by the implementation and known to the compiler (fast, full type info). `any P` is an **existential**: a box that can hold *any* conforming type, which can change at runtime (flexible, small runtime cost). Use `some` by default and `any` when you truly need mixed types.

NOTES

- `some P` as a parameter = shorthand for a generic `<T: P>`
- A function returning `some P` must return one concrete type on every path
- `any P` allows heterogeneous arrays: `[any Shape]`
- Primary associated types: `some Collection<Int>`

OPAQUE & EXISTENTIAL

```
protocol Shape { func area() -> Double }
struct Circle: Shape { var r = 1.0; func area() -> Double { .pi * r * r } }
struct Square: Shape { var s = 2.0; func area() -> Double { s * s } }

func makeShape() -> some Shape { Circle() }   // one hidden concrete type
// func bad(_ b: Bool) -> some Shape { b ? Circle() : Square() }  // error

var shapes: [any Shape] = [Circle(), Square()]   // mixed: needs any
shapes.append(Circle(r: 3))

func printArea(_ s: some Shape) { print(s.area()) }   // generic shorthand
func total(_ list: [any Shape]) -> Double { list.map { $0.area() }.reduce(0, +) }

func evens(_ xs: some Collection<Int>) -> [Int] { xs.filter { $0 % 2 == 0 } }
evens(1...10)                                  // [2, 4, 6, 8, 10]

var current: any Shape = Circle()
current = Square()                             // can change type
```

CODE EXPLAINED

- → `-> some Shape` — opaque return
- → `[any Shape]` — heterogeneous collection
- → `some Shape` parameter — generic shorthand
- → `some Collection<Int>` — primary associated type
- → `any Shape` variable — type may change

THE SWIFTY WAY

✗ `func f(_ s: any Shape)` for a single value
✓ `func f(_ s: some Shape)` — static dispatch, no boxing

COMING FROM…

- **Kotlin** Interfaces as types ≈ `any P`; no `some` equivalent
- **Rust** `impl Trait` ≈ `some`; `dyn Trait` ≈ `any`
- **TypeScript** Interfaces as types ≈ `any P`

WHICH TO USE

NEED	USE
Return a hidden concrete type	some P
Accept any one conforming type	some P
Store different types together	any P
Swap types at runtime	any P

SWIFT VERSIONS

- 5.1 — opaque return types (`some`)
- 5.6 — `any` keyword
- 5.7 — `some` parameters, primary associated types

IN SWIFTUI

- `var body: some View` — one concrete (huge) type the compiler knows
- See SwiftUI 55 · Performance & Debugging

⚠ COMMON MISTAKES

- Returning different concrete types from a `some P` function
- Using `any P` everywhere "to be safe": slower and loses type info
- Forgetting `any` before a protocol type in Swift 6 mode: warning/error

✓ TIPS & BEST PRACTICES

- Default to `some`; reach for `any` only for mixed collections
- You can often open an `any P` by passing it to a `some P` parameter
- Use `@ViewBuilder` to return different views from one function

QUICK SUMMARY

- `some P` — one hidden concrete type
- `any P` — box for any conformer
- prefer `some`

Subscripts & Callables

CUSTOM SUBSCRIPTS & CALLABLE TYPES

A `subscript` lets instances be indexed with `[ ]`, using any parameter types, labels and even several parameters. `callAsFunction` lets an instance be **called** like a function. `@dynamicMemberLookup` routes dot-syntax property access through a subscript, which powers wrappers like SwiftUI's `Binding` projections.

NOTES

- Subscripts can be get-only or get/set
- Static subscripts work on the type: `Config["key"]`
- `callAsFunction` can be overloaded and can take any parameters
- Dynamic member lookup with key paths stays type-safe

SUBSCRIPTS & CALLABLES

```
struct Grid {
    let cols: Int
    var cells: [Int]
    subscript(row: Int, col: Int) -> Int {
        get { cells[row * cols + col] }
        set { cells[row * cols + col] = newValue }
    }
}
var g = Grid(cols: 3, cells: Array(repeating: 0, count: 9))
g[row: 1, col: 2] = 7
g[row: 1, col: 2]                          // 7

struct Multiplier {
    let k: Int
    func callAsFunction(_ x: Int) -> Int { x * k }
}
let triple = Multiplier(k: 3)
triple(5)                                  // 15

@dynamicMemberLookup
struct Wrapper<T> {
    var value: T
    subscript<V>(dynamicMember kp: KeyPath<T, V>) -> V { value[keyPath: kp] }
}
Wrapper(value: "hello").count              // 5
```

CODE EXPLAINED

→ `subscript(row:col:)` — custom indexing

→ `get` / `set` — read and write

→ `callAsFunction` — instance is callable

→ `@dynamicMemberLookup` — forward properties

THE SWIFTY WAY

✗ `grid.value(atRow: 1, col: 2)`

✓ `grid[row: 1, col: 2]` — when access is the main operation

COMING FROM…

- **Python** `__getitem__` ≈ subscript; `__call__` ≈ callAsFunction
- **Kotlin** `operator fun get` / `invoke` ≈ same
- **TypeScript** Index signatures; no callable objects

SPECIAL MEMBERS

MEMBER	ENABLES
`subscript(_:)`	x[i]
`static subscript`	Type[key]
`callAsFunction`	x(args)
`subscript(dynamicMember:)`	x.anyName

⚠ COMMON MISTAKES

- Unchecked index math in subscripts: traps on bad input
- Using `@dynamicMemberLookup` with String keys and losing type safety
- Overusing `callAsFunction` where a named method reads better

✓ TIPS & BEST PRACTICES

- `subscript(safe i: Int) -> Element?` on Array gives optional access
- Key-path dynamic members give autocomplete and type checking
- SwiftUI's `$model.name` uses dynamic member lookup on `Bindable`

QUICK SUMMARY

- `subscript` — `[ ]` access
- `callAsFunction` — `x()`
- `@dynamicMemberLookup` — forwarded `.prop`

FileManager & Paths

LOCATING FILES

Foundation represents file locations as URLs, and `FileManager` performs file-system operations: checking existence, creating folders, listing, moving and deleting. On Apple platforms, apps are sandboxed, so you write into well-known folders such as **Documents**, **Caches** and **tmp**, found through static URL properties.

NOTES

- Prefer `URL` over path strings everywhere
- `URL.documentsDirectory` etc. need iOS 16 / macOS 13
- Documents is backed up; Caches can be purged by the system
- `appending(path:)` adds a component; `path()` returns the string

PATHS & FILEMANAGER

```
import Foundation
let fm = FileManager.default

let docs = URL.documentsDirectory
let file = docs.appending(path: "notes/today.txt")
file.lastPathComponent                          // "today.txt"
file.pathExtension                              // "txt"
file.deletingLastPathComponent()                // …/Documents/notes/

let folder = docs.appending(path: "notes")
try fm.createDirectory(at: folder, withIntermediateDirectories: true)
fm.fileExists(atPath: file.path())              // false

let items = try fm.contentsOfDirectory(at: docs,
                includingPropertiesForKeys: [.fileSizeKey])
for url in items where url.pathExtension == "txt" { print(url.lastPathComponent) }

try fm.moveItem(at: file, to: docs.appending(path: "old.txt"))
try fm.removeItem(at: folder)
let size = try file.resourceValues(forKeys: [.fileSizeKey]).fileSize
```

CODE EXPLAINED

- → `URL.documentsDirectory` — sandbox folder
- → `appending(path:)` — build a path
- → `createDirectory(… withIntermediateDirectories:)`
- → `contentsOfDirectory(at:…)` — list a folder
- → `moveItem` / `removeItem` — throwing operations

THE SWIFTY WAY

✗ `NSHomeDirectory() + "/Documents/x.txt"`
✓ `URL.documentsDirectory.appending(path: "x.txt")`

COMING FROM…

- **Python** `pathlib.Path` ≈ `URL`; `os` ≈ `FileManager`
- **Kotlin** `File(context.filesDir, …)` ≈ documentsDirectory
- **TypeScript** Node `path` + `fs` ≈ URL + FileManager

APP FOLDERS

URL	USE
`documentsDirectory`	User data (backed up)
`applicationSupportDirectory`	App data, hidden
`cachesDirectory`	Re-creatable data
`temporaryDirectory`	Short-lived files
`Bundle.main.url(…)`	Read-only bundled files

⚠ COMMON MISTAKES

- Hard-coding absolute paths: the sandbox path changes between installs
- Using `url.absoluteString` as a file path: it includes `file://`
- Storing big re-downloadable files in Documents: bloats backups

✓ TIPS & BEST PRACTICES

- Use `url.path(percentEncoded: false)` for display
- Check `fileExists` only for UI; just try the operation and catch errors
- Create app-support folders at launch with intermediate directories

QUICK SUMMARY

- `URL` — file locations
- `FileManager.default` — file ops
- `documentsDirectory` / `cachesDirectory`

Reading & Writing Files

READING & WRITING

Small files can be read and written in one call: `String(contentsOf:encoding:)` and `Data(contentsOf:)` for reading, `write(to:atomically:encoding:)` and `Data.write(to:options:)` for writing. For large files, stream with `FileHandle` or iterate lines asynchronously with `url.lines`. All of these **throw**.

NOTES

- `atomically: true` writes to a temp file, then swaps it in
- `Data` is the raw-bytes type
- `url.lines` is an `AsyncSequence` (iOS 15+)
- Specify `.utf8` encoding explicitly

READ, WRITE, APPEND

```swift
import Foundation
let url = URL.documentsDirectory.appending(path: "log.txt")

try "first line\n".write(to: url, atomically: true, encoding: .utf8)
let text = try String(contentsOf: url, encoding: .utf8)   // "first line\n"

let data = try Data(contentsOf: url)                      // bytes
try data.write(to: url, options: [.atomic, .completeFileProtection])

// append
let h = try FileHandle(forWritingTo: url)
defer { try? h.close() }
try h.seekToEnd()
try h.write(contentsOf: Data("second line\n".utf8))

// large files: stream lines
for try await line in url.lines {
    print(line)                                           // first line …
}

let lines = text.split(whereSeparator: \.isNewline)       // small files
```

CODE EXPLAINED

- → `write(to:atomically:encoding:)` — save text
- → `String(contentsOf:encoding:)` — load text
- → `Data(contentsOf:)` / `data.write(to:)` — bytes
- → `FileHandle` + `seekToEnd` — append
- → `url.lines` — async line streaming

THE SWIFTY WAY

✗ reading a 2 GB log with `String(contentsOf:)`
✓ `for try await line in url.lines` — constant memory

COMING FROM…

- **Python** `open(p).read()` ≈ `String(contentsOf:)`
- **Kotlin** `File.readText()` / `writeText()` ≈ same calls
- **TypeScript** `fs.readFileSync` / `fs.promises` ≈ these

WRITE OPTIONS

OPTION	EFFECT
`.atomic`	Temp file + swap
`.withoutOverwriting`	Fail if file exists
`.completeFileProtection`	Encrypted while locked
`.noFileProtection`	Always readable

⚠ COMMON MISTAKES

- Using `String(contentsOf:)` without `encoding:` (deprecated)
- Writing on the main thread in a hot path: move large I/O to a background task
- Forgetting `try`: every file call throws

✓ TIPS & BEST PRACTICES

- Wrap file access in a small type that owns the URL and errors
- Use `Data("text".utf8)` to turn Strings into bytes without optionals
- Use `FileHandle.standardInput` / `readLine()` for CLI input

QUICK SUMMARY

- `String(contentsOf:)` / `write(to:)` — text
- `Data` — bytes
- `FileHandle` / `url.lines` — stream

Codable & JSON

JSON WITH CODABLE

Conform a type to `Codable` (= `Encodable & Decodable`) and `JSONEncoder` / `JSONDecoder` convert it to and from JSON automatically. Property names map to JSON keys; when they differ, declare a `CodingKeys` enum or use a key strategy such as `.convertFromSnakeCase`. Optionals become missing or `null` keys.

NOTES

- All stored properties must be Codable (String, Int, arrays, nested Codables…)
- Extra JSON keys are ignored by default
- Missing non-optional keys throw `keyNotFound`
- Dates need a `dateDecodingStrategy`

ENCODE & DECODE

```swift
import Foundation
struct Post: Codable {
    let id: Int
    let title: String
    let createdAt: Date
    var tags: [String]?
    enum CodingKeys: String, CodingKey {
        case id, title, tags
        case createdAt = "created"                // JSON key differs
    }
}

let json = #"{"id":1,"title":"Hi","created":"2026-10-26T12:00:00Z"}"#
let dec = JSONDecoder()
dec.dateDecodingStrategy = .iso8601
let post = try dec.decode(Post.self, from: Data(json.utf8))
post.tags                                         // nil

let enc = JSONEncoder()
enc.outputFormatting = [.prettyPrinted, .sortedKeys]
enc.dateEncodingStrategy = .iso8601
let out = String(decoding: try enc.encode(post), as: UTF8.self)
let posts = try dec.decode([Post].self, from: arrayData)   // arrays too
```

CODE EXPLAINED

- → `Codable` — encode + decode
- → `CodingKeys` — rename keys
- → `decode(Post.self, from:)` — JSON → struct
- → `.iso8601` — date strategy
- → `.prettyPrinted, .sortedKeys` — readable output

THE SWIFTY WAY

✗ `JSONSerialization` + `as? [String: Any]` casts
✓ a `Codable` struct

COMING FROM…

- **Kotlin** kotlinx.serialization `@Serializable` ≈ Codable
- **TypeScript** `JSON.parse` has no types; Codable validates
- **Python** pydantic models ≈ Codable structs

DECODER STRATEGIES

PROPERTY	COMMON VALUE
keyDecodingStrategy	.convertFromSnakeCase
dateDecodingStrategy	.iso8601 / .secondsSince1970
dataDecodingStrategy	.base64
nonConformingFloatDecodingStrategy	NaN/∞ strings

⚠ COMMON MISTAKES

- Non-optional property for a key the server sometimes omits: whole decode fails
- Forgetting the date strategy: default expects seconds since 2001
- Mixing `CodingKeys` and `.convertFromSnakeCase` incorrectly (keys are converted first)

✓ TIPS & BEST PRACTICES

- Print `DecodingError` in full while debugging; it names the failing key path
- Make fields optional only when the API can omit them
- Use `.convertFromSnakeCase` instead of `CodingKeys` for simple snake_case APIs

QUICK SUMMARY

- `Codable` + `JSONDecoder` / `JSONEncoder`
- `CodingKeys` — rename
- strategies — keys, dates

Codable — Custom Coding

HAND-WRITTEN CODING

When the JSON shape doesn't match your model, implement `init(from:)` and `encode(to:)` yourself. You work with **containers**: a keyed container for objects, an unkeyed container for arrays, and a single-value container for plain values. Nested containers flatten or build nested JSON.

NOTES

- Implementing one side doesn't stop synthesis of the other
- `decodeIfPresent` returns nil for missing keys
- Use `try container.decode(T.self, forKey:)` for required fields
- `PropertyListEncoder` works with the same Codable types

CUSTOM INIT(FROM:)

```
// JSON: {"name":"Ada","stats":{"age":36},"role":"ADMIN"}
struct User: Codable {
    var name: String
    var age: Int
    var isAdmin: Bool

    enum CodingKeys: String, CodingKey { case name, stats, role }
    enum StatsKeys: String, CodingKey { case age }

    init(from decoder: Decoder) throws {
        let c = try decoder.container(keyedBy: CodingKeys.self)
        name = try c.decode(String.self, forKey: .name)
        let stats = try c.nestedContainer(keyedBy: StatsKeys.self, forKey: .stats)
        age = try stats.decodeIfPresent(Int.self, forKey: .age) ?? 0
        isAdmin = (try c.decodeIfPresent(String.self, forKey: .role)) == "ADMIN"
    }

    func encode(to encoder: Encoder) throws {
        var c = encoder.container(keyedBy: CodingKeys.self)
        try c.encode(name, forKey: .name)
        var stats = c.nestedContainer(keyedBy: StatsKeys.self, forKey: .stats)
        try stats.encode(age, forKey: .age)
        try c.encode(isAdmin ? "ADMIN" : "USER", forKey: .role)
    }
}
```

CODE EXPLAINED

- → `container(keyedBy:)` — JSON object
- → `nestedContainer` — reach into a sub-object
- → `decodeIfPresent … ?? 0` — default for missing
- → transform on decode — String → Bool
- → `encode(to:)` — mirror the shape back

THE SWIFTY WAY

✗ an API-shaped model used across the app
✓ decode into a DTO, map to a clean model
— keeps API quirks at the edge

COMING FROM…

- **Kotlin** Custom `KSerializer` ≈ init(from:)
- **Java** Jackson `@JsonDeserialize` ≈ custom decoding
- **Python** pydantic validators ≈ custom decode logic

CONTAINER TYPES

CONTAINER	JSON SHAPE
KeyedDecodingContainer	{ "k": v }
UnkeyedDecodingContainer	[v, v]
SingleValueDecodingContainer	v
nestedContainer	{ "k": { … } }

⚠ COMMON MISTAKES

- Forgetting a key in `CodingKeys` : that property silently isn't coded
- Using `decode` for optional fields: throws on missing key
- Encoding a different shape than you decode: round-trips break

✓ TIPS & BEST PRACTICES

- Decode enums with unknown values safely via a single-value container and a fallback case
- Use a property wrapper for repeated custom rules (e.g. default values)
- Write a round-trip test: encode → decode → `==`

QUICK SUMMARY

- `init(from:)` / `encode(to:)` — custom
- keyed / unkeyed / single-value containers
- `decodeIfPresent` — optional keys

UserDefaults & Keychain

SMALL PERSISTENT VALUES

`UserDefaults` stores small preferences (Bools, numbers, strings, dates, small Data) as a property list, keyed by string. It is **not encrypted**, so secrets like tokens and passwords belong in the **Keychain**, Apple's encrypted credential store, accessed through the `SecItem…` C API (or a small wrapper library).

NOTES

- UserDefaults is loaded into memory: keep it small
- `register(defaults:)` sets fallback values (not saved)
- App groups share defaults with extensions via `suiteName`
- Keychain items survive app deletion on iOS by default

USERDEFAULTS & KEYCHAIN

```
import Foundation
let d = UserDefaults.standard
d.register(defaults: ["volume": 0.8])
d.set(true, forKey: "onboarded")
d.set("dark", forKey: "theme")
d.bool(forKey: "onboarded")                 // true
d.double(forKey: "volume")                  // 0.8 (registered)
d.string(forKey: "missing")                 // nil
d.removeObject(forKey: "theme")

import Security
func saveToken(_ token: String) -> Bool {
    let q: [String: Any] = [
        kSecClass as String: kSecClassGenericPassword,
        kSecAttrAccount as String: "apiToken",
        kSecValueData as String: Data(token.utf8),
    ]
    SecItemDelete(q as CFDictionary)        // replace if present
    return SecItemAdd(q as CFDictionary, nil) == errSecSuccess
}
```

CODE EXPLAINED

- → `register(defaults:)` — fallback values
- → `set(_:forKey:)` / `bool(forKey:)` — read/write
- → `removeObject(forKey:)` — delete
- → `kSecClassGenericPassword` — keychain item kind
- → `SecItemAdd` → `errSecSuccess` — store securely

THE SWIFTY WAY

✗ `UserDefaults.standard.set(token, forKey: "token")`
✓ store tokens in the Keychain — UserDefaults is a plain-text plist

COMING FROM…

- **Kotlin** SharedPreferences ≈ UserDefaults; Keystore ≈ Keychain
- **TypeScript** localStorage ≈ UserDefaults (also not secure)
- **Python** keyring library ≈ Keychain

WHAT GOES WHERE

DATA	STORE IN
Settings, flags	UserDefaults / @AppStorage
Tokens, passwords	Keychain
Documents, images	Files
Structured records	SwiftData / SQLite

⚠ COMMON MISTAKES

- Storing large arrays or images in UserDefaults: slows launch
- Storing secrets in UserDefaults or Info.plist
- Ignoring the `OSStatus` from `SecItemAdd`: failures go unnoticed

✓ TIPS & BEST PRACTICES

- Wrap keys in a `enum DefaultsKey: String` to avoid typos
- Use `@AppStorage("theme")` in SwiftUI to bind to UserDefaults
- Read with `SecItemCopyMatching` using `kSecReturnData: true`

QUICK SUMMARY

- `UserDefaults` — small prefs
- Keychain — secrets
- `SecItemAdd` / `SecItemCopyMatching`

Modules & Imports

MODULES AND IMPORT

A **module** is a unit of code distribution: an app target, a framework, or a Swift package target. Everything inside a module shares one namespace, so files don't import each other. You `import` other modules to use their `public` API. Foundation and Swift's standard library are the two you'll see most.

NOTES

- The stdlib (`Swift` module) is imported automatically
- Two modules can define the same name; qualify with `Module.Name`
- `@testable import` exposes `internal` symbols to tests
- `import struct Module.Type` imports a single declaration

IMPORT FORMS

```
import Foundation                  // dates, URLs, files, JSON
import SwiftUI                     // UI (Apple platforms)
import OSLog                       // Logger
import struct Foundation.UUID      // just one type
@testable import MyApp             // tests can see internal

import Collections                 // a package product

#if canImport(UIKit)
import UIKit
#endif

// Name clash: qualify with the module
let a = Swift.Result<Int, Error>.success(1)
let b = MyApp.Result.ok            // your own type named Result

// Same module = no import needed
// File A.swift: struct Helper {}
// File B.swift: let h = Helper()
```

CODE EXPLAINED

- → `import Foundation` — the core library
- → `import struct M.T` — import one symbol
- → `@testable import` — tests see internals
- → `#if canImport` — optional imports
- → `Swift.Result` — module-qualified name

THE SWIFTY WAY

✗ `import UIKit` in every model file
✓ import only what the file uses — keeps models portable and builds faster

COMING FROM…

- **Java** `import pkg.Class` ≈ `import struct Module.Type`
- **Python** `import x` ≈ `import X`; no per-file imports inside a module
- **TypeScript** No per-file imports needed inside a module

COMMON MODULES

MODULE	PROVIDES
`Foundation`	Date, URL, Data, JSON, FileManager
`SwiftUI`	Declarative UI
`Observation`	@Observable
`OSLog`	Logger
`Testing`	Swift Testing
`SwiftData`	@Model persistence

⚠ COMMON MISTAKES

- Expecting `import` between files in the same target: not needed
- Naming your type the same as a common module type (`Result`, `Task`)
- Importing SwiftUI in a package that should build on Linux

✓ TIPS & BEST PRACTICES

- Split large apps into local Swift packages (one module per feature)
- Use `@_exported import` sparingly: it re-exports a module
- Watch build times: fewer imports per file helps incremental builds

QUICK SUMMARY

- module = target / package product
- `import` other modules only
- `@testable import` for tests

Swift Package Manager

SWIFTPM

The **Swift Package Manager** builds, tests and manages dependencies for Swift code. A package is described by `Package.swift`, a Swift file that declares **products** (libraries or executables), **targets** (modules), and **dependencies** on other packages. SwiftPM resolves versions with semantic versioning and records them in `Package.resolved`.

NOTES

- The first line `// swift-tools-version:` sets the manifest API version
- Sources go in `Sources/<Target>/`, tests in `Tests/<Target>Tests/`
- Commit `Package.resolved` for apps; libraries usually don't
- Xcode opens a `Package.swift` directly

PACKAGE.SWIFT

```swift
// swift-tools-version: 6.0
import PackageDescription

let package = Package(
    name: "Weather",
    platforms: [.iOS(.v17), .macOS(.v14)],
    products: [
        .library(name: "WeatherKitLite", targets: ["WeatherKitLite"]),
        .executable(name: "weather", targets: ["CLI"]),
    ],
    dependencies: [
        .package(url: "https://github.com/apple/swift-argument-parser",
                 from: "1.5.0"),
    ],
    targets: [
        .target(name: "WeatherKitLite"),
        .executableTarget(name: "CLI", dependencies: [
            "WeatherKitLite",
            .product(name: "ArgumentParser", package: "swift-argument-parser"),
        ]),
        .testTarget(name: "WeatherKitLiteTests", dependencies: ["WeatherKitLite"]),
    ]
)
```

CODE EXPLAINED

- → `swift-tools-version` — manifest version
- → `platforms` — minimum OS versions
- → `products` — what others can use
- → `.package(url:from:)` — up to next major
- → `.product(name:package:)` — depend on a product

THE SWIFTY WAY

✗ `.package(url: …, branch: "main")` in releases

✓ `.package(url: …, from: "1.5.0")` — branches aren't reproducible

COMING FROM…

- **TypeScript** `Package.swift` ≈ package.json; resolved ≈ lockfile
- **Kotlin** ≈ Gradle build script with dependencies
- **Python** ≈ pyproject.toml + lock file

CLI COMMANDS

COMMAND	DOES
`swift package init --type library`	New package
`swift build [-c release]`	Build
`swift run <exe>`	Build + run
`swift test`	Run tests
`swift package update`	Update deps
`swift package show-dependencies`	Dependency tree

⚠ COMMON MISTAKES

- Mismatching target names and folder names under `Sources/`
- Depending on a package name instead of its product name
- Pinning `exact:` versions everywhere: blocks bug-fix updates

✓ TIPS & BEST PRACTICES

- Use `.upToNextMinor(from:)` for 0.x packages
- Add resources with `resources: [.process("Assets")]` and read via `Bundle.module`
- Search swiftpackageindex.com for well-maintained packages

QUICK SUMMARY

- `Package.swift` — products, targets, deps
- `swift build / run / test`
- `Package.resolved` — locked versions

Packages in Xcode

USING PACKAGES IN XCODE

Xcode has SwiftPM built in. Add a remote package through **File › Add Package Dependencies…**, pick a version rule, and choose which products to link to which targets. Local packages (a folder with `Package.swift`) can be dragged into the project to split an app into modules you edit side by side.

NOTES

- Locked versions: `Package.resolved`, inside the project's `xcshareddata/swiftpm` folder
- Package settings are under the project's **Package Dependencies** tab
- Local packages override a remote package with the same name
- CI: `xcodebuild -resolvePackageDependencies`

TYPICAL WORKFLOW

```
// 1. File › Add Package Dependencies…
//    URL: https://github.com/apple/swift-collections
//    Rule: Up to Next Major Version  1.1.0 < 2.0.0
//    Add product "Collections" to target "MyApp"

// 2. Use it
import Collections
var recent: OrderedSet = ["home", "search"]

// 3. Local package for a feature module
//    File › New › Package…  → Packages/Profile
//    Drag Packages/Profile into the project navigator
//    Target › General › Frameworks: + Profile
import Profile

// 4. Housekeeping (File › Packages)
//    Resolve Package Versions
//    Update to Latest Package Versions
//    Reset Package Caches              (fixes odd resolve errors)
```

CODE EXPLAINED

→ Add Package Dependencies — search or paste a URL

→ version rule — how updates are allowed

→ link product to target — needed before `import`

→ local package — feature modules in-repo

→ Reset Package Caches — first fix for weird errors

THE SWIFTY WAY

✗ copying a library's source files into the app
✓ add it as a package dependency — updates and licenses stay clear

COMING FROM…

- **Kotlin** Adding a Gradle dependency in Android Studio
- **TypeScript** `npm install` + import
- **Python** `pip install` + requirements file

VERSION RULES

RULE	ACCEPTS
`Up to Next Major`	1.2.0 ..< 2.0.0
`Up to Next Minor`	1.2.0 ..< 1.3.0
`Range`	Custom lower ..< upper
`Exact`	1.2.3 only
`Branch / Commit`	Moving / fixed ref

⚠ COMMON MISTAKES

- Adding the package but not linking its product to the target: "No such module"
- Not committing `Package.resolved` for apps: teammates get different versions
- Pointing at a branch for production builds

✓ TIPS & BEST PRACTICES

- Group local packages in a `Packages/` folder
- Use `swift package dump-package` to inspect a package's products
- When resolution fails after branch switches, Reset Package Caches first

QUICK SUMMARY

- File › Add Package Dependencies…
- link products to targets
- local packages = in-repo modules

Foundation — Dates & Times

WORKING WITH TIME

`Date` is a single point in time, independent of time zones and calendars. To get human parts like day or month, ask a `Calendar` (which knows the user's locale and time zone). For display, use `.formatted()`; for exchanging with servers, use ISO 8601. `Duration` and clocks measure elapsed time.

NOTES

- `Date.now` is the current moment
- Never add 86 400 seconds for "one day": use `Calendar`
- `DateComponents` holds year/month/day/hour pieces
- Use `ContinuousClock` to measure code

DATES & CALENDAR

```
import Foundation
let now = Date.now
let cal = Calendar.current

let nextWeek = cal.date(byAdding: .day, value: 7, to: now)!
cal.component(.weekday, from: now)                // 1 = Sunday
cal.isDateInToday(now)                            // true
cal.dateComponents([.day], from: now, to: nextWeek).day   // 7

let birthday = cal.date(from: DateComponents(year: 1990, month: 4, day: 2))!

now.formatted(date: .abbreviated, time: .shortened)   // "Oct 26, 2026, 9:41 AM"
now.formatted(.iso8601)                           // "2026-10-26T14:41:00Z"
now.formatted(.relative(presentation: .named))    // "now"
let parsed = try Date("2026-10-26T12:00:00Z", strategy: .iso8601)

now.timeIntervalSince(birthday) / 86_400          // days (approx)

let elapsed = ContinuousClock().measure { heavyWork() }
print(elapsed)                                    // 0.123 seconds
try await Task.sleep(for: .seconds(2))
```

CODE EXPLAINED

- → `Date.now` — current instant
- → `date(byAdding:value:to:)` — calendar math
- → `DateComponents` — build a date from parts
- → `.formatted(...)` — display / ISO 8601
- → `ContinuousClock().measure` — timing

THE SWIFTY WAY

✗ `date.addingTimeInterval(86_400)`
✓ `cal.date(byAdding: .day, value: 1, to: date)` — DST days aren't 24 h

COMING FROM…

- **Java** `Instant` ≈ Date; `ZonedDateTime` ≈ Calendar math
- **Python** `datetime` with tz ≈ Date + Calendar
- **TypeScript** JS `Date` ≈ Date; Intl ≈ formatted()

CALENDAR HELPERS

CALL	RETURNS
`startOfDay(for:)`	Midnight of that day
`isDate(_:inSameDayAs:)`	Bool
`isDateInWeekend(_:)`	Bool
`range(of: .day, in: .month, for:)`	Days in month

⚠ COMMON MISTAKES

- `DateFormatter` without `en_US_POSIX` locale for fixed formats
- Storing local time strings instead of UTC ISO 8601
- Comparing dates for "same day" with `==`

✓ TIPS & BEST PRACTICES

- Use `Date.ISO8601FormatStyle` for API timestamps
- Reuse formatters; creating `DateFormatter` repeatedly is slow
- Use `Text(date, style: .relative)` in SwiftUI

QUICK SUMMARY

- `Date` — instant; `Calendar` — human units
- `.formatted()` / ISO 8601
- `Duration` / clocks — elapsed time

Foundation — Math & Random

NUMBERS, MATH & RANDOMNESS

Basic math (`abs`, `min`, `max`, `squareRoot()`, `rounded()`) is in the standard library; C-style functions like `pow`, `sin` and `log` come from Foundation. Every numeric type has `random(in:)`, and collections have `randomElement()` and `shuffled()`. `Measurement` adds unit-aware values and conversions.

NOTES

- `random(in:)` uses a cryptographically secure generator by default
- Pass your own `RandomNumberGenerator` for repeatable results
- `Double.pi`, `.squareRoot()` need no Foundation
- `Measurement` formats in the user's preferred units

MATH & RANDOM

```
import Foundation
abs(-7)                                  // 7
min(3, 8, 1); max(3, 8, 1)               // 1, 8
16.0.squareRoot()                        // 4.0
pow(2.0, 10)                             // 1024.0
sin(Double.pi / 2)                       // 1.0
log10(1000.0)                            // 3.0
(7.0).rounded(.up)                       // 7.0

Int.random(in: 1...6)                    // dice roll
Double.random(in: 0..<1)
Bool.random()
["red", "green", "blue"].randomElement()  // Optional("green")
[1, 2, 3, 4].shuffled()

let run = Measurement(value: 5, unit: UnitLength.kilometers)
run.converted(to: .miles).value          // 3.106…
run.formatted()                          // "5 km" (or "3.1 mi" in US)

var g = SystemRandomNumberGenerator()
Int.random(in: 1...100, using: &g)
```

CODE EXPLAINED

- → `abs` / `min` / `max` — stdlib
- → `pow` / `sin` / `log10` — Foundation
- → `random(in:)` — secure random by default
- → `randomElement()` / `shuffled()` — collections
- → `Measurement` — unit conversion

THE SWIFTY WAY

✗ `arr[Int(arc4random_uniform(UInt32(arr.count)))]`

✓ `arr.randomElement()`

COMING FROM…

- **Python** `random.randint(1, 6)` ≈ `Int.random(in: 1...6)`
- **TypeScript** `Math.random()` ≈ `Double.random(in: 0..<1)`
- **Kotlin** `(1..6).random()` ≈ same

UNIT TYPES

DIMENSION	EXAMPLES
`UnitLength`	.meters, .miles
`UnitMass`	.kilograms, .pounds
`UnitTemperature`	.celsius, .fahrenheit
`UnitDuration`	.seconds, .hours
`UnitSpeed`	.kilometersPerHour

⚠ COMMON MISTAKES

- Calling `pow` with Int arguments: it expects floating point
- Seeding your own generator for security-sensitive values
- Showing raw unit numbers instead of `Measurement.formatted()`

✓ TIPS & BEST PRACTICES

- Write a seeded generator for reproducible tests and games
- Use Swift Numerics for complex numbers and more math functions
- `Decimal` for money; `Double` for measurements

QUICK SUMMARY

- stdlib + Foundation math functions
- `random(in:)` / `randomElement()`
- `Measurement` — units

Foundation — URLs & System

URLS, ENVIRONMENT & BUNDLES

`URLComponents` builds and edits URLs safely, handling percent-encoding of query items for you. `ProcessInfo` exposes the environment, arguments and system details. `Bundle` reads resources and Info.plist values shipped with the app. `Locale` describes the user's language, region and formatting preferences.

NOTES

- `URL(string:)` returns nil for invalid strings
- Build query strings with `URLQueryItem`, never by concatenation
- `Bundle.main` = app; `Bundle.module` = package resources
- Environment variables are set in Xcode's scheme editor

URLS & SYSTEM INFO

```swift
import Foundation
var c = URLComponents()
c.scheme = "https"
c.host = "api.example.com"
c.path = "/search"
c.queryItems = [URLQueryItem(name: "q", value: "swift & ios"),
                URLQueryItem(name: "page", value: "2")]
c.url   // …/search?q=swift%20%26%20ios&page=2

let u = URL(string: "https://example.com:8080/a/b?x=1")!
u.host()                  // "example.com"
u.port                    // 8080
u.query()                 // "x=1"

let env = ProcessInfo.processInfo.environment
let apiKey = env["API_KEY"] ?? ""
ProcessInfo.processInfo.isLowPowerModeEnabled
CommandLine.arguments     // ["/path/to/tool", …]

let version = Bundle.main.infoDictionary?["CFBundleShortVersionString"] as? String
let seed = Bundle.main.url(forResource: "seed", withExtension: "json")
Locale.current.identifier // "en_US"
```

CODE EXPLAINED

- → `URLComponents` — safe URL building
- → `URLQueryItem` — encoded query pairs
- → `host()` / `port` / `query()` — read URL parts
- → `ProcessInfo` — env and system state
- → `Bundle.main` — resources and Info.plist

THE SWIFTY WAY

✗ `URL(string: base + "?q=" + text)!`
✓ `URLComponents` + `queryItems` — encodes `&`, spaces, Unicode

COMING FROM…

- **TypeScript** `new URL()` + `searchParams` ≈ URLComponents
- **Python** `urllib.parse.urlencode` ≈ queryItems
- **Kotlin** `Uri.Builder` ≈ URLComponents

INFO.PLIST KEYS

KEY	VALUE
CFBundleShortVersionString	1.4.0 (marketing)
CFBundleVersion	Build number
CFBundleIdentifier	com.you.app
CFBundleDisplayName	Home-screen name

⚠ COMMON MISTAKES

- Force-unwrapping `URL(string:)` built from user input
- Shipping API keys in the bundle: anyone can extract them
- Hard-coding `"en"` instead of reading `Locale.current`

✓ TIPS & BEST PRACTICES

- Use `Bundle.main.bundleIdentifier` as your Logger subsystem
- `URL.appending(queryItems:)` (iOS 16+) adds query items to a URL
- Read build settings into code via Info.plist `$(VARIABLE)` entries

QUICK SUMMARY

- `URLComponents` — build URLs
- `ProcessInfo` / `CommandLine` — environment
- `Bundle` / `Locale` — app & user info

Swift Testing

SWIFT TESTING

Swift Testing is the modern test framework that ships with Xcode 16 and the Swift 6 toolchain. Tests are plain functions marked `@Test`; checks use the `#expect` and `#require` macros, which print the actual values on failure. Suites are just types, and parameterized tests run one function over many inputs.

NOTES

- Runs alongside XCTest in the same target
- Tests run in parallel by default
- `#require` stops the test on failure; `#expect` continues
- Tests can be `async` and `throws`

TESTS WITH @TEST

```
import Testing
@testable import MyApp

@Test func addition() {
    #expect(2 + 2 == 4)
}

@Suite("Cart")
struct CartTests {
    var cart = Cart()                          // fresh per test

    @Test("Adding an item increases count")
    mutating func add() throws {
        cart.add("apple")
        #expect(cart.count == 1)
        let first = try #require(cart.items.first)   // stop if nil
        #expect(first == "apple")
    }

    @Test(arguments: ["", " ", "\n"])
    func rejectsBlank(name: String) {
        #expect(throws: CartError.blankName) { try Cart().add(named: name) }
    }

}
```

CODE EXPLAINED

- → `@Test` — mark a test function
- → `#expect(cond)` — check, show values on failure
- → `#require` — unwrap or stop the test
- → `arguments:` — parameterized test
- → `@Suite` — group tests in a type

THE SWIFTY WAY

✗ `XCTAssertEqual(a, b)` in new tests
✓ `#expect(a == b)` — one macro, better failure messages

COMING FROM…

- **Kotlin** JUnit 5 `@ParameterizedTest` ≈ `arguments:`
- **TypeScript** Jest `test.each` ≈ `arguments:`
- **Python** pytest functions + `parametrize` ≈ @Test

TRAITS

TRAIT	EFFECT
`.disabled("reason")`	Skip
`.enabled(if: cond)`	Conditional
`.tags(.x)`	Group / filter
`.timeLimit(.minutes(1))`	Fail if slow
`.serialized`	No parallelism (suite)
`.bug("url")`	Link an issue

SWIFT VERSIONS

- 6.0 toolchain / Xcode 16 — Swift Testing ships built in

⚠ COMMON MISTAKES

- Sharing mutable static state between parallel tests
- Using `#expect(x != nil)` then force-unwrapping: use `try #require(x)`
- Forgetting `@testable import` for internal symbols

✓ TIPS & BEST PRACTICES

- Run from the CLI with `swift test --filter CartTests`
- Use `confirmation { }` to test callbacks or events
- Use `withKnownIssue { }` for expected failures

QUICK SUMMARY

- `@Test` + `#expect` / `#require`
- `arguments:` — parameterized
- `@Suite` + traits — organize

XCTest

XCTEST

XCTest is the long-standing test framework, still required for **UI tests** and **performance tests** and found in most existing projects. Tests are methods starting with `test` inside an `XCTestCase` subclass, with `setUp`/`tearDown` around each one and `XCTAssert…` functions for checks.

NOTES

- Test methods must start with `test` and take no arguments
- A new instance of the class is created for each test
- UI testing uses `XCUIApplication` (XCTest only)
- Can coexist with Swift Testing in one target

XCTESTCASE

```
import XCTest
@testable import MyApp

final class CartTests: XCTestCase {
    var cart: Cart!

    override func setUp() { cart = Cart() }
    override func tearDown() { cart = nil }

    func testAddIncreasesCount() {
        cart.add("apple")
        XCTAssertEqual(cart.count, 1)
    }

    func testBlankNameThrows() {
        XCTAssertThrowsError(try cart.add(named: "")) { error in
            XCTAssertEqual(error as? CartError, .blankName)
        }
    }

    func testFetch() async throws {
        let items = try await API.fetch()
        XCTAssertFalse(items.isEmpty)
    }

    func testPerformance() { measure { _ = (1...10_000).sorted(by: >) } }
}
```

CODE EXPLAINED

- → `XCTestCase` — test class
- → `setUp` / `tearDown` — per-test fixtures
- → `XCTAssertEqual` / `XCTAssertThrowsError`
- → `async throws` test methods
- → `measure { }` — performance baseline

THE SWIFTY WAY

✗ `XCTAssertTrue(a == b)`
✓ `XCTAssertEqual(a, b)` — the failure message shows both values

ASSERTIONS

FUNCTION	CHECKS
`XCTAssertEqual(a, b)`	a == b
`XCTAssertTrue / False`	Bool
`XCTAssertNil / NotNil`	Optional
`XCTAssertThrowsError`	Throws
`XCTUnwrap(x)`	Unwrap or fail

COMING FROM…

- **Java** JUnit 4 `TestCase` ≈ XCTestCase
- **Python** `unittest.TestCase` is almost identical
- **TypeScript** Jest `beforeEach` ≈ setUp

⚠ COMMON MISTAKES

- Naming a method `checkAdd()` : it won't run without the `test` prefix
- Leaving state in static vars between tests
- Using `XCTestExpectation` + `wait` for async code that could just be `async`

✓ TIPS & BEST PRACTICES

- Use `try XCTUnwrap(x)` instead of force unwrapping in tests
- Run one test: click the diamond in the gutter, or `-only-testing:` in xcodebuild
- Keep XCTest for UI tests; write new unit tests with Swift Testing

QUICK SUMMARY

- `XCTestCase` + `test…` methods
- `XCTAssert…` family
- needed for UI & performance tests

Logging with OSLog

LOGGING WITH OSLOG

`Logger` from the **OSLog** framework is Apple's unified logging API. Messages are cheap, tagged with a **subsystem** and **category**, filterable by level in Xcode's console and Console.app, and **private by default**: dynamic values are redacted in logs collected off-device unless you mark them public.

NOTES

- Interpolated values are redacted unless `privacy: .public`
- Debug-level messages aren't persisted
- View logs in Xcode's console, Console.app or `log stream`
- Prefer Logger over `print` in app code

LOGGER

```
import OSLog

extension Logger {
    static let network = Logger(subsystem: "com.example.app", category: "network")
    static let ui = Logger(subsystem: "com.example.app", category: "ui")
}

let userID = "u_42", ms = 128
Logger.network.debug("Starting request")
Logger.network.info("Loaded \(ms) ms for user \(userID, privacy: .private)")
Logger.network.notice("Cache hit")
Logger.network.error("Status \(503, privacy: .public)")
Logger.network.fault("Unexpected nil token")

// Formatting options
Logger.ui.info("Progress \(0.4567, format: .fixed(precision: 2))")
Logger.ui.info("ID \(userID, privacy: .private(mask: .hash))")

// Terminal on a Mac:
// log stream --predicate 'subsystem == "com.example.app"' --level debug
```

CODE EXPLAINED

- → `Logger(subsystem:category:)` — tagged logger
- → `.debug` … `.fault` — levels
- → `privacy: .public / .private` — redaction
- → `format: .fixed(precision:)` — number formatting
- → `log stream --predicate` — live filter

THE SWIFTY WAY

✗ `print("user: \(email)")`
✓ `logger.info("user: \(email, privacy: .private)")` — fast, filterable, private

COMING FROM…

- **Kotlin** Android `Log.d(TAG, …)` ≈ Logger with category
- **Python** `logging.getLogger("net")` ≈ category logger
- **TypeScript** `console.debug/info/error` with no privacy controls

LOG LEVELS

LEVEL	PERSISTED?
`debug`	No (memory only)
`info`	Only when collected
`notice (default)`	Yes
`error`	Yes
`fault`	Yes, with extra context

⚠ COMMON MISTAKES

- Logging tokens or emails as `.public`
- Pre-building a String, then logging `\(message)`: the whole message becomes one redacted value
- Expecting debug messages in a user's sysdiagnose

✓ TIPS & BEST PRACTICES

- One static Logger per category in an extension
- Use `OSSignposter` to measure intervals in Instruments
- Filter Xcode's console by subsystem and level

QUICK SUMMARY

- `Logger` — unified logging
- subsystem + category
- private by default

Networking with URLSession

HTTP WITH URLSESSION

`URLSession` handles HTTP. With async/await, a request is one line: `try await URLSession.shared.data(from: url)` returns the body `Data` and a `URLResponse`. Check the status code yourself (non-2xx responses don't throw), then decode the body with `JSONDecoder`. Use `URLRequest` to set the method, headers and body.

NOTES

- Only transport failures throw; 404 and 500 come back normally
- Cast the response to `HTTPURLResponse` for `statusCode`
- App Transport Security requires HTTPS by default
- Make your own `URLSession(configuration:)` for timeouts/caching

GET & POST

```
import Foundation
struct Todo: Codable { let id: Int; let title: String }
enum APIError: Error { case status(Int) }

func fetchTodos() async throws -> [Todo] {
    let url = URL(string: "https://api.example.com/todos")!
    let (data, response) = try await URLSession.shared.data(from: url)
    guard let http = response as? HTTPURLResponse,
          (200..<300).contains(http.statusCode) else {
        throw APIError.status((response as? HTTPURLResponse)?.statusCode ?? -1)
    }
    return try JSONDecoder().decode([Todo].self, from: data)
}

func create(_ todo: Todo) async throws {
    var req = URLRequest(url: URL(string: "https://api.example.com/todos")!)
    req.httpMethod = "POST"
    req.setValue("application/json", forHTTPHeaderField: "Content-Type")
    req.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
    req.httpBody = try JSONEncoder().encode(todo)
    let (_, resp) = try await URLSession.shared.data(for: req)
    print((resp as! HTTPURLResponse).statusCode)        // 201
}
```

CODE EXPLAINED

- → `data(from:)` — simple GET
- → `HTTPURLResponse.statusCode` — check it yourself
- → `URLRequest` — method, headers, body
- → `data(for:)` — send a custom request
- → `JSONDecoder().decode` — parse the body

THE SWIFTY WAY

✗ `dataTask(with:) { data, resp, err in … }.resume()`
✓ `try await URLSession.shared.data(from:)`

SESSION METHODS

METHOD	USE
`data(from:) / data(for:)`	Request → Data
`upload(for:from:)`	Send a body
`download(from:)`	Large file to disk
`bytes(from:)`	Stream (AsyncSequence)

COMING FROM…

- **TypeScript** `await fetch(url)` ≈ `data(from:)`; also no throw on 404
- **Python** `requests.get` ≈ `data(from:)`; use `raise_for_status` ≈ status check
- **Kotlin** Ktor / Retrofit suspend calls ≈ async URLSession

⚠ COMMON MISTAKES

- Assuming a 404 throws: check `statusCode`
- Building query strings by hand: use `URLComponents`
- Decoding on the main actor in a tight loop: fine for small JSON, move large work off

✓ TIPS & BEST PRACTICES

- Wrap requests in one `APIClient` type with a generic `send<T: Decodable>`
- Set `config.timeoutIntervalForRequest` on a custom session
- Use `URLProtocol` subclasses to stub the network in tests

QUICK SUMMARY

- `try await URLSession.shared.data(…)`
- check `HTTPURLResponse.statusCode`
- `URLRequest` for method/headers/body

Swift Collections & Algorithms

EXTRA COLLECTIONS & ALGORITHMS

Two official Apple packages fill gaps in the standard library. **swift-collections** adds data structures: `Deque`, `OrderedSet`, `OrderedDictionary`, `Heap` and more. **swift-algorithms** adds sequence operations: chunking, windows, combinations, permutations, `uniqued()` and more. Both are maintained by the Swift team.

NOTES

- Add `https://github.com/apple/swift-collections` and `…/swift-algorithms`
- Import the umbrella `Collections` or a single module (`DequeModule`)
- `Heap` is a min-max heap: `popMin()` and `popMax()`
- Algorithms methods are lazy where possible

COLLECTIONS & ALGORITHMS

```
import Collections
import Algorithms

var q: Deque = [1, 2, 3]
q.prepend(0); q.popFirst()                  // 0, O(1) at both ends

var tags: OrderedSet = ["b", "a", "b"]      // ["b", "a"] keeps order
var scores: OrderedDictionary = ["zed": 1, "amy": 2]
scores.keys.first                           // "zed" (insertion order)

var heap = Heap([5, 1, 9, 3])
heap.popMin()                               // 1
heap.popMax()                               // 9

let nums = Array(1...7)
nums.chunks(ofCount: 3).map(Array.init)     // [[1,2,3],[4,5,6],[7]]
nums.windows(ofCount: 3).first.map(Array.init)   // [1, 2, 3]
[1, 2, 1, 3].uniqued().map { $0 }           // [1, 2, 3]
["a", "b", "c"].combinations(ofCount: 2).count   // 3
[1, 2, 3].permutations().count              // 6
nums.adjacentPairs().first                  // (1, 2)
```

CODE EXPLAINED

- → `Deque` — fast at both ends
- → `OrderedSet` / `OrderedDictionary` — keep order
- → `Heap` — min and max priority queue
- → `chunks` / `windows` — split sequences
- → `uniqued` / `combinations` / `permutations`

THE SWIFTY WAY

✗ `Array(Set(xs))` to dedupe
✓ `xs.uniqued()` — keeps the original order

COMING FROM…

- **Python** `collections.deque`, `itertools` ≈ these packages
- **Kotlin** `ArrayDeque`, `chunked`, `windowed` built in
- **Java** `ArrayDeque`, `LinkedHashMap`, `PriorityQueue`

DATA STRUCTURES

TYPE	LIKE
`Deque`	Double-ended queue
`OrderedSet`	LinkedHashSet
`OrderedDictionary`	LinkedHashMap
`Heap`	Priority queue (min + max)
`BitSet`	Compact set of Ints
`TreeDictionary`	Persistent hash map

⚠ COMMON MISTAKES

- Pulling in a third-party queue when `Deque` exists
- Forgetting `Array(...)` around chunks/windows when you need arrays
- Expecting `OrderedDictionary` subscripts by Int index (use `.elements[i]`)

✓ TIPS & BEST PRACTICES

- Import only the module you need (`HeapModule`) to keep builds lean
- `partitioningIndex(where:)` gives binary search on sorted data
- `striding(by:)` and `product(_:_:)` replace nested index loops

QUICK SUMMARY

- swift-collections — Deque, Ordered*, Heap
- swift-algorithms — chunks, windows, uniqued
- official Apple packages

SwiftLint & swift-format

LINTING & FORMATTING

swift-format (Apple's formatter and linter) ships with the Swift 6 toolchain as `swift format`. **SwiftLint** (community, by Realm) is the most common linter in iOS teams, with hundreds of configurable rules. Run them in Xcode build phases, pre-commit hooks or CI so style is checked by tools, not reviewers.

NOTES

- swift-format config: `.swift-format` (JSON)
- SwiftLint config: `.swiftlint.yml` (YAML)
- SwiftLint has a build tool plugin for SwiftPM and Xcode
- `swiftlint --fix` auto-corrects some rules

COMMANDS & CONFIG

```
# swift-format (in the Swift 6 toolchain)
swift format --in-place --recursive Sources Tests
swift format lint --recursive Sources
swift format dump-configuration > .swift-format

# SwiftLint
brew install swiftlint
swiftlint                       # lint current folder
swiftlint --fix                 # auto-correct
swiftlint --strict              # warnings fail the run (CI)

# .swiftlint.yml
disabled_rules: [trailing_comma]
opt_in_rules: [empty_count, force_unwrapping]
excluded: [Pods, .build]
line_length: { warning: 120, error: 160 }

// In code
// swiftlint:disable:next force_cast
let v = obj as! UIView
```

CODE EXPLAINED

- → `swift format --in-place` — rewrite files
- → `swift format lint` — report only
- → `swiftlint --fix` / `--strict`
- → `opt_in_rules` — enable extra rules
- → `// swiftlint:disable:next rule` — local exception

THE SWIFTY WAY

✗ style comments in code review
✓ a formatter in a pre-commit hook — machines argue about commas

COMING FROM…

- **TypeScript** Prettier ≈ swift-format; ESLint ≈ SwiftLint
- **Python** black ≈ swift-format; ruff/flake8 ≈ SwiftLint
- **Kotlin** ktlint / detekt ≈ SwiftLint

USEFUL OPT-IN RULES

RULE	CATCHES
force_unwrapping	`x!`
empty_count	`count == 0`
first_where	`filter{}.first`
implicit_return	Redundant `return`
sorted_imports	Unsorted imports

⚠ COMMON MISTAKES

- Running formatter and linter with conflicting rules (e.g. trailing commas)
- Disabling rules file-wide instead of per line
- Linting generated code: add it to `excluded`

✓ TIPS & BEST PRACTICES

- Format on save in VS Code with the Swift extension
- Add `swiftlint --strict` to CI so warnings can't pile up
- Commit the config files so everyone shares the same rules

QUICK SUMMARY

- `swift format` — official formatter
- SwiftLint — community linter
- run both in CI

Regex

REGULAR EXPRESSIONS IN SWIFT

Swift has native, type-checked regex. A **regex literal** **/…/** is compiled and checked at build time, and its capture groups become **typed tuple elements** on the match. **RegexBuilder** spells the same patterns as a readable DSL. String methods like `firstMatch(of:)`, `wholeMatch(of:)`, `matches(of:)` and `replacing(_:with:)` accept either form. Pattern syntax is covered in the **RegEx** set.

NOTES

- Literals need Swift 5.7+ and iOS 16 / macOS 13
- Captures: `match.1`, `match.2` …; `match.0` is the whole match
- `#/ … /#` allows spaces and `/` inside (extended literal)
- `try Regex(string)` builds one at runtime (untyped captures)

REGEX LITERALS & BUILDER

```
let log = "2026-10-26 ERROR disk full"

if let m = log.firstMatch(of: /(\d{4})-(\d{2})-(\d{2}) (\w+)/) {
    m.1                                  // "2026" (Substring)
    m.4                                  // "ERROR"
}
"abc123".contains(/\d+/)                 // true
"2026-10-26".wholeMatch(of: /\d{4}-\d{2}-\d{2}/) != nil   // true
"a1b22c333".matches(of: /\d+/).map(\.output)              // ["1","22","333"]
"tel: 555-1234".replacing(/\d/, with: "#")                // "tel: ###-####"

let named = /(?<user>\w+)@(?<host>[\w.]+)/
if let m = "ada@swift.org".firstMatch(of: named) { m.host }  // "swift.org"

import RegexBuilder
let price = Regex {
    "$"
    TryCapture { OneOrMore(.digit) } transform: { Int($0) }
}
"Total: $42".firstMatch(of: price)?.1                     // 42 (Int)
```

CODE EXPLAINED

- → `/…/` — compile-time checked literal
- → `m.1` — typed captures
- → `firstMatch` / `wholeMatch` / `matches(of:)`
- → `(?<name>…)` — named capture → `m.name`
- → `TryCapture … transform:` — capture as Int

THE SWIFTY WAY

✗ `NSRegularExpression(pattern: "…")` + NSRange math
✓ a `/…/` literal with typed captures

METHODS

METHOD	RETURNS
`firstMatch(of:)`	First match?
`wholeMatch(of:)`	Match if whole string
`prefixMatch(of:)`	Match at start
`matches(of:)`	[Match]
`replacing(_:with:)`	New String
`split(separator: /…/)`	[Substring]

COMING FROM…

- **TypeScript** `/…/` literals ≈ same, but typed captures here
- **Python** `re.search` ≈ `firstMatch(of:)`
- **Kotlin** `Regex("…").find` ≈ firstMatch

SWIFT VERSIONS

- 5.7 — Regex type, literals, RegexBuilder (iOS 16 / macOS 13)

⚠ COMMON MISTAKES

- Using `firstMatch` when the whole input must match: use `wholeMatch`
- Forgetting captures are `Substring`: wrap with `String(...)` to store
- Building a runtime `Regex(string)` from user input without `try`

✓ TIPS & BEST PRACTICES

- Add `.ignoresCase()` to a regex instead of `(?i)`
- Use RegexBuilder for long patterns that need comments
- See the RegEx set for pattern syntax and common patterns

QUICK SUMMARY

- `/…/` — typed regex literal
- `firstMatch` / `wholeMatch` / `replacing`
- RegexBuilder — readable DSL

Key Paths

KEY PATHS

A **key path** like `\User.name` is a reference to a property itself, not its value. Read or write through it with `value[keyPath: kp]`. Key paths can be passed around, stored and composed, and they work anywhere a function from root to value is expected, so `users.map(\.name)` is valid.

NOTES

- `\.name` uses inference when the root type is known
- `KeyPath` is read-only; `WritableKeyPath` allows writes (vars)
- `ReferenceWritableKeyPath` writes through class references
- Key paths are Hashable: usable as dictionary keys

USING KEY PATHS

```
struct User { var name: String; var age: Int; var address: Address }
struct Address { var city: String }
var u = User(name: "Ada", age: 36, address: Address(city: "London"))

let kp = \User.name                          // WritableKeyPath<User, String>
u[keyPath: kp]                               // "Ada"
u[keyPath: \.address.city] = "Paris"         // nested, writable

let users = [u, User(name: "Bo", age: 20, address: .init(city: "Rome"))]
users.map(\.name)                            // ["Ada", "Bo"]
users.filter(\.address.city.isEmpty)         // [] (Bool key path)
users.sorted(using: KeyPathComparator(\.age))   // Bo, Ada
users.max { $0.age < $1.age }?.name          // "Ada"

func sortBy<T, V: Comparable>(_ xs: [T], _ kp: KeyPath<T, V>) -> [T] {
    xs.sorted { $0[keyPath: kp] < $1[keyPath: kp] }
}
sortBy(users, \.name)

let city = (\User.address).appending(path: \.city)   // composed
```

CODE EXPLAINED

- → `\User.name` — key path literal
- → `[keyPath:]` — read / write through it
- → `map(\.name)` — key path as a function
- → `KeyPathComparator` — sort descriptors
- → `KeyPath<T, V>` — generic over properties

THE SWIFTY WAY

✗ `users.map { $0.name }`
✓ `users.map(\.name)`

COMING FROM…

- **Kotlin** `User::name` property references ≈ key paths
- **TypeScript** `keyof T` + `obj[key]` (weaker typing)
- **Python** `operator.attrgetter('name')` ≈ `\.name`

IN SWIFTUI

- `ForEach(items, id: \.id)` takes a key path
- `$model.name` bindings use key paths under the hood
- See SwiftUI 17 · @Bindable & Sharing Models

KEY PATH TYPES

TYPE	CAN
`KeyPath<R, V>`	Read
`WritableKeyPath<R, V>`	Read / write (value types)
`ReferenceWritableKeyPath`	Write through a class ref
`PartialKeyPath<R>`	Value type erased
`AnyKeyPath`	Fully erased

⚠ COMMON MISTAKES

- Trying to write through a `KeyPath` to a `let` property
- Expecting `\.count` to infer without context: specify `\String.count`
- Using key paths to private properties from outside the type

✓ TIPS & BEST PRACTICES

- Bool key paths work as predicates: `filter(\.isDone)`
- Store sort options as `[KeyPathComparator<T>]` for UI tables
- Use key paths in generic helpers instead of closures for simple access

QUICK SUMMARY

- `\Type.prop` — property reference
- `x[keyPath:]` — read/write
- `map(\.prop)` — key path as function

Property Wrappers

PROPERTY WRAPPERS

A property wrapper packages the get/set logic for a property into a reusable type. Mark a struct `@propertyWrapper`, give it a `wrappedValue`, then apply it as an attribute: `@Clamped(0...100) var volume = 50`. A `projectedValue` is exposed with a `$` prefix. SwiftUI's `@State`, `@Binding` and `@AppStorage` are all property wrappers.

NOTES

- The compiler creates hidden storage `_name` of the wrapper type
- `$name` gives the `projectedValue` (if defined)
- Works on properties and local variables, and function parameters
- Can't be combined with `lazy`, `weak` or observers in some cases

WRITING A WRAPPER

```
@propertyWrapper
struct Clamped<V: Comparable> {
    private var value: V
    let range: ClosedRange<V>
    init(wrappedValue: V, _ range: ClosedRange<V>) {
        self.range = range
        self.value = min(max(wrappedValue, range.lowerBound), range.upperBound)
    }
    var wrappedValue: V {
        get { value }
        set { value = min(max(newValue, range.lowerBound), range.upperBound) }
    }
    var projectedValue: ClosedRange<V> { range }     // $volume
}

struct Player {
    @Clamped(0...100) var volume = 50
}
var p = Player()
p.volume = 140
p.volume                                  // 100
p.$volume                                 // 0...100 (projected)

@propertyWrapper struct Trimmed {
    var wrappedValue: String { didSet { wrappedValue = wrappedValue.trimmed } }
}
```

CODE EXPLAINED

- → `@propertyWrapper` — declare the wrapper
- → `wrappedValue` — what the property reads/writes
- → `init(wrappedValue:_:)` — receives the default + args
- → `projectedValue` — exposed as `$name`
- → `@Clamped(0...100)` — apply with arguments

THE SWIFTY WAY

✗ the same `didSet` clamping in five types
✓ one `@Clamped` wrapper

COMING FROM…

- **Kotlin** Delegated properties `by` ≈ property wrappers
- **Python** Descriptors ≈ property wrappers
- **TypeScript** Decorators on accessors are similar

SWIFT VERSIONS

- 5.1 — property wrappers (SE-0258)
- 5.5 — wrappers on function parameters

BUILT-IN WRAPPERS

WRAPPER	FROM
`@State / @Binding`	SwiftUI
`@Environment`	SwiftUI
`@AppStorage`	SwiftUI
`@Query`	SwiftData
`@Option / @Flag`	ArgumentParser

IN SWIFTUI

- `@State var x` — storage owned by the view; `$x` is a `Binding`
- `@AppStorage("key")` wraps UserDefaults
- See SwiftUI 14 · @State

⚠ COMMON MISTAKES

- Expecting a wrapper to work on computed properties: only stored ones
- Forgetting `init(wrappedValue:)` so `= 50` default doesn't compile
- Hiding expensive work in `wrappedValue` getters

✓ TIPS & BEST PRACTICES

- Access the wrapper itself inside the type with `_volume`
- Use wrappers for cross-cutting rules: clamping, trimming, user defaults
- Macros (Sheet 096) have replaced some wrapper uses, e.g. `@Observable`

QUICK SUMMARY

- `@propertyWrapper` + `wrappedValue`
- `$name` — projectedValue
- SwiftUI state is built on them

Macros

SWIFT MACROS

Macros generate code at compile time. **Freestanding** macros start with # and produce an expression or declarations (`#Preview`, `#expect`). **Attached** macros start with @ and add members, accessors, conformances or peers to a declaration (`@Observable`, `@Model`, `@Test`). Macros are implemented with **swift-syntax** in a separate compiler-plugin target.

NOTES

- Right-click a macro › **Expand Macro** in Xcode to see the code
- Macros are sandboxed and can't read files or the network
- Third-party macro packages ask for trust on first build
- Expansion must type-check, so errors point at real code

USING & DEFINING MACROS

```
import Observation
@Observable final class Store {            // attached macro
    var items: [String] = []               // tracked automatically
}

#Preview { ContentView() }                 // freestanding: declarations
#expect(total == 42)                       // freestanding: expression
let id = #URL("https://swift.org")         // custom macro (e.g. a package)

// Declaring a macro (in the library target)
@freestanding(expression)
public macro stringify<T>(_ value: T) -> (T, String) =
    #externalMacro(module: "MyMacrosPlugin", type: "StringifyMacro")

let (v, code) = #stringify(2 + 3)          // (5, "2 + 3")

// Package.swift
// .macro(name: "MyMacrosPlugin", dependencies: [
//     .product(name: "SwiftSyntaxMacros", package: "swift-syntax"),
//     .product(name: "SwiftCompilerPlugin", package: "swift-syntax")])
```

CODE EXPLAINED

- → `@Observable` — attached macro adds tracking
- → `#Preview` / `#expect` — freestanding macros
- → `@freestanding(expression)` — declare a macro
- → `#externalMacro` — points at the implementation
- → `.macro` target — swift-syntax plugin

THE SWIFTY WAY

✗ hand-written boilerplate in every model
✓ an attached macro (if many types need it) — but plain functions first

COMING FROM…

- **Kotlin** KSP / compiler plugins ≈ macros
- **Rust** Procedural macros ≈ Swift macros
- **TypeScript** Decorators (runtime) vs macros (compile time)

MACRO ROLES

ROLE	ADDS
`expression`	A value (#stringify)
`declaration`	Declarations (#Preview)
`member`	Members in a type
`accessor`	get/set on a property
`extension`	Protocol conformances
`peer`	Sibling declarations

SWIFT VERSIONS

- 5.9 — macros (SE-0382, SE-0389, SE-0397)

IN SWIFTUI

- `#Preview` replaces `PreviewProvider`
- `@Observable` replaces `ObservableObject` + `@Published`
- See SwiftUI 04 · Previews

⚠ COMMON MISTAKES

- Writing a macro where a generic function would do
- Debugging generated code without expanding it
- Adding swift-syntax to the app target: it belongs in the macro plugin

✓ TIPS & BEST PRACTICES

- Use the Xcode "Swift Macro" package template to start
- Test with `assertMacroExpansion` from SwiftSyntaxMacrosTestSupport
- Keep macro output small and readable

QUICK SUMMARY

- `#name` — freestanding; `@Name` — attached
- compile-time code generation
- built on swift-syntax

Objective-C & C Interop

CALLING OTHER LANGUAGES

Swift interoperates directly with **Objective-C** (most Apple frameworks), **C** (system libraries), and, since 5.9, **C++**. Mark Swift members `@objc` to expose them to Objective-C, use a **bridging header** to see Objective-C in an app target, and call C functions directly through module maps. Raw memory uses `Unsafe…Pointer` types.

NOTES

- Swift classes need `NSObject` or `@objc` to be seen by Obj-C
- Selectors: `#selector(method)` needs an `@objc` method
- `Unsafe` APIs skip safety checks: keep them small and local
- C++ interop: Build setting "C++ and Objective-C Interoperability"

INTEROP BASICS

```
import UIKit

final class Handler: NSObject {
    @objc func tapped(_ sender: UIButton) { print("tap") }
}
button.addTarget(handler, action: #selector(Handler.tapped(_:)),
                 for: .touchUpInside)

// MyApp-Bridging-Header.h   →  #import "LegacyParser.h"
let parser = LegacyParser()              // Obj-C class, used directly

import Darwin                            // C standard library
let len = strlen("hello")                // 5
var buf = [UInt8](repeating: 0, count: 8)
buf.withUnsafeMutableBufferPointer { p in
    p[0] = 0xFF                          // raw buffer access
}

var n: Int32 = 0
withUnsafeMutablePointer(to: &n) { $0.pointee = 7 }   // n == 7

// C++ (5.9+): -cxx-interoperability-mode=default
// import CxxStdlib; let s = std.string("hi")
```

CODE EXPLAINED

- → `NSObject` + `@objc` — visible to Obj-C
- → `#selector` — reference an `@objc` method
- → bridging header — Obj-C into an app target
- → C functions — called like Swift functions
- → `withUnsafe…Pointer` — scoped raw memory

THE SWIFTY WAY

✗ returning `UnsafeMutablePointer` from a function
✓ keep pointers inside `withUnsafe…` closures — pointers must not escape

INTEROP PIECES

PIECE	PURPOSE
`@objc / @objcMembers`	Expose to Obj-C
`dynamic`	Obj-C message dispatch (KVO)
`Bridging header`	Obj-C → Swift (app)
`module.modulemap`	C/C++ library as a module
`NS_SWIFT_NAME`	Rename Obj-C API for Swift

COMING FROM…

- **`Kotlin`** JNI / Kotlin/Native cinterop ≈ C interop
- **`Java`** JNI ≈ C interop (much more ceremony)
- **`Python`** ctypes ≈ C interop

⚠ COMMON MISTAKES

- Forgetting `@objc` on a selector target method: compile error
- Letting an `UnsafePointer` escape its `withUnsafe…` closure
- Assuming Obj-C APIs never return nil: check implicitly unwrapped results

✓ TIPS & BEST PRACTICES

- Add nullability annotations to Obj-C headers for clean Swift types
- Wrap C APIs in a small Swift type that handles errors and memory
- Prefer `Span` / buffer pointers over raw pointers when available

QUICK SUMMARY

- `@objc` — Swift → Obj-C
- bridging header — Obj-C → Swift
- C / C++ via modules; `Unsafe` pointers

async / await

ASYNC FUNCTIONS

An `async` function can **suspend** while waiting (for the network, disk, or another task) without blocking a thread. You call it with `await`, which marks each possible suspension point. `async let` starts child work in parallel, and `Task { }` starts async work from synchronous code such as a button action.

NOTES

- `await` can only appear in an async context
- Order: `try await` (try first)
- A suspended function may resume on a different thread
- Properties can be `async` getters: `var data: Data { get async throws }`

ASYNC / AWAIT

```swift
func fetchUser(id: Int) async throws -> User {
    let url = URL(string: "https://api.example.com/users/\(id)")!
    let (data, _) = try await URLSession.shared.data(from: url)
    return try JSONDecoder().decode(User.self, from: data)
}

func loadProfile() async throws -> Profile {
    async let user = fetchUser(id: 1)            // starts now
    async let posts = fetchPosts(userID: 1)      // runs in parallel
    return try await Profile(user: user, posts: posts)   // wait for both
}

// From synchronous code (e.g. a button action)
func refreshTapped() {
    Task {
        do { let p = try await loadProfile(); show(p) }
        catch { showError(error) }
    }
}

@main struct Tool {
    static func main() async throws { print(try await loadProfile()) }
}
```

CODE EXPLAINED

- → `async` — function may suspend
- → `try await` — call a throwing async function
- → `async let` — child task, runs in parallel
- → `Task { }` — bridge from sync code
- → `static func main() async` — async entry point

THE SWIFTY WAY

✗ `func load(completion: @escaping (Result<User, Error>) -> Void)`
✓ `func load() async throws -> User`

COMING FROM…

- **TypeScript** `async/await` ≈ same; `async let` ≈ Promise.all
- **Kotlin** `suspend fun` ≈ async; `async {}` ≈ async let
- **Python** asyncio `await` ≈ same; `gather` ≈ async let

SWIFT VERSIONS

- 5.5 — async/await, async let, Task (back-deployed to iOS 13)

IN SWIFTUI

- `.task { await model.load() }` runs when a view appears and cancels on disappear
- Button actions start work with `Task { await … }`
- See SwiftUI 37 · .task, onAppear & onChange

WAYS TO RUN ASYNC WORK

FORM	USE
`await f()`	Sequential
`async let`	Fixed number in parallel
`TaskGroup`	Dynamic number in parallel
`Task { }`	From sync code

⚠ COMMON MISTAKES

- Awaiting calls one by one when they could run in parallel with `async let`
- Calling blocking APIs (`sleep` , `semaphore.wait`) in async code
- Creating `Task { }` in loops without keeping or cancelling them

✓ TIPS & BEST PRACTICES

- Wrap old callback APIs with continuations (Sheet 103)
- Use `Task.sleep(for: .seconds(1))` for delays
- Mark UI-touching functions `@MainActor` (Sheet 101)

QUICK SUMMARY

- `async` / `await` — suspend, don't block
- `async let` — parallel children
- `Task { }` — start from sync code

Tasks & Task Groups

STRUCTURED CONCURRENCY

A `TaskGroup` runs a **dynamic** number of child tasks in parallel and collects their results as they finish. Child tasks are **structured**: the group waits for all of them, cancels the rest if one throws, and can't outlive its scope. Unstructured `Task { }` and `Task.detached { }` create independent tasks you manage yourself.

NOTES

- Results arrive in completion order, not submission order
- Throwing group: first error cancels the remaining children
- `Task` inherits priority and actor context; `detached` doesn't
- Keep a `Task` handle to `await` its value or `cancel()` it

TASK GROUPS & TASKS

```swift
func downloadAll(_ urls: [URL]) async throws -> [URL: Data] {
    try await withThrowingTaskGroup(of: (URL, Data).self) { group in
        for url in urls {
            group.addTask {
                let (data, _) = try await URLSession.shared.data(from: url)
                return (url, data)
            }
        }
        var results: [URL: Data] = [:]
        for try await (url, data) in group { results[url] = data }
        return results
    }
}

let handle = Task(priority: .userInitiated) { try await downloadAll(urls) }
let files = try await handle.value                // wait for result
handle.cancel()                                   // or cancel it

Task.detached(priority: .background) { await cleanUpCache() }
try await Task.sleep(for: .milliseconds(250))
await Task.yield()                                // let others run
```

CODE EXPLAINED

- → `withThrowingTaskGroup(of:)` — parallel children
- → `group.addTask { }` — add a child
- → `for try await … in group` — collect results
- → `Task(priority:)` / `.value` — unstructured task
- → `Task.detached` — no inherited context

THE SWIFTY WAY

✗ `Task.detached` by default
✓ `Task { }` or a task group — detached loses actor and priority context

COMING FROM…

- **Kotlin** `coroutineScope { launch {} }` ≈ TaskGroup
- **TypeScript** `Promise.all(urls.map(f))` ≈ group (no cancel)
- **Python** `asyncio.TaskGroup` ≈ withThrowingTaskGroup

TASK APIS

API	PURPOSE
`withTaskGroup`	Non-throwing group
`withThrowingTaskGroup`	Throwing group
`withDiscardingTaskGroup`	Fire-and-forget children
`Task.sleep(for:)`	Delay (cancellable)
`Task.yield()`	Give up the thread briefly

⚠ COMMON MISTAKES

- Mutating a shared array from inside `addTask`: data race; return values instead
- Expecting results in input order
- Launching thousands of children at once: limit concurrency for network calls

✓ TIPS & BEST PRACTICES

- Limit parallelism by adding the next task as each one finishes
- Return `(index, value)` pairs when you need the original order
- Use `withDiscardingTaskGroup` for servers handling many connections

QUICK SUMMARY

- TaskGroup — dynamic parallel children
- structured: waits, cancels on error
- `Task { }` — unstructured, keep the handle

Task Cancellation

CANCELLING TASKS

Cancellation in Swift is **cooperative**: calling `cancel()` only sets a flag. Your code checks it with `Task.isCancelled` or `try Task.checkCancellation()`, which throws `CancellationError`. Many system APIs (`Task.sleep`, `URLSession`) check automatically. Cancelling a parent cancels all its child tasks.

NOTES

- Nothing stops a task that never checks for cancellation
- `Task.sleep` throws `CancellationError` when cancelled
- SwiftUI's `.task` modifier cancels when the view disappears
- `withTaskCancellationHandler` reacts immediately to cancel

CHECKING FOR CANCELLATION

```
func process(_ items: [Item]) async throws -> [Result] {
    var out: [Result] = []
    for item in items {
        try Task.checkCancellation()          // throws if cancelled
        out.append(await transform(item))
    }
    return out
}

func pollForever() async {
    while !Task.isCancelled {                 // non-throwing check
        await refresh()
        try? await Task.sleep(for: .seconds(30))
    }
}

let job = Task { try await process(items) }
job.cancel()
do { _ = try await job.value }
catch is CancellationError { print("stopped") }

func download(_ url: URL) async throws -> Data {
    let op = LegacyDownloader(url)
    return try await withTaskCancellationHandler {
        try await op.start()
    } onCancel: { op.abort() }                // runs right away
}
```

CODE EXPLAINED

- → `try Task.checkCancellation()` — throw if cancelled
- → `Task.isCancelled` — Bool check
- → `job.cancel()` — request cancellation
- → `catch is CancellationError` — handle it
- → `withTaskCancellationHandler` — immediate hook

THE SWIFTY WAY

✗ a `var shouldStop` flag in your class
✓ `Task.isCancelled` / `checkCancellation()`

COMING FROM…

- **Kotlin** Coroutine cancellation is also cooperative (`isActive`)
- **TypeScript** `AbortController` ≈ cancel(); must be passed manually there
- **Python** `task.cancel()` raises CancelledError at the next await

AUTO-CANCELLING APIS

API	BEHAVIOR
`Task.sleep`	Throws CancellationError
`URLSession.data`	Throws URLError.cancelled
`TaskGroup`	Cancels children on throw
`SwiftUI .task`	Cancels on disappear

⚠ COMMON MISTAKES

- Long CPU loops with no cancellation checks
- Showing an error alert for `CancellationError`: it's expected, not a failure
- Using `try?` on `Task.sleep` inside a loop and never re-checking `isCancelled`

✓ TIPS & BEST PRACTICES

- Cancel the previous search Task before starting a new one
- Store tasks in a property and cancel them in `deinit` or `onDisappear`
- Use `.task(id:)` in SwiftUI to restart work when an input changes

QUICK SUMMARY

- cancellation is cooperative
- `checkCancellation()` / `isCancelled`
- parent cancel → children cancel

Actors & @MainActor

ACTORS

An `actor` is a reference type that protects its mutable state: only one task at a time can run code **inside** it, so data races are impossible. Code outside must `await` to call into the actor. `@MainActor` is a global actor for the main thread; mark UI types and functions with it so they always run there.

NOTES

- Actor methods are synchronous inside, `await` from outside
- `nonisolated` members skip isolation (only for immutable data)
- Actors are **re-entrant**: state can change across an `await`
- `MainActor.run { }` hops to the main actor briefly

ACTORS & @MAINACTOR

```
actor BankAccount {
    private(set) var balance = 0
    let id = UUID()

    func deposit(_ amount: Int) { balance += amount }   // isolated

    func withdraw(_ amount: Int) throws {
        guard balance >= amount else { throw BankError.insufficient }
        balance -= amount
    }
    nonisolated var label: String { "Account \(id)" }   // no await needed
}

let acct = BankAccount()
await acct.deposit(100)                  // await from outside
try await acct.withdraw(30)
print(await acct.balance)                // 70
print(acct.label)                        // nonisolated

@MainActor
@Observable final class FeedModel {      // all members on main
    var posts: [Post] = []
    func load() async {
        posts = (try? await api.fetchPosts()) ?? []   // back on main
    }
}
```

CODE EXPLAINED

- → `actor` — isolated mutable state
- → `await acct.deposit` — cross-actor call
- → `nonisolated` — opt out for immutable data
- → `@MainActor` — run on the main thread
- → after `await`, back on the model's actor

THE SWIFTY WAY

✗ `DispatchQueue.main.async { self.posts = … }`
✓ `@MainActor` on the type — the compiler enforces it

COMING FROM…

- **Kotlin** `Mutex` / single-thread dispatcher ≈ actor
- **Java** `synchronized` class ≈ actor, without await
- **TypeScript** Single-threaded JS ≈ everything on MainActor

SWIFT VERSIONS

- 5.5 — actors and `@MainActor`
- 6.2 — opt-in default `MainActor` isolation for app targets

ISOLATION KEYWORDS

KEYWORD	MEANING
`actor`	Isolated reference type
`@MainActor`	Main-thread isolation
`@globalActor`	Define your own global actor
`nonisolated`	No isolation for this member
`isolated param`	Run in the param's actor

IN SWIFTUI

- `View` is `@MainActor`; mark view models `@MainActor` too
- Heavy work belongs in another actor or a `@concurrent` function
- See SwiftUI 16 · @Observable Models

⚠ COMMON MISTAKES

- Assuming state is unchanged after an `await` inside an actor (re-entrancy)
- Calling `DispatchQueue.main.sync` from the main actor: deadlock
- Making everything an actor: adds awaits; use structs for data

✓ TIPS & BEST PRACTICES

- Re-check conditions after each `await` inside actor methods
- Keep actor methods small and synchronous where possible
- Use `Mutex` (Synchronization) for tiny, non-async critical sections

QUICK SUMMARY

- `actor` — one task inside at a time
- `await` to call from outside
- `@MainActor` — UI code

Sendable & Strict Concurrency

DATA-RACE SAFETY

`Sendable` marks types that are safe to share between concurrent tasks. Value types with Sendable members, actors and immutable final classes qualify. In the **Swift 6 language mode**, the compiler checks every crossing between isolation domains and turns potential data races into **compile errors**.

NOTES

- Structs/enums are implicitly Sendable when all members are (non-public)
- Closures crossing tasks must be `@Sendable`
- `sending` parameters transfer ownership to another domain
- `@unchecked Sendable` = you promise thread safety manually

SENDABLE IN PRACTICE

```
struct Point: Sendable { var x, y: Double }          // value type: easy

final class Config: Sendable {                       // immutable class
    let apiURL: URL
    init(apiURL: URL) { self.apiURL = apiURL }
}

final class Counter: @unchecked Sendable {           // manual safety
    private let lock = NSLock()
    private var value = 0
    func increment() { lock.withLock { value += 1 } }
}

class Cache { var items: [String] = [] }             // not Sendable
let cache = Cache()
Task { cache.items.append("x") }                     // Swift 6: error
// "sending 'cache' risks causing data races"

func process(_ job: sending Job) async { … }         // ownership moves

// Package.swift
// swiftLanguageModes: [.v6]
// Xcode: Build Settings › Swift Language Version = 6
```

CODE EXPLAINED

- → `Sendable` struct — copied safely
- → final class with only `let`s — Sendable
- → `@unchecked Sendable` — you guarantee it
- → non-Sendable capture — compile error in Swift 6
- → `sending` — transfer ownership

THE SWIFTY WAY

✗ `@unchecked Sendable` to silence errors
✓ make it a struct, an actor, or immutable — unchecked hides real races

COMING FROM…

- **Rust** `Send` / `Sync` ≈ Sendable checks
- **Kotlin** No compile-time race checks
- **Java** No equivalent; races found at runtime

IS IT SENDABLE?

TYPE	SENDABLE?
`Int, String, Array<Int>`	✔
`struct of Sendables`	✔ (implicit)
`actor`	✔
`final class, only let Sendables`	✔ (declare it)
`class with var`	✖

SWIFT VERSIONS

- 5.5 — Sendable introduced
- 6.0 — Swift 6 mode: full data-race checking, `sending`
- 6.2 — `nonisolated(nonsending)`, `@concurrent`, approachable defaults

⚠ COMMON MISTAKES

- Turning on Swift 6 mode for a big app in one step: migrate module by module
- Spreading `@unchecked Sendable` and `nonisolated(unsafe)` everywhere
- Capturing a non-Sendable class in `Task { }`

✓ TIPS & BEST PRACTICES

- Enable `-strict-concurrency=complete` warnings first, then switch modes
- Prefer value types for data passed between tasks
- Use `Mutex` from the Synchronization module for small shared state

QUICK SUMMARY

- `Sendable` — safe to share
- Swift 6 mode — races are errors
- `@unchecked` — last resort

AsyncSequence & Continuations

ASYNC STREAMS & BRIDGES

An `AsyncSequence` delivers values over time; you loop over it with `for await`. `AsyncStream` builds one from callbacks or events. To wrap a single callback-based API in an `async` function, use a **continuation**: `withCheckedThrowingContinuation` suspends until you call `resume` exactly once.

NOTES

- `for try await` for throwing sequences
- A continuation must be resumed exactly once (checked at runtime)
- `AsyncStream.makeStream()` returns stream + continuation (5.9)
- Many Apple APIs expose AsyncSequences: `url.lines`, `NotificationCenter.notifications`

STREAMS & CONTINUATIONS

```swift
// Wrap a completion-handler API
func currentLocation() async throws -> CLLocation {
    try await withCheckedThrowingContinuation { cont in
        legacyLocator.request { location, error in
            if let location { cont.resume(returning: location) }
            else { cont.resume(throwing: error ?? LocError.unknown) }
        }
    }
}

// A stream of values over time
func ticks(every s: Double) -> AsyncStream<Date> {
    AsyncStream { continuation in
        let timer = Timer.scheduledTimer(withTimeInterval: s, repeats: true) { _ in
            continuation.yield(.now)
        }
        continuation.onTermination = { _ in timer.invalidate() }
    }
}
for await t in ticks(every: 1).prefix(3) { print(t) }    // 3 ticks

let (stream, cont) = AsyncStream.makeStream(of: Int.self)
cont.yield(1); cont.finish()
for await n in stream { print(n) }                       // 1
```

CODE EXPLAINED

- → `withCheckedThrowingContinuation` — callback → async
- → `resume(returning:)` / `resume(throwing:)` — once
- → `AsyncStream { continuation in }` — events → stream
- → `onTermination` — clean up when the loop ends
- → `for await` — consume values

THE SWIFTY WAY

✗ a new callback-based API
✓ `async` functions and `AsyncSequence`s

COMING FROM…

- **Kotlin** `Flow` ≈ AsyncSequence; `suspendCoroutine` ≈ continuation
- **TypeScript** `for await…of` + async generators ≈ same
- **Python** async generators + `asyncio.Future` ≈ same

BUILT-IN ASYNCSEQUENCES

SOURCE	YIELDS
`url.lines`	Lines of a file / URL
`URLSession.bytes(from:)`	Bytes
`NotificationCenter.notifications(named:)`	Notifications
`TaskGroup`	Child results

SWIFT VERSIONS

- 5.5 — AsyncSequence, AsyncStream, continuations
- 5.9 — `AsyncStream.makeStream(of:)`

⚠ COMMON MISTAKES

- Resuming a continuation twice, or never: runtime warning or crash, or a hung task
- Forgetting `onTermination` cleanup: timers and observers leak
- Iterating one AsyncStream from two places: values are split, not copied

✓ TIPS & BEST PRACTICES

- Use `AsyncAlgorithms` (Apple package) for `debounce`, `merge`, `combineLatest`
- Use `withCheckedContinuation` in debug; only switch to unsafe after profiling
- Set a `bufferingPolicy` on high-volume streams

QUICK SUMMARY

- `AsyncSequence` + `for await`
- `AsyncStream` — build from events
- continuations — wrap callbacks

ARC Basics

AUTOMATIC REFERENCE COUNTING

Swift manages class instance memory with **ARC**: each strong reference adds to a count, and when the count drops to zero the instance is freed immediately and its `deinit` runs. There is no garbage collector and no pauses. Value types (structs, enums) aren't reference counted themselves, though class references stored inside them are.

NOTES

- Deallocation is deterministic: right when the last strong ref goes
- `deinit` is the place for cleanup (observers, files)
- Copy-on-write collections use ARC on their hidden buffers
- The optimizer removes many retain/release pairs

WATCHING ARC WORK

```
final class Session {
    let name: String
    init(name: String) { self.name = name; print("open \(name)") }
    deinit { print("close \(name)") }
}

var a: Session? = Session(name: "A")      // open A   (count 1)
var b = a                                 // count 2
a = nil                                   // count 1
b = nil                                   // close A  (count 0)

do {
    let temp = Session(name: "T")         // open T
    print(temp.name)
}                                         // close T (scope ended)

func make() -> Session { Session(name: "R") }
let kept = make()                         // returned: stays alive

isKnownUniquelyReferenced(&storage)       // copy-on-write check
```

CODE EXPLAINED

- → strong reference — keeps the object alive
- → `= nil` — releases a reference
- → `deinit` — runs at count 0
- → end of scope — locals released
- → `isKnownUniquelyReferenced` — used for COW

THE SWIFTY WAY

✗ calling a `close()` you might forget
✓ clean up in `deinit` (or `defer`)

COMING FROM…

- **Java** GC frees later; ARC frees immediately
- **Kotlin** JVM GC; Kotlin/Native uses a GC too
- **Python** Ref counting + cycle collector; Swift has no cycle collector

REFERENCE KINDS

KIND	KEEPS ALIVE?
`strong (default)`	Yes
`weak`	No, becomes nil
`unowned`	No, must outlive use
`unowned(unsafe)`	No, no checks

⚠ COMMON MISTAKES

- Expecting a cycle collector: cycles leak forever
- Doing heavy work or async calls in `deinit`
- Relying on `deinit` timing for objects captured by escaping closures

✓ TIPS & BEST PRACTICES

- Add a `deinit` print to confirm screens and models are freed
- Use Instruments › Allocations to watch object counts
- Prefer value types to avoid ARC traffic in hot paths

QUICK SUMMARY

- ARC — reference counting, no GC
- freed at count 0; `deinit` runs
- structs aren't counted

Sheet 105

Retain Cycles, weak & unowned

retain cycles · weak · unowned · [weak self] · Memory Graph

BREAKING REFERENCE CYCLES

If two objects hold **strong** references to each other, neither count reaches zero and both leak. Break the cycle by making one side `weak` (becomes `nil` when the target is freed; must be an optional `var`) or `unowned` (no nil, crashes if used after the target is freed). Closures stored by `self` are the most common cycle source.

NOTES

- Parent → child strong; child → parent `weak`
- Delegates are almost always `weak var delegate`
- Use `unowned` only when the target always outlives the reference
- Timers and NotificationCenter blocks retain their closures

WEAK, UNOWNED, CLOSURES

```swift
final class Owner {
    var pet: Pet?
    deinit { print("owner freed") }
}
final class Pet {
    weak var owner: Owner?               // weak breaks the cycle
    deinit { print("pet freed") }
}
var o: Owner? = Owner()
o!.pet = Pet(); o!.pet!.owner = o
o = nil                                  // owner freed, pet freed

protocol PlayerDelegate: AnyObject { func didFinish() }
final class Player { weak var delegate: PlayerDelegate? }

final class Customer {
    let card: CreditCard
    init() { card = CreditCard(); card.holder = self }
}
final class CreditCard { unowned var holder: Customer! }   // never outlives

final class ViewModel {
    var onUpdate: (() -> Void)?
    func bind() { onUpdate = { [weak self] in self?.refresh() } }
    func refresh() {}
}
```

CODE EXPLAINED

- → `weak var owner` — optional, auto-nil
- → `protocol …: AnyObject` — needed for weak delegates
- → `unowned` — non-optional, must outlive
- → `[weak self]` — closure doesn't retain self

THE SWIFTY WAY

✗ `var delegate: MyDelegate?` (strong)
✓ `weak var delegate: MyDelegate?` — delegates usually own you

COMING FROM…

- **Java** GC handles cycles; no weak needed in most code
- **Kotlin** Same as Java on JVM
- **Python** `weakref` ≈ weak; cycles are collected eventually there

FINDING LEAKS

TOOL	SHOWS
Debug Memory Graph	Live objects + cycles
Instruments › Leaks	Leaked allocations
deinit print	Whether an object is freed
Malloc Stack Logging	Where it was allocated

⚠ COMMON MISTAKES

- Strong delegate properties: classic view-controller leak
- `unowned self` in a network callback that can outlive the object: crash
- Timers retain their target, so `deinit` never runs to stop them: invalidate when the screen closes

✓ TIPS & BEST PRACTICES

- Make delegate protocols `AnyObject`-constrained so they can be weak
- Use the Memory Graph button in Xcode's debug bar after closing a screen
- Prefer `[weak self]` + `guard let self` over `unowned`

QUICK SUMMARY

- strong ↔ strong = leak
- `weak` (optional) / `unowned` (non-optional)
- `[weak self]` in stored closures

Ownership & Noncopyable Types

OWNERSHIP CONTROL

Swift 5.9 added tools for performance-critical code. A **noncopyable** type (`~Copyable`) can't be copied, only moved, so it models unique resources like file handles. Parameter modifiers `borrowing` and `consuming` state whether a function reads a value or takes it over, and `consume` ends a variable's lifetime explicitly.

NOTES

- Most code never needs these; they're for libraries and hot paths
- Noncopyable structs can have a `deinit`
- `consuming` methods can `discard self` to skip `deinit`
- Swift 6 supports noncopyable types in generics (`T: ~Copyable`)

NONCOPYABLE & OWNERSHIP

```
struct FileDescriptor: ~Copyable {
    let fd: Int32
    init(path: String) { fd = open(path, O_RDONLY) }
    borrowing func read() -> [UInt8] { /* uses fd */ [] }
    consuming func close() {
        Darwin.close(fd)
        discard self                        // skip deinit
    }
    deinit { Darwin.close(fd) }             // auto-close if not closed
}

let file = FileDescriptor(path: "/etc/hosts")
let bytes = file.read()                     // borrow
// let copy = file                          // error: can't copy
file.close()                                // consumed
// file.read()                              // error: used after consume

func log(_ msg: borrowing String) { print(msg) }   // read only
func store(_ s: consuming [Int]) { cache = s }     // takes ownership

var big = [Int](repeating: 0, count: 1_000_000)
store(consume big)                          // move, no copy
// big is no longer usable here
```

CODE EXPLAINED

- → `~Copyable` — move-only type
- → `borrowing` — read without taking ownership
- → `consuming` — take ownership
- → `discard self` — end without deinit
- → `consume x` — explicit move

THE SWIFTY WAY

✗ a class + manual `close()` for a unique handle
✓ a `~Copyable` struct with `deinit` — the compiler enforces single ownership

COMING FROM…

- **Rust** Move semantics, borrowing ≈ the same ideas
- **C++** `std::move` ≈ `consume`; unique_ptr ≈ ~Copyable
- **Kotlin** No equivalent

SWIFT VERSIONS

- 5.9 — `~Copyable`, `borrowing`, `consuming`, `consume`
- 6.0 — noncopyable generics

MODIFIERS

KEYWORD	MEANING
`~Copyable`	Type can't be copied
`borrowing`	Temporary read access
`consuming`	Callee owns the value
`inout`	Exclusive mutable access
`consume x`	End x's lifetime now

⚠ COMMON MISTAKES

- Using these everywhere "for speed": measure first
- Trying to put a `~Copyable` value in an Array on older toolchains
- Using a variable after `consume`: compile error

✓ TIPS & BEST PRACTICES

- Profile with Instruments before reaching for ownership modifiers
- Model unique resources (locks, handles) as `~Copyable` structs
- Read SE-0390 and SE-0377 for the full rules

QUICK SUMMARY

- `~Copyable` — move-only types
- `borrowing` / `consuming` parameters
- `consume` — explicit move

Quick Reference — Syntax

DECLARATIONS

SYNTAX	MEANING
`let x = 1`	Constant
`var y: Int = 2`	Variable + type
`typealias ID = String`	Alias
`let (a, b) = (1, 2)`	Destructure
`lazy var z = make()`	Deferred init
`static let k = 0`	Type property

OPTIONALS

SYNTAX	MEANING
`T?`	Value or nil
`if let x { }`	Unwrap in branch
`guard let x else {}`	Unwrap or exit
`a?.b`	Chain
`a ?? d`	Default
`a!`	Force (crash on nil)

OPERATORS

OP	MEANING
`a...b / a..<b`	Ranges
`&+ &- &*`	Wrapping math
`=== / !==`	Same instance
`c ? a : b`	Ternary
`is / as? / as!`	Type check/cast
`x ~= y`	Pattern match

CONTROL FLOW

SYNTAX	MEANING
`if a, let b {}`	Conditions
`let v = if c {1} else {2}`	If expression
`switch x { case 1...5: }`	Exhaustive match
`case let .ok(v) where v>0`	Pattern + guard
`for x in seq where p {}`	Filtered loop

FUNCTIONS & CLOSURES

SYNTAX	MEANING
`func f(_ x: Int) -> Int`	Unlabeled param
`func f(to y: Int)`	Label ≠ name
`func f(_ n: Int...)`	Variadic
`{ a, b in a < b }`	Closure
`{ $0 * 2 }`	Shorthand

TYPES

SYNTAX	MEANING
`struct S { }`	Value type
`final class C { }`	Reference type
`enum E { case a(Int) }`	Tagged union
`protocol P { func f() }`	Interface
`actor A { }`	Isolated state
`extension S: P { }`	Add conformance

GENERICS & PROTOCOLS

SYNTAX	MEANING
`func f<T: P>(_ x: T)`	Generic
`func f(_ x: some P)`	Opaque param
`[any P]`	Existential
`where T.Element == Int`	Constraint
`associatedtype Item`	Protocol generic
`@propertyWrapper`	Wrapper type

ERRORS & ASYNC

SYNTAX	MEANING
`func f() throws`	Can throw
`throws(MyErr)`	Typed throws
`do { try f() } catch {}`	Handle
`try? / try!`	Optional / crash
`async let x = f()`	Parallel child
`Task { await f() }`	Start async

PROPERTIES

SYNTAX	MEANING
`var x: Int { 2 * y }`	Computed
`var x = 0 { didSet {} }`	Observer
`private(set) var x`	Read-only outside
`@Clamped var x`	Property wrapper

ACCESS

LEVEL	VISIBLE TO
`open / public`	Other modules
`package`	Same package
`internal`	Same module (default)
`private`	Type + same file

MEMORY

SYNTAX	MEANING
`weak var d: D?`	No retain, auto-nil
`unowned let o: O`	No retain, no nil
`{ [weak self] in }`	Avoid cycles
`deinit { }`	Cleanup at release

⚠ COMMON MISTAKES

- `arr[0]` on empty arrays and `x!` on nil both crash
- `=` returns Void: `if x = y` won't compile
- Integer overflow traps; use `&+` to wrap

✓ TIPS & BEST PRACTICES

- Default to `let`, `struct`, `final class`, `some P`
- Prefer `guard` for early exits
- See the sheet numbers on the index for details on each row

QUICK SUMMARY

- `let` / `var`, optionals, ranges
- exhaustive `switch`, patterns
- struct · class · enum · protocol · actor

Quick Reference — Standard Library & Idioms

ARRAY

CALL	RESULT
`append / insert(_:at:)`	Add
`remove(at:) / popLast()`	Remove
`first / last`	Optional end
`sorted(by:) / sort()`	Copy / in place
`firstIndex(of:)`	Optional index
`removeAll(where:)`	Filter in place
`prefix(n) / dropFirst(n)`	Slices

DICTIONARY & SET

CALL	RESULT
`d[k]`	Optional value
`d[k, default: 0] += 1`	Count
`d[k] = nil`	Remove key
`mapValues { }`	Keep keys
`Dictionary(grouping:by:)`	Bucket
`s.insert / contains`	Set ops O(1)
`union / intersection`	Set algebra

SEQUENCE PIPELINE

CALL	DOES
`map / compactMap`	Transform
`filter`	Keep matches
`reduce(0, +)`	Fold
`contains(where:)`	Any match
`allSatisfy`	All match
`enumerated() / zip`	Index / pair
`.lazy`	Defer work

STRING

CALL	RESULT
`"\(x)"`	Interpolate
`s.count / isEmpty`	Length / empty
`hasPrefix / contains`	Search
`split(separator:)`	[Substring]
`replacing(_:with:)`	Replace
`trimmingCharacters(in:)`	Trim
`joined(separator:)`	Join

FOUNDATION

CALL	DOES
`Date.now.formatted()`	Display date
`Calendar.current.date(byAdding:…)`	Date math
`x.formatted(.percent)`	Format number
`URL.documentsDirectory`	App folder
`JSONDecoder().decode(T.self, from:)`	Parse JSON
`URLSession.shared.data(from:)`	HTTP GET

CONCURRENCY

CALL	DOES
`try await f()`	Call async
`async let a = f()`	Parallel
`withTaskGroup(of:)`	Dynamic parallel
`Task.checkCancellation()`	Stop if cancelled
`@MainActor`	UI thread
`actor X { }`	Safe shared state

IDIOMS

PREFER	OVER
`arr.isEmpty`	`arr.count == 0`
`if let x`	`if x != nil … x!`
`users.map(\.name)`	`map { $0.name }`
`(a, b) = (b, a)`	`temp variable`
`d[k, default: 0] += 1`	`nil check + insert`
`defer { cleanup() }`	`cleanup at each return`

⚠ COMMON MISTAKES

- `map` on a Dictionary returns an Array: use `mapValues`
- `reversed()` returns a view: wrap with `Array(...)`
- Non-2xx HTTP responses don't throw: check `statusCode`

✓ TIPS & BEST PRACTICES

- Use `.formatted()` for anything users read
- Add swift-collections for Deque, OrderedSet, Heap
- Prefer `async` APIs over completion handlers

QUICK SUMMARY

- collections: Array, Dictionary, Set
- pipelines: map / filter / reduce
- Foundation + concurrency one-liners